

國立臺北商業大學

資訊管理系

114' 資訊系統專案設計 系統手冊



組 別：第 114402 組

題 目：AI 底家

指導老師：蒯思齊老師

組 長：11146005 沈宛宣

組 員：11146012 邱鈺婷 11146018 趙明威

11146033 陳欣怡 11146039 金峻瑋

中 華 民 國 114 年 12 月 31 日

國立臺北商業大學專題課程作品

電子文件上網授權書

本授權書所授權之作品為授權人在 國立臺北商業大學 資訊管理(科/系/所)

114402組 114 學年度 第一學期 專題課程之成果報告作品。

專題題目：AI 底象

指導教授：簡思齊

☒ 本人 ☒ 團體著作人

☒ 同意 提供讀者基於個人非營利性質之線上檢索、閱覽、下載或列印。

茲同意將授權人擁有著作權之上列論文全文(含摘要)無償授權本人就讀學校(國立臺北商業大學)圖書館，不限地域、時間與次數，以微縮、光碟或其他各種數位化方式將上列作品收錄、重製與利用，並得將數位化之上列作品及其電子檔上載資料庫系統。於著作權法合理使用範圍內，讀者得進行線上檢索、閱覽、下載或列印。

專題課程成果報告作品全文上載網路公開之範圍及時間：

本校區域網路 ☒ 立即公開

☒ 自中華民國114年12月31日公開

校外網際網路 ☒ 立即公開

☒ 自中華民國114年12月31日公開

☐ 不同意 提供讀者基於個人非營利性質之線上檢索、閱覽、下載或列印。

授權人簽名 沈宛宜、邱金玲、趙明威、金峻璋、陳欣怡

中 華 民 國 114 年 12 月 31 日

目錄

第 1 章 前言	10
1-1 背景介紹	10
1-2 動機	10
1-3 系統目的與目標	12
1-4 預期成果	13
第 2 章 營運計畫	14
2-1 可行性分析	14
2-2 商業模式—Business model	18
2-3 市場分析—STP	19
2-4 競爭力分析-SWOT-TOWS	20
第 3 章 系統規格	21
3-1 系統架構	21
3-2 系統軟、硬體需求與技術平台	22
3-3 使用標準與工具	23
第 4 章 專案時程與組織分工	27
4-1 專案時程	27
4-2 專案組織與分工	28
4-3 上傳 GitHub 紀錄	31
第 5 章 需求模型	33
5-1 使用者需求	33
5-2 使用個案圖(Use case diagram)	34
5-3 使用個案描述	35
5-4 分析類別圖(Analysis class diagram)	39
第 6 章 設計模型	40
6-1 循序圖(Sequential diagram)或通訊圖(Communication diagram)	40
6-2 設計類別圖(Design class diagram)	46
第 7 章 實作模型	47
7-1 佈署圖(Deployment diagram)	47
7-2 套件圖(Package diagram)	47
7-3 元件圖(Component diagram)	51
7-4 狀態機(State machine)	51
第 8 章 資料庫設計	59
8-1 資料庫關聯表	59
8-2 表格及其 Meta data	60
第 9 章 程式	66
9-1 元件清單及其規格描述	66
9-2 其他附屬之各種元件	70

第 10 章 測試模型	99
10-1 測試計畫.....	99
10-2 測試個案與測試結果資料.....	100
第 11 章 操作手冊	104
11-1 系統元件.....	104
11-2 系統下載及安裝.....	104
第 12 章 使用畫面	110
第 13 章 感想	123
第 14 章 參考資料	126
附錄 會議記錄	128
附錄 初評之評審建議與意見	132
附錄 複評之評審建議與意見	134

圖目錄

圖 1-1 2020~2025 年台灣 65 歲以上人口比例變化.....	11
圖 1-2 2020~2025 年長者跌倒與住院推估圖.....	11
圖 2-1 說明-即時骨架與物件資料傳輸結構.....	14
圖 2-2 說明-只保留骨架不保留影像.....	16
圖 3-1 系統流程圖.....	22
圖 3-2 近 10 年全球 Android 與 IOS 作業系統占比比較圖.....	23
圖 4-1 專案時程甘特圖.....	27
圖 4-2 11146005 沈宛宣 上傳 GitHub 紀錄.....	31
圖 4-3 11146012 邱鈺婷 上傳 GitHub 紀錄.....	31
圖 4-4 11146018 趙明威 上傳 GitHub 紀錄.....	31
圖 4-5 11146033 陳欣怡 上傳 GitHub 紀錄.....	32
圖 4-6 11146039 金峻瑋 上傳 GitHub 紀錄.....	32
圖 4-7 GitHub 總上傳紀錄.....	32
圖 5-1 使用個案圖.....	34
圖 5-2 分析類別圖.....	39
圖 6-1 循序圖-註冊會員.....	40
圖 6-2 循序圖-登入.....	40
圖 6-3 循序圖-更改密碼.....	41
圖 6-4 循序圖-攝影機鏡頭傳送至伺服器.....	42
圖 6-5 循序圖-查詢跌倒列表.....	43
圖 6-6 循序圖-查詢互動報告.....	43
圖 6-7 循序圖-查詢緊急連絡人.....	44
圖 6-8 循序圖-查看睡眠狀態列表.....	44
圖 6-9 循序圖-查看久坐狀態列表.....	45
圖 6-10 循序圖-FB 授權流程.....	45
圖 6-11 設計類別圖.....	46
圖 7-1 佈署圖.....	47
圖 7-2 套件圖-資料視覺化與分析.....	47
圖 7-3 套件圖-資料存入 SQL.....	48
圖 7-4 套件圖-AI 跌倒偵測.....	48
圖 7-5 套件圖-WebAPI 服務.....	49
圖 7-6 套件圖-LINE 整合通知.....	49
圖 7-7 套件圖-N8N 自動化流程與推理模組.....	50
圖 7-8 元件圖.....	51
圖 7-9 狀態機-登入.....	52
圖 7-10 狀態機-帳號註冊.....	52
圖 7-11 狀態機-系統偵測端.....	53

圖 7-12 狀態機-更改密碼	54
圖 7-13 狀態機-狀態清單查看	55
圖 7-14 狀態機-語音互動紀錄	56
圖 7-15 狀態機-緊急連絡人新增及刪除	57
圖 7-16 狀態機-Facebook 授權.....	58
圖 8-1 MySQL 資料庫 ER 圖	59
圖 11-1 【AI 底家】QR code.....	104
▲圖 11-2 步驟一	105
▲圖 11-3 步驟二.....	106
▲圖 11-4 步驟三.....	107
▲圖 11-5 步驟四.....	108
▲圖 11-6 步驟五.....	109
圖 12-1 系統主要操作流程狀態轉移圖-1	110
圖 12-2 系統主要操作流程狀態轉移圖-2	111
▲圖 12-3 登入頁面	112
▲圖 12-4 註冊頁面.....	113
▲圖 12-5 登入頁面.....	114
▲圖 12-6 編輯頁面	115
▲圖 12-7 Linebot	115
▲圖 12-8 FB 授權	116
▲圖 12-9 修改密碼.....	117
▲圖 12-10 刪除密碼.....	117
▲圖 12-11 選擇被照護者.....	118
▲圖 12-12 狀態清單	119
▲圖 12-13 狀態清單頁面(跌倒、睡眠、互動、久坐).....	120
▲圖 12-14 語音互動紀錄.....	121
▲圖 12-15 語音互動紀錄.....	122
▲圖 12-16 新增緊急聯絡人.....	122

表目錄

表 1-3-1 市售防跌倒系統及產品與本系統之功能比較表	13
表 2-2-1 商業模式－Business model	18
表 2-4-1 競爭力分析-SWOT-TOWS	20
表 3-2-1 Android 與 IOS 開發平台比較表	22
表 3-2-2 系統硬體需求-手機	23
表 3-3-1 使用標準與工具表	26
表 4-2-1 專案組織與分工表	28
表 4-2-2 專案成果內容與共貢獻度表	30
表 5-1-1 使用者需求表	33
表 5-3-1 查看跌倒狀態列表	35
表 5-3-2 即時突發異常狀況通知	35
表 5-3-3 異常統計報表通知	36
表 5-3-4 查看互動報告	36
表 5-3-5 查看睡眠紀錄	36
表 5-3-6 查看久坐紀錄	37
表 5-3-7 查看語音互動紀錄	37
表 5-3-8 撥打緊急聯絡人	37
表 5-3-9 查看報告圖表	38
表 5-3-10 FB 授權-播報子女貼文	38
表 8-2-1 資料表	60
表 8-2-2 資料表描述-01 使用者資料	60
表 8-2-3 資料表描述-02 角色資料	61
表 8-2-4 資料表描述-03 通知聯絡資料	61
表 8-2-5 資料表描述-04 line 帳號綁定	61
表 8-2-6 資料表描述-05 通知偏好設定	62
表 8-2-7 資料表描述-06 社交平台帳號資訊	62
表 8-2-8 資料表描述-07 照護者社交貼文	63
表 8-2-9 資料表描述-08 跌倒事件紀錄	63
表 8-2-10 資料表描述-09 坐下行為紀錄	63
表 8-2-11 資料表描述-10 睡眠紀錄	64
表 8-2-12 資料表描述-11 AI 互動紀錄	64
表 8-2-13 資料表描述-12 AI 互動報告	65
表 9-1-1 元件清單與規格描述表(後端)-資料庫(MySQL)	66
表 9-1-2 元件清單與規格描述表(後端)- API (Routes)	66
表 9-1-3 元件清單與規格描述表(後端)- Service (業務層)	66
表 9-1-4 元件清單與規格描述表(後端)- 資料層 (DAO / DB)	67
表 9-1-5 元件清單與規格描述表(後端)- 即時推論與 WS (核心 Utils)	68

表 9-1-6 元件清單與規格描述表(後端)- 通用工具 (Utils)	68
表 9-1-7 元件清單與規格描述表 (模型資源)	68
表 9-1-8 元件清單與規格描述表 (啟動與環境)	69
表 9-1-9 訓練與資料前處理 (離線)	69
表 9-1-10 Json 檔案(後端)-邊緣運算	69
表 9-1-11 元件清單與規格描述表(前端).....	69
表 9-2-1 n8n 流程-處理 Facebook OAuth_Authorization 的回呼	70
表 9-2-2 n8n 流程- FB 社群貼文抓取與處理貼文	71
表 9-2-3 n8n 流程- 分析互動紀錄產生週報	73
表 9-2-4 n8n 流程- 將收到的 json 依據格式不同執行不同子工作流	75
表 9-2-5 n8n 流程- 自然語言理解	76
表 9-2-6 將收到的 json 依據格式不同執行不同子工作流	80
表 9-2-7 FB 社群貼文抓取與處理貼文	82
表 9-2-8 虛擬碼—db.py	82
表 9-2-9 虛擬碼—ws_connection_manager.py	85
表 9-2-10 虛擬碼—ws_message_dispatcher.py	86
表 9-2-11 虛擬碼—kalman_filter.py	87
表 9-2-12 虛擬碼—cnn_lstm_utils.py	88
表 9-2-13 虛擬碼—response_util.py	90
表 9-2-14 虛擬碼—notify_prefs_service.py	91
表 9-2-15 虛擬碼—notify_line_service.py	92
表 9-2-16 虛擬碼—weekly_report_push_service.py	93
表 9-2-17 虛擬碼—line_binding_service.py	95
表 9-2-18 虛擬碼—binary_cnn_lstm_trainer.py	96
表 10-2-1 功能測試-會員註冊	100
表 10-2-2 功能測試-會員登入	100
表 10-2-3 功能測試-更改密碼	100
表 10-2-4 功能測試-刪除帳號	100
表 10-2-5 功能測試-編輯個人資料 (含長輩資料)	101
表 10-2-6 功能測試-查看跌倒列表	101
表 10-2-7 功能測試-查看長者與系統互動對話紀錄	101
表 10-2-8 功能測試-查看緊急聯絡人	101
表 10-2-9 功能測試-新增緊急聯絡人	101
表 10-2-10 功能測試-刪除緊急聯絡人	102
表 10-2-11 功能測試-查看活動量	102
表 10-2-12 功能測試-查看睡眠列表	102
表 10-2-13 功能測試-查看語音互動報告	102
表 10-2-14 功能測試-查看久坐資訊	103

表 11-1-1 系統安裝元件資訊	104
-------------------------	-----

第1章 前言

1-1 背景介紹

隨著全球高齡化加劇，長者的居家照護成為社會關注的焦點。跌倒是高齡者最常見且造成後果嚴重的意外之一，但實務上長者對穿戴式裝置的接受度有限（不想戴、忘記充電、配戴不適），而傳統居家監控又常需要多設備與專業安裝，導入成本高、維護複雜。更重要的是，單純的安全監測往往忽略了情感陪伴，對長者的心理健康與長期使用意願影響甚大。因此，我們開發了一套結合 AI 影像分析與溫暖語音互動的智慧照護系統。超越傳統的被動式監測，融入主動式情感陪伴的核心理念。讓一支既有的 Android 手機（鏡頭+麥克風）就能提供姿態與行為分析、即時關懷提醒與有溫度的語音互動。系統結合電腦視覺與多模態 AI，並透過 LINE 生態與自動化工作流（n8n）整合新聞與家屬社群摘要，用於日常播報與互動；為兼顧隱私與可接受度，系統目前不保存事件短片、以結構化資料為主。目標是兼顧安全與陪伴，提升長者的安全性、獨立性與生活幸福感，也有效減輕照護者與子女的負擔。

1-2 動機

截至 2025 年 1 月，臺灣 65 歲以上人口已超過 450 萬人，佔總人口超過 19%，正式進入超高齡社會。高齡長者除了面臨慢性疾病困擾，居家跌倒更是常見而嚴重的問題。根據國民健康署調查，每 6 位長者中就有 1 位曾經跌倒，且每 12 位長者中就有 1 位因此就醫，跌倒已成為長者事故傷害死亡的第二大原因。跌倒是造成 65 歲以上長者受傷住院和急診的最主要原因，其發生率亦隨年齡增加而隨之提高。特別是獨居長者，若無即時通報與協助，容易延誤就醫並引發嚴重後果。

目前市面上多數照護輔具仍以穿戴式設備為主，如手環、綁帶等，雖可進行健康監測，卻常因配戴不適、操作繁瑣，導致長輩抗拒使用，甚至影響照護成效。本系統採用非穿戴式設計，結合 AI 姿勢辨識與生成式語音互動技術，達到即時辨識，更透過語音提醒與互動陪伴，如主動噓寒問暖、說明子女近況、聊天互動等，提升長輩接受度與使用意願，並協助家屬即時掌握長輩狀況，進而提升照護品質與安全。

圖 1-2-1 呈現 2020 至 2025 年台灣 65 歲以上人口比例的變化趨勢。可見台灣高齡人口比例逐年上升，從 2020 年的 15.37% 增加至 2025 年預估的 19.27%，顯示出高齡化社會持續加劇，對長者照護服務的需求亦隨之提升。

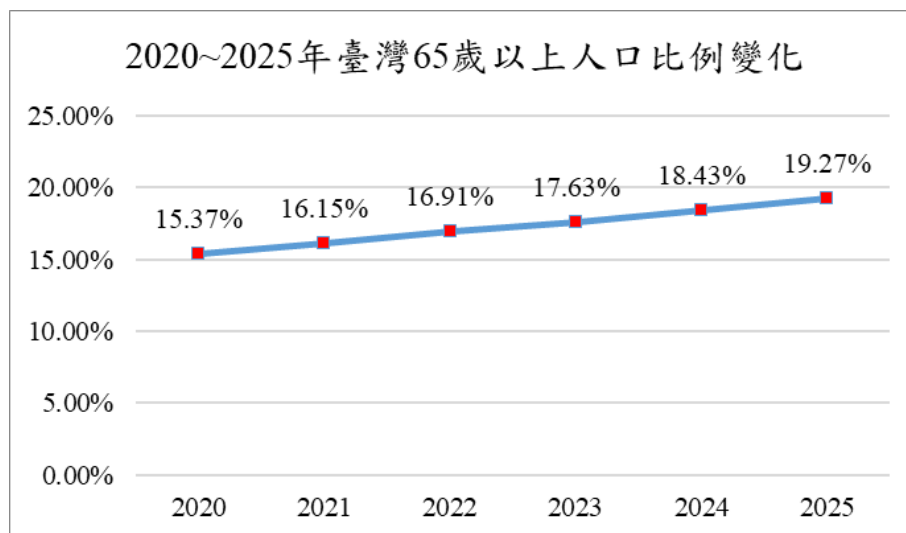


圖 1-1 2020~2025 年台灣 65 歲以上人口比例變化

根據圖 1-2-2 顯示 2020 至 2025 年長者跌倒與住院人數的推估變化。從圖中可見，隨著年份增加，跌倒人數與因跌倒而住院的人數皆呈現穩定上升趨勢，顯示跌倒已成為高齡者健康風險中不可忽視的議題，需積極介入預防與即時應對措施。

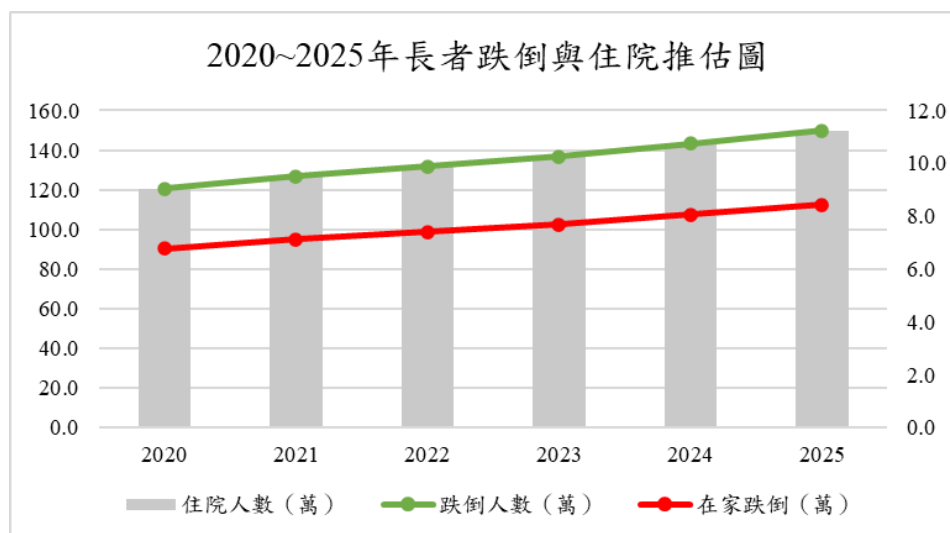


圖 1-2 2020~2025 年長者跌倒與住院推估圖

1-3 系統目的與目標

隨著年齡增長，許多長者會因肌少症、器官退化等因素，逐漸面臨行動力下降與跌倒風險的增加。傳統的防跌倒系統多數依賴穿戴式設備，如手環或智慧手錶，這些裝置雖可進行健康監測，但長時間佩戴會增加身體負擔，並且容易引發長者的抗拒與排斥感。這些問題使得現有的系統無法長期被長者接受，進而降低了其使用率與效果。

本系統旨在建立一個**高效且可擴展的智慧照護平台**，致力於提升長者的居家安全與生活品質。與傳統的監控系統不同，我們的系統結合了先進的**非穿戴式 AI 影像辨識技術**與**生成式語音技術**，並無需額外穿戴設備即可實現高效的跌倒風險預測與主動式智能陪伴。

我們的系統設計遵循以下核心目標：

- **構建高效且可擴展的系統架構**：我們的系統能夠在**有限的邊緣計算資源**下實現長者行為的**精準辨識與智能陪伴**，有效克服硬體資源的限制，確保系統的高效運行和可擴展性。
- **實現多模態數據的深度融合與推論**：系統能夠有效結合來自影像分析的數字化人體骨架、環境物件、時序變化，與來自語音識別的文本信息，通過**大型語言模型（LLM）**進行綜合語義理解與決策。這種深度融合的數據處理方法，使得系統能夠進行**更準確的行為判斷**，並提供個性化的反應。
- **確保異常行為的即時響應**：當發現如跌倒等緊急情況時，我們的系統能夠在極短時間內完成**精準判斷**，並透過多種渠道發出即時預警，保障長者的安全，並迅速通知照護者或家人。
- **提供個性化、有溫度的智慧陪伴**：本系統不僅具備基本的跌倒偵測功能，還融入了**情感陪伴**的核心理念。通過**自然親切的語音提醒**，系統能主動向長者問候，提供生活小提醒、天氣播報，甚至能根據子女的社交平台動態播放家人的貼文，減少長者的孤獨感，提升他們的生活質量與心理福祉。
- **建立自動化與可維護的工作流**：利用 **n8n 平台**協調複雜的數據流與異構服務集成，我們不僅簡化了系統的部署過程，還實現了系統的可維護性。這使得未來的升級與擴展變得更加高效，並能快速應對照護需求的變化。

從表 1-3-1 可看出，相較於其他市售防跌倒系統，『AI 底家』無需配戴裝置，操作更簡便，能減少長者因穿戴不適而產生的抗拒感；同時具備彈性語音互動功能，並導入情感式陪伴設計，如聊天互動、主動提醒等，讓系統在偵測功能之外，也能融入長者日常生活，提供更友善且貼近人心的使用體驗。

表 1-3-1 市售防跌倒系統及產品與本系統之功能比較表

項目 系統	AI 底家	Osmile ED1000 獨居 老人照顧系統	HoneyCare 安 照小護士	9200N-WiFi 老人跌倒偵測 感測器	技嘉-智慧感 知跌倒偵測系 統
穿戴裝置	X	V	V	X	X
語音功能	V	X	固定語音提醒	X	X
情感互動	V	X	X	X	X
長者接受 度	非穿戴+ 有溫度 語音	易忘記配戴	穿戴負擔偏高	中等	對鏡頭有排斥 感

1-4 預期成果

本系統希望能在長輩發生跌到時，於第一時間發現並通知，同時提供**有溫度的語音陪伴**，以縮短從「發生」到「被看見、被協助」的時間，並在日常中建立穩定的陪伴互動。透過 AI 影像辨識技術實現即時行為分析，，搭配「只送骨架」等資料最小化策略，以兼顧判讀需要與隱私保護。偵測模型採二階段設計：先以二元模型快速判定「跌倒/非跌倒」，在維持靈敏度的前提下降低誤報造成的干擾。此外，系統結合 n8n 聚合的每日新聞與家屬社群摘要進行早晨播報，讓安全提醒與日常陪伴並行，提升接受度與使用意願。系統免配戴、無操作負擔，由照護者遠端管理與掌握，即便家人不在身邊，也能讓長者感受到被關心、被看見，實現科技與情感兼具的智慧照護體驗。

第2章 營運計畫

2-1 可行性分析

整體挑戰與策略對應：在開發本智慧照護系統過程中，面臨多項技術與市場挑戰，同時也帶來對應的解決策略與機會

- 硬體資源限制：採「非穿戴、輕量化」策略，前端只用現成 Android 手機；以骨架與人框資料取代整張影像參與判斷，並用二階段偵測（先判「跌倒／非跌倒」，再做基本動作理解）降低計算量。
- 影片處理即時性：系統以 WebSocket 持續傳輸骨架與偵測框的 JSON 資料，以提升即時性並降低頻寬消耗。目前資料傳輸頻率為 Pose 5 fps（每秒 5 次）、Detect 1 fps（每秒 1 次），其中 Pose 每傳送四次後，第五次會與 Detect 一起傳送，以保持時序同步。伺服器採用時序判斷機制（可調整觸發幀數），在偵測到異常事件時，會即時切出前後數秒的短片段進行推播。此外，系統具備弱網自動重連與降級機制，必要時會啟用本地暫存作為備援，確保資料不遺失與偵測不中斷。



圖 2-1 說明-即時骨架與物件資料傳輸結構

- 行為判斷複雜與上下文依賴：跌倒與躺臥、蹲下在外觀上相似，且多人或遮擋會干擾判斷；系統以連續幀的時序特徵與追蹤穩定化維持一致性，先由二元偵測過濾出疑似事件，再進行基本動作理解輔助後續判讀與報表，並逐步納入環境線索（如地面、床/椅相對位置）以提升區分度。
- 如何提供「有溫度」的陪伴：透過 n8n 聚合天氣與家屬貼文摘要進行晨間播報與關懷提示；語音互動僅在合適時機啟用（如每日固定時間或事件後的安撫提醒），訊息語氣維持友善、簡潔，家屬可自訂額外語音提醒(如：吃藥提醒等等)，提升系統接受度。
- 系統流程協調與擴充：以 n8n 作為流程中台（含 Facebook 授權、貼文抓取與摘要、排程與重試），前後端以清晰 API/WS 協定交換資料，設定集中管理；部署採雲端容器化，模組間低耦合，未來可平滑擴充睡眠小結、多人監看等功能。
- 隱私與倫理：涉及影像、語音、家庭社群資訊。遵循資料最小化原則，預設只傳骨架、不長期留原始影像，僅保存事件短片段；推播採最小必要資訊；家屬社群內容在明確授權下取得，遵循法規要求。

1. 技術可行性

AI 影像辨識技術：本系統以 MediaPipe/OpenPose 擷取人物骨架與人框，搭配「連續多幀的時序分析」完成日常行為的即時辨識；資料傳輸採骨架/人框 JSON（非原始影像），有效降低頻寬與裝置端壓力。判斷流程先快速區分「跌倒/非跌倒」，在非跌倒情境下再做基本動作理解以支援後續判讀與報表。

語音生成技術：以 LLM（Ollama + Gemma 3）進行內容選擇與語氣調整，並搭配 TTS 產生友善的語音播報；透過 n8n 聚合天氣與家屬社群貼文，形成晨間播報與關懷提醒的固定節奏。語音互動依時段與情境啟用，可由家屬設定禁言時段與內容偏好，在不打擾的前提下維持日常陪伴。

市場潛力：高齡化推動在宅照護需求增長，對「低安裝成本＋非穿戴＋可陪伴」的方案接受度高。切入上以 B2C 試點建立口碑與資料，再導入 B2B 的長照機構、社區據點、保險與居家照護服務合作（作為加值服務），並配合地方政府或社福單位的補助與專案採購。由於不需客製硬體與佈線，導入週期短、可快速擴張。

3. 法律可行性

數據隱私保護：以臺灣《個人資料保護法（PDPA）》為基礎，視資料跨境情況對應 GDPR/CCPA 要求。落實「目的特定、資料最小化」原則：預設僅傳輸與運算骨架/框線資料，不保存原始影像；資料傳輸採 TLS 加密，雲端儲存加密，金鑰與機密參數以環境變數管理；安裝與使用前取得明確告知與同意（含同住者/照護者），App 內提供隱私設定與一鍵暫停錄影/錄音。

智慧財產權保護：第三方模型、程式庫與工具（如影像/語音/工作流與雲端套件）均依其授權條款合法使用並保留標示；平台 API（如 LINE、Facebook Graph）遵循平台政策與最小權限原則，社群內容僅於授權目的下使用，不作二次商業利用或模型訓練。

符合法規要求：本系統定位為家用輔助照護與安全告警工具，不做醫療診斷/治療之宣稱，避免落入醫療器材監管範疇；若未來擬提供「醫療用途/風險預測」等功能，需評估 TFDA 醫療器材分類並依據 ISO 13485、IEC 62304、ISO 14971 等規範建立品質與風險管理體系，另行辦理註冊。B2C 銷售遵循《消費者保護法》（遠距交易資訊揭露、七日猶豫期/退費機制）；服務條款與隱私政策清楚揭露資料處理、保留、權利與責任限制。

攝影/錄影告知與場域合規：家中/機構安裝時張貼攝影/錄音告示，App 首次啟用顯示隱私說明與同意介面；提供「只骨架模式」與錄音開關、遮罩/隱私區域設定，並可設定禁錄時段或臨時暫停；推播內容以最小必要資訊呈現，避免過度揭露。

4. 操作可行性

本系統採用非穿戴式設計，系統安裝於家中任一手機後即可自動執行姿勢辨識與行為監測，長者無需操作裝置或學習使用介面，降低系統介入造成的不適感與排斥感。所有功能皆由照護者透過遠端操作與管理，前端 Android App 可提供事件通知、影片回放與語音互動紀錄等資訊，後端亦具備管理介面可查詢跌倒事件與活動量統計，使用流程簡單直覺，具備良好的使用體驗。整體操作流程符合實際照護情境需求，長者與照護者之間的資訊互通順暢，操作上具有高度可行性與實務應用價值。

2-2 商業模式—Business model

表 2-2-1 商業模式—Business model

關鍵合作夥伴 ● 家屬與照護者 ● 長照機構 ● 社福單位 ● 社區關懷協會(社區據點)	關鍵活動 ● 二階段判斷調校 ● 語音互動優化 ● 事件片段管理 ● LINE 綁定與推播 ● 用戶管理維護 關鍵資源 ● 跌倒辨識 AI ● 語音技術 ● 前後端平台 ● 資料庫 ● Android App、後端與 n8n 流程、LINE 模組 ● 雲端基礎(運算、儲存、備份)	價值主張 ● 第一時間通知 ● 提升長者接受度高 ● 隱私友善 ● 有溫度的語音提醒互動，提升陪伴感與安心感 ● 照護者可遠端即時掌握	顧客關係 ● 會員制 ● 客服服務 通路 ● Android Play 商店 ● 合作社區、照護機構 ● LINE 官方帳號導流	目標客戶 ● 獨居 ● 子女無法常陪伴的家庭 ● 長照機構、小型照護院所、社區照護據點 ● 行動不便或跌倒高風險長者之家戶
成本結構 ● 系統開發與維護費 ● 雲端平台 ● 行銷推廣、社區合作、客服支援成本	收益流 ● 訂閱制：家庭版(199/299/月)、機構版(起149/長者/月) ● 照護者席次加購：NT\$59／席／月(家庭)；大量≥50 席 NT\$25／席／月(機構)			

根據表 2-2-1「商業模式—Business model」，本系統以子女照護者為主要客群，並同步面向長照／社福機構做方案化導入；透過非穿戴、低門檻的手機部署，提供「第一時間通知」與「日常語音陪伴」的核心價值。家屬可在 App 遠端管理長者裝置與通知偏好，並透過 LINE 推播即時接收異常事件；日常則由 n8n 聚合天氣與家屬貼文摘要進行晨間播報，強化陪伴感。商業模式以訂閱制為主（家庭／機構分級），通路以 Google Play、LINE 官方帳號與社區照護據點、合作機構共同推廣，逐步擴大滲透。全程遵循資料最小化與隱私合規（預設只傳骨架、僅留事件短片、授權存取社群內容），在維持信任與接受度的同時，建立可持續、可擴充的長期服務關係。

2-3 市場分析－STP

1. 市場區分 (Segmentation)

- 高齡者家庭：特別是那些希望提高長者居家安全和管理健康的家庭。隨著老齡化社會的發展，許多家庭希望通過智能系統來監控長者的日常生活，防止跌倒等事故發生。
- 獨居長者：未與家人同住的長者，尤其是居住於都市中、無法獲得即時照顧者。這些長者對智慧居家照護系統的需求更為迫切，因為他們更需要即時的安全監控與情感陪伴。
- 社區關懷機構與長期照護機構：這些機構需要監控長者的居家生活，並且提供緊急反應系統，協助提供專業照護服務。社區關懷機構可以合作並運用本系統來提高長者的生活質量，特別是在獨居長者的居家照護方面。

2. 目標市場 (Targeting)

- 主要目標市場：獨居長者，尤其是那些無法得到子女直接照顧的高齡者。這部分市場對智能居家照護系統的需求較高，因為他們需要更多的遠程監控和情感陪伴功能來保證居家安全。
- 次要目標市場：高齡者家庭，特別是關注長者安全健康的家庭。這些家庭希望能夠通過系統來提升居家照護的效率，減少家屬的照護負擔。
- 合作市場：與社區關懷機構與長期照護機構合作，這些機構對智能健康管理系統的需求較大，能夠利用本系統來協助監控長者的健康與安全狀況，並進行及時干預。

3. 產品定位 (Positioning)

- 即時通報與非穿戴式裝置：以「第一時間通知」為核心，一支現有的Android手機即可部署；偵測到跌倒事件立即透過LINE推播給家屬。預設只傳骨架圖，遠端即可完成管理與設定，降低導入與使用門檻。
- 情感陪伴功能：系統會根據長者的日常作息、健康狀況和社交動態生成個性化的語音內容，進行語音提醒，讓長者感受到來自家人或社區的關心，減少孤單感，增強對系統的信任。
- 與長期照護機構的合作：本系統不僅服務於家庭環境，還能與社區關懷機構及長期照護機構合作，提供專業照護服務，並能進行多長者的集中監控與管理，進一步擴大市場的影響力。

2-4 競爭力分析-SWOT-TOWS

表 2-4-1 從內外部因素探討系統的優勢、劣勢、機會與威脅。其中強調本系統結合 AI 影像辨識與生成式語音技術具備創新優勢，能提升互動性與親切感；但也面臨如語音辨識準確度、網路穩定性等技術挑戰，需在開發與應用上持續優化。

表 2-4-1 競爭力分析-SWOT-TOWS

內部 外部	優勢(Strength)	劣勢(Weakness)
機會 (Opportunity)	1.運用骨架偵測（MediaPipe／OpenPose 等）與連續幀時序分析，第一時間辨識疑似跌倒並通報。 2.非穿戴、低門檻與 LINE 推播，符合在宅照護普及趨勢，家屬導入意願高。 3.活動量紀錄，供照護者方便遠端了解長者一天活動量。 4.日常語音陪伴（n8n 聚合天氣+家屬貼文摘要）提升長者接受度與黏著。	1. 初期系統需持續優化跌倒辨識與語音內容生成精準度，增加開發成本。。 2. 長者對鏡頭存疑，但透過有溫度的語音互動，可逐步建立信任、降低排斥感，加強隱私告知、同意與告示溝通。
威脅(Threat)	1. 市場競爭激烈，競爭者跟進速度快。 2. 使用生成式語音與 AI 模型會依賴第三方 API 與雲端資源，成本受供應端變動影響。	1. 系統依賴數據處理，若出現偏差會影響效果。 2. 系統運行需穩定的網路連接，如網路不穩定則有一定的影響。

第3章 系統規格

3-1 系統架構

「AI 底家」主要設計目標為提供高齡者智慧居家照護服務，結合非穿戴、第一時間通知＋日常陪伴為核心，結合電腦視覺行為辨識與語音播報，提供高齡者在宅照護的即時告警與溫和關懷。整體架構分為三個部分：影像感知與模型推論、伺服器後端與流程中台、行動端使用介面，並部署於 Azure 雲端以便遠端管理與擴充。

1. 影像感知與模型推論端：由 Android 裝置固定擺放並持續供電進行拍攝，透過 MediaPipe Pose 擷取骨架關鍵點，先以「無人不推論」機制過濾畫面，僅在偵測到有人體活動時才進入辨識流程。資料以 WebSocket 傳送骨架／人框的結構化 JSON 至伺服器，採 CNN-LSTM 進行時序判斷：先快速區分「跌倒／非跌倒」，在非跌倒情境下再做基本動作理解以輔助報表與回看。觸發疑似事件時，自動切出前後數秒短片段以供家屬判讀，兼顧即時性與隱私。
2. 伺服器後端服務：後端以 Flask 提供 HTTP API 與 WebSocket 服務，整合 OpenCV 與 TensorFlow/Keras 進行推論，並將事件、互動紀錄與片段索引存入 MySQL。通知由 LINE Bot 推播至照護者；日常陪伴內容則由 n8n 聚合（例如天氣與家屬社群貼文摘要）後交由 TTS 生成語音播放。系統部署於 Azure VM，採容器化與設定集中管理，支援擴充與維運；全程遵循資料最小化原則（預設只傳骨架、不長期保存原始影像，僅保留事件短片段）。
3. 行動端使用介面：以 Android Studio 開發，提供「事件列表」、「緊急聯絡人管理」、「語音互動紀錄」與「活動量統計」等功能。照護者可遠端管理裝置與通知偏好，並透過 APP 即時接收告警與統計摘要，快速掌握照護情況。

本系統透過「只傳骨架」的設計降低頻寬與隱私風險；以二階段時序判斷提升偵測穩定度；並結合 LINE 推播與日常播報，在安全與陪伴之間取得平衡，實際提升事件被看見的速度與照護效率。讓通知不再只是冷冰冰的警報，而是搭配有溫度的語音關懷或文字提醒，提升長者接受度與系統親和力。以下為本系統流程圖示意：

圖 3-1-1 為系統流程圖，手機端啟動後，由 Android 相機持續擷取畫面與必要音訊；在感知處理階段，以 YOLOv8／MediaPipe 萃取人物與姿態資訊，語音則由 ASR 轉為文字。上述資料被結構化為 JSON 並送入 n8n 流程中台；由「JSON 分流器」將資料分派至兩條路徑——其一進入 Smart Alert Decision 進行連續幀的時序判斷，產出「即時警報／語音提醒／起床統計」等決策；其二先經 Prompt Construction 組裝情境欄位，再交由 LLM（Gemma3）產生指令（如是否通知、是否查詢／寫入資料、是否做文字轉語音），必要時可透過 Web Search API 或從子女社群內容取得素材並進行摘要。最終，結果由 Output & Interaction 模組寫入資料庫並呈現在 App 端，必要時觸發 LINE 通知照護者，或透過 TTS 於手機端播放語音，完成「偵測 → 決策 → 呈現/通知」的閉環；整體流程以 JSON 傳輸為主，兼顧即時性與資料最小化。

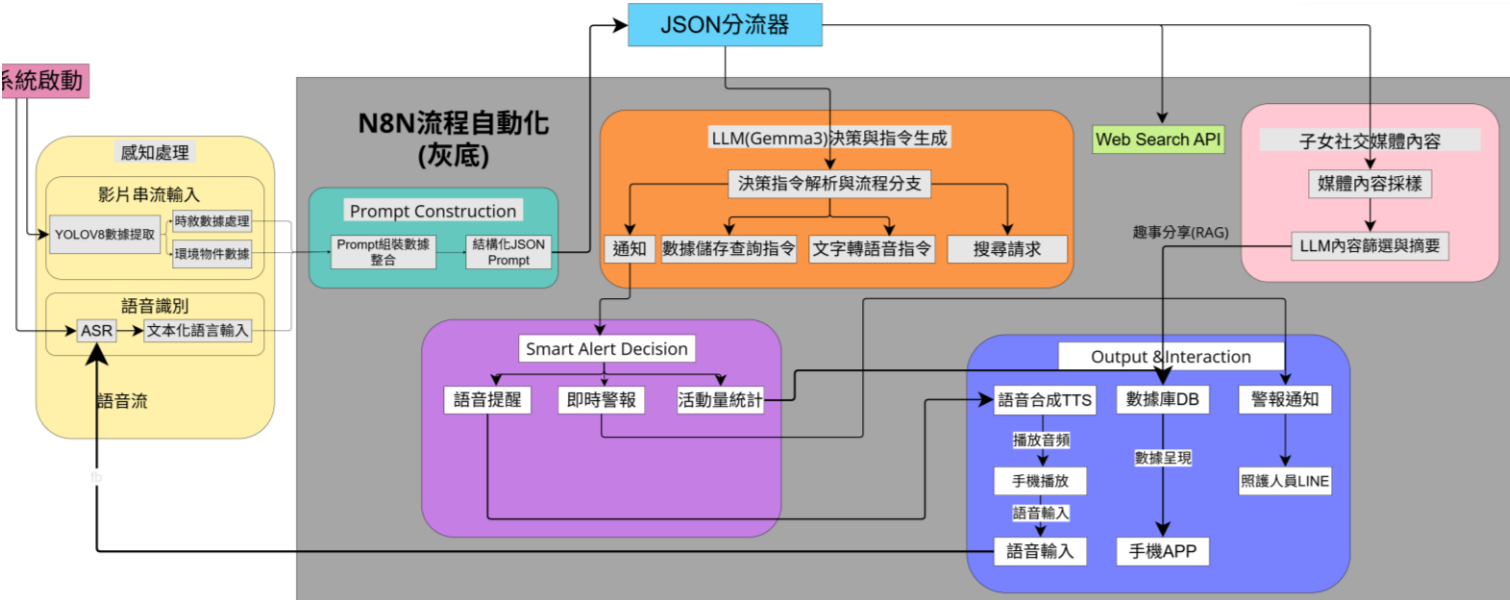


圖 3-1 系統流程圖

3-2 系統軟、硬體需求與技術平台

表 3-2-1 Android 與 IOS 開發平台比較表

比較項目	Android	ios
開發成本	不須上架費	年費 USD\$99
部署測試	可直接打包 APK 實機測試	測試需經過 TestFlight 或開發者帳號
開發資源與生態	支援較多開源工具	較多封閉架構

本系統選擇以 Android 作為開發平台，主要考量在於其開源、免費、支援 AI 工具整合、部署彈性高等多項優勢，特別適合學術專題開發與實地部署測試。相較於 IOS 平台需透過 macOS 系統與開發者帳號進行編譯與上架，Android 平台不僅能於 Windows 系統下順利開發，且能直接產出 APK 於裝置端測試，降低操作門檻與開發成本，提升實作效率與展示靈活性。

根據圖 3-2-1 近十年全球作 Android 與 IOS 作業系統占比比較圖顯示，Android 系統自 2015 年至今，維持領先優勢，其市佔率長年穩定在 65% 以上。相比之下，IOS 系統市佔率雖逐年略升，但整體仍處於次位。此趨勢顯示 Android 系統在全球具有更高的普及率與使用彈性，特別是在裝置多樣化、價格親民與開放性平台支援等面向。考量本專題開發需求及未來應用場域，本組選擇以 Android 為主要開發平台，搭配免費開源工具及無需額外上架費的特性，不僅符合學生開發條件，也有助於降低實測成本並提升應用推廣可行性。

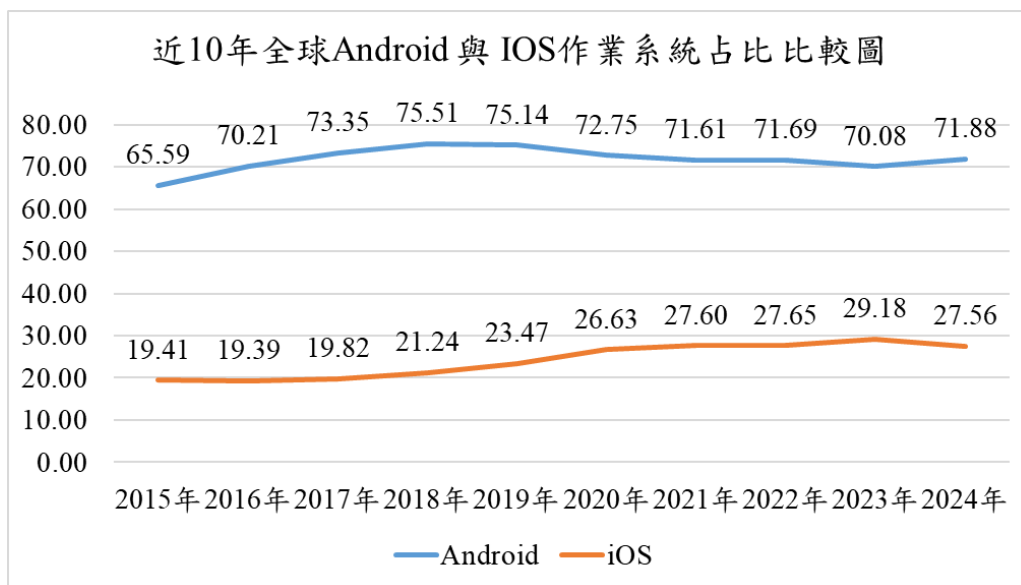


圖 3-2 近 10 年全球 Android 與 IOS 作業系統占比比較圖

考量到智慧照護系統需長時間穩定運作並支援影像處理與即時通知功能，本系統選擇以 Android 系統為主要平台，並將裝置需求訂為 Android 10.0 以上版本，以確保相容性與效能表現。為因應遠端通訊與即時通知機制，系統亦需支援穩定的網路環境，因此我們設備將支援 Wi-Fi、4G 與 5G 網路，使用者可依照實際場域彈性連線。這樣的配置將使本系統在居家與機構應用中，皆能提供穩定、順暢的資料傳輸體驗，提升整體照護效率與使用便利性。

表 3-2-2 系統硬體需求-手機

系統硬體需求-手機	
手機版本	Android 10.0 以上
網路需求	Wi-Fi/4G/5G

3-3 使用標準與工具

以下列出了本組所選用的開發工具與技術，並說明了每項的特點與選擇理由：

- Python：具備簡潔且易維護的語法，擁有豐富的機器學習與影像處理套件，是開發 AI 模型與後端系統的核心語言。本系統以 Python 建構伺服器端邏輯、AI 模型推論與資料處理流程，提升系統整體效能與可擴充性。

- MySQL：作為穩定的開源關聯式資料庫系統，負責儲存使用者資料、行為紀錄、影片資訊與互動紀錄。系統透過非同步連線池提升查詢效率，並支援多用戶同步查詢與分析。
- FastAPI + Uvicorn + WebSocket：FastAPI 為高效能的 Python 非同步框架，取代原本的 Flask，搭配 Uvicorn 作為 ASGI 伺服器以支援即時通訊與串流資料。WebSocket 則負責即時傳輸骨架資訊與 AI 推論結果，使跌倒偵測能即時回應。
- Android Studio：作為主要的前端開發環境，用於設計 Android App 的使用者介面與互動邏輯。內建模擬器支援多裝置測試，能快速驗證功能與畫面配置。
- Microsoft Azure(VM)+Docker：Azure 提供彈性的雲端虛擬機與容器部署能力，用於後端 FastAPI 系統與 AI 模型服務的部署。系統使用 Docker 管理環境版本與依賴，並以環境變數保護金鑰與機密資訊，確保安全與可維運性。
- Visual Paradigm：這是一款支援 UML 標準的建模工具，能繪製使用者案例圖、循序圖、活動圖等系統設計圖。本團隊透過 Visual Paradigm 進行系統流程與角色互動建模，讓文件與邏輯設計更具結構性與可讀性。
- Torch (PyTorch)：作為主要的深度學習框架，取代原本的 Keras。系統利用 Torch 建構 CNN 與 LSTM 架構的跌倒與動作辨識模型，支援 GPU 加速與高彈性模型管理。
- CNN (卷積神經網路)：用於擷取影像中空間特徵，辨識人體骨架變化與動作姿態，為時序模型提供視覺特徵基礎。
- LSTM (長短期記憶網路)：用於分析動作的時間序列特徵，協助模型學習動作間的長期依賴關係，提升跌倒與非跌倒的判斷準確度。
- OpenCV：影像處理核心函式庫，用於影格擷取、裁切、降噪與資料前處理。模型輸入前的所有影像操作皆透過 OpenCV 完成。
- YOLO / YOLO-pose：取代原本的 MediaPipe，作為本系統的物件與骨架生成核心。YOLO 用於偵測人物與環境物件框；YOLO-pose 則用於即時擷取人體關鍵點骨架資訊，提供 AI 模型輸入使用，實現多人物姿態分析與跌倒偵測。
- ChatGPT Text-to-Speech：用於語音關懷功能。系統將家屬貼文與每日新聞以文字轉語音方式播放給長者，增加互動性與陪伴感。

- n8n：作為自動化流程中台，負責 Facebook 授權貼文抓取、新聞摘要生成、週報推播及例行排程任務。可自動串接 API、資料庫與 LINE Bot，實現高整合度的智慧照護流程。
- SupabaseDB：作為資料存儲解決方案，並專門用來存放關鍵資料，如 family_post 和 emergency_contact。family_post 用來儲存家屬的社交平台貼文，這些貼文會被自動抓取並摘要，並且能夠透過 n8n 的自動化流程進行管理和推送。而 emergency_contact 則用來存儲長者的緊急聯絡人資料，這些資料能在系統偵測到異常情況時，快速發送通知給照護者或家屬。透過 Supabase DB，我們能夠實現資料的即時存取與處理。
- Facebook Graph API：授權後可存取家屬貼文內容，作為每日或每週互動播報素材來源，並由系統自動摘要與語音轉播。
- Line Messaging API：用於通知與互動機制，系統可透過 LINE 即時傳送跌倒警報、互動紀錄與每週報告，提升照護者與長者間的互通性。
- Google App Script (GAS)：用於資料報表自動化，將 n8n 或 API 分析結果同步到 Google Sheet，並與 LINE 通知功能整合。
- Figma：負責 UI/UX 介面設計，提升前端畫面一致性與操作直覺性。
- Canva、Word、Excel：用於簡報、報告與文件撰寫，呈現系統成果
- draw.io、ibis Paint X、Gantt.io：輔助繪製元件圖、架構圖與開發時程圖。
- InShot：用於成果影片剪輯與系統展示影片製作。
- GitHub：作為專案版本控制與協作平台，負責程式碼儲存、分支管理與版本追蹤。

表 3-3-1 使用標準與工具表

系統開發環境	
作業系統	Windows11
程式開發工具	
UI/UX 設計	Figma
前端	Android Studio
後端	Python、Microsoft Azure、MySQL、FastAPI、Uvicorn、WebSocket、SupabaseDB
AI 技術運用	
AI 模型開發	Torch、CNN、LSTM
影像處理	OpenCV
骨架生成	YOLO-pose
物件框生成	YOLO
語音關懷	ChatGPT Text-to-speech
自動化流程串接	n8n、Facebook Graph API、LINE Messaging API、Google App Script
文件美工工具	
文件	Microsoft Word、Microsoft Excel
簡報	Canva
圖檔	UML、Visual Paradigm、draw.io、ibis Paint X、Gantt.io
影片	InShot
專案管理平台	
專案管理	GitHub
檔案存放	GitHub

第4章 專案時程與組織分工

4-1 專案時程

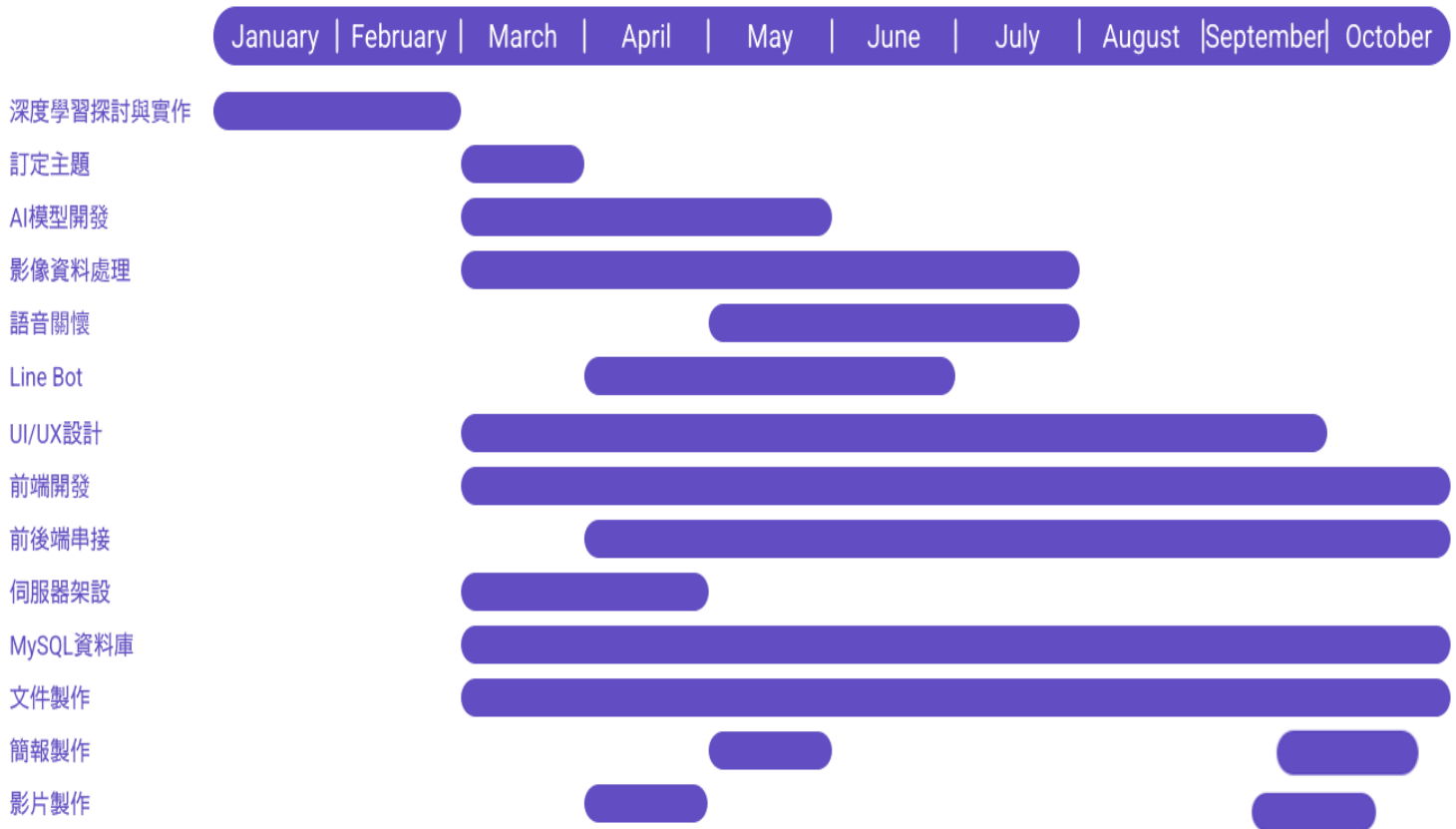


圖 4-1 專案時程甘特圖

自今年 1 月起，我們團隊依照專題規劃開始進行深度學習技術的初步研究，並在老師引導下閱讀相關教材，使組員對深度學習有一定了解。2 月至 3 月期間，我們完成專題主題的確認與系統功能討論，開始著手進行模型設計、資料蒐集與前端畫面規劃。至 4 月底前，已進行初步的資料處理、AI 模型架構建置與 UI 介面設計，並同步進行後端環境建置與資料庫串接作業。目前整體系統雛形已初步完成，並進入前後端整合與語音互動功能開發的階段，接著，我們在 5–6 月完成 AI 模型與影像處理流程的優化，並同步導入語音關懷模組與 LINE Bot 通知；7 月進行前後端串接的整合測試。8–9 月持續調校 MySQL 查詢效能，並針對跌倒偵測與睡眠/久坐紀錄進行壓力測試與錯誤情境驗證，同步修正 UI/UX 細節與文件。9–10 月聚焦穩定度與可維護性：整理技術文件與操作手冊、完成最終簡報與展示影片，並進行系統總整（前端、後端、資料庫、通知、語音互動）與情境式情境演練，確保期末成果展示可順利呈現完整功能與實際使用流程。

4-2 專案組織與分工

表 4-2-1 專案組織與分工表

項目/組員		11146005 沈宛宣	11146012 邱鈺婷	11146018 趙明威	11146033 陳欣怡	11146039 金峻瑋
	Azure 開發環境建置			●		○
	Line messaging API					●
	ChatGPT API					●
	LineBot API					●
	Elevenlabs API					●
	ENlevenlabs 文字轉語音					●
	N8N-分流器					●
	N8N-語音互動工作流					●
	N8N-骨架分析工作流					●
	N8N-MySQL 存入 postgresDB					●
	N8N-line 週報推送					●
	資料清洗、整理、切分			●		○
	跌倒判斷二分類模型訓練			●		
	動作多分類模型訓練			●		
	姿態二階段判斷 API			●		
	骨架、物件框輸入前處理			●		
	跌倒、動作判斷後處理			●		
	資料庫建置-MySQL				●	
	FastAPI			●		○
	MySQL-使用者資訊表				●	
	MySQL-使用者關係表				●	
	MySQL-通知相關表				●	
	MySQL-事件行為表				●	
	MySQL-社群帳戶表				●	
	MySQL-互動紀錄表				●	
	MySQL-互動報告表				●	
	MySQL-社群貼文內容表				●	
	N8N-FB 授權流程				●	
	N8N-FB 貼文抓取				●	
	N8N-FB 貼文前處理				●	
	N8N-互動紀錄分析				●	
前端	UI/ UX 設計		●			

項目/組員		11146005 沈宛宣	11146012 邱鈺婷	11146018 趙明威	11146033 陳欣怡	11146039 金峻瑋
開發	App 介面設計		●			
	App-主選單		●			
	App-偵測系統		●			
	App-個人資料		●			
	App-跌倒列表		●			
	App-互動報告		●			
	App-睡眠列表		●			
	App-久坐資訊		●			
	App-語音對話紀錄		●			
	App-緊急聯絡人		●			
	App-選擇長者		●			
	APP-物件偵測鏡頭串接 YOLOv8			●		
	APP-人物偵測鏡頭串接 YOLOv8-pose			●		
美術設計	色彩設計	●				
	Logo 設計	●				
文件撰寫	統整	●				
	第 1 章 前言	●				
	第 2 章 營運計畫	●				
	第 3 章 系統規格	●				
	第 4 章 專題時程與組織分工	●				
	第 5 章 需求模型	●	○			
	第 6 章 設計模型		●	○		
	第 7 章 實作模型	●	○			○
	第 8 章 資料庫設計				●	
	第 9 章 程式			●	○	○
	第 10 章 測試模型	●				
	第 11 章 操作手冊		●			
	第 12 章 使用手冊	○	●			
報告	簡報製作	●				
	影片製作	●				

●主要負責人 ○次要負責人

表 4-2-2 專案成果內容與共貢獻度表

序號	姓名	工作內容<各限 100 字以內>	貢獻度
1	組長 <u>沈宛宣</u>	文件：統整、CH1-CH5、CH7(元件圖)、CH10、CH12(狀態轉移圖)、競賽相關文件與準備、會議記錄 影片：影片構思、拍攝、剪輯 美編：簡報製作及統整、logo 設計、名牌製作	18%
2	組員 <u>邱鈺婷</u>	App：UI/UX 設計、介面設計、頁面框架撰寫、Android 應用程式開發 API：應用程式介面框架與 API 整合、串接鏡頭影像接收 文件：使用個案描述、分析類別圖、循序圖、設計類別圖、狀態機、操作手冊、使用手冊	21%
3	組員 <u>趙明威</u>	雲端：Azure 開發環境建置與配置 資料前處理：取得學習與訓練用影像、影像與骨架資料之清洗、轉換、分段標記 模型：跌倒二分類模型之訓練與測試、動作多分類模型之訓練與測試 API：實裝模型並撰寫二階段偵測(跌倒二分類、動作多分類) API(WebSocket 串流) 文件：循序圖、參考資料(模型學習資料集)、CH9 程式	22%
4	組員 <u>陳欣怡</u>	資料庫：MySQL 資料庫建置、設計及管理 文件：ER-model 圖、資料庫資料表、程式 n8n 與 sql API：設計及管理、串聯資料庫 API 資料 N8N：fb 授權、貼文前處理、抓取貼文、互動分析流程設計與測試、CH9 程式(n8n)	20%
5	組員 <u>金峻瑋</u>	雲端伺服器：Azure 伺服器建置、管理 後端：LineBot API、Line Messaging API 文件：CH7 套件圖、CH9 程式(n8n) N8N：語音互動工作流、分流器、MySQL 資料轉入 postgresDB、社群貼文 RAG、骨架分析與通知工作流、上下文對話記憶設計 其他：協助影像判斷資料剪輯	19%
			總計:100%

當我們確立專題方向後，團隊將整體任務分為五大面向，分別是：後端系統建置、前端設計開發、AI 模型訓練與偵測整合、美術與介面設計，以及文件撰寫與管理。為了提升分工效率，我們依照每位組員的專長與經驗，進一步細分各項工作內容，並採用角色導向的任務分配模式。這樣的安排不僅讓每位組員都能專注於自己擅長的領域，也提升了整體協作效率。

4-3 上傳 GitHub 紀錄

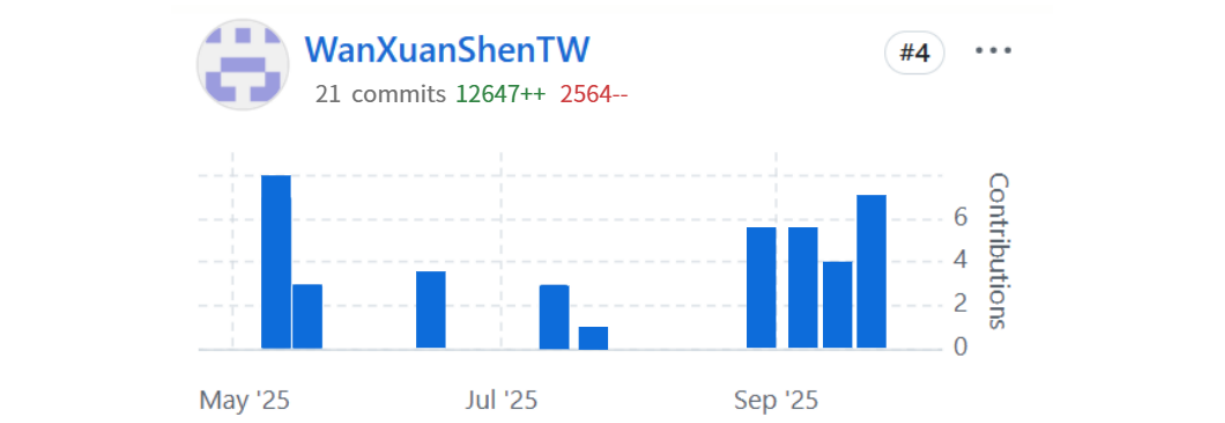


圖 4-2 11146005 沈宛宣 上傳 GitHub 紀錄

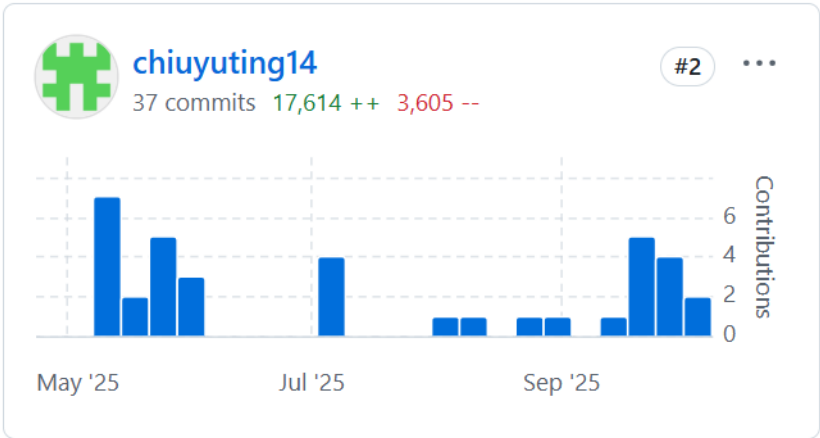
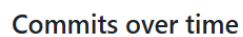
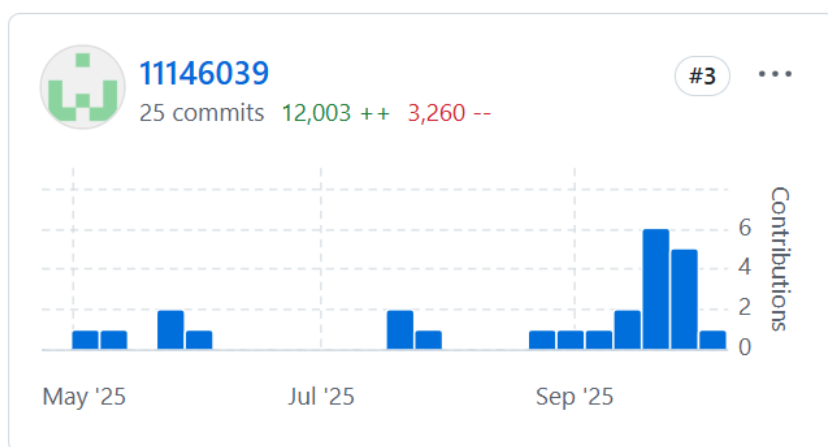
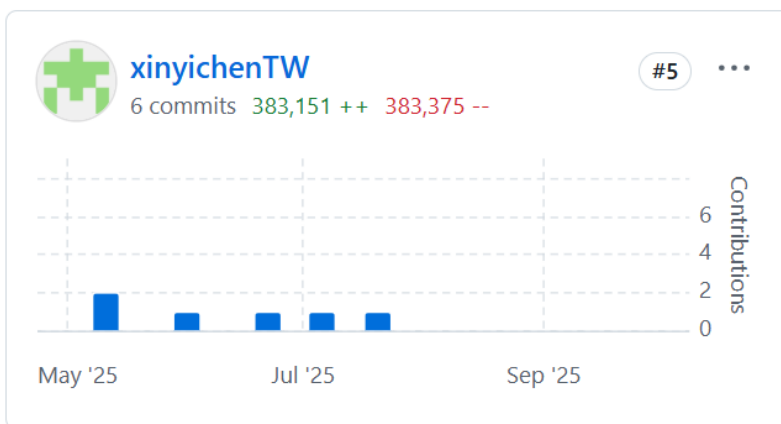


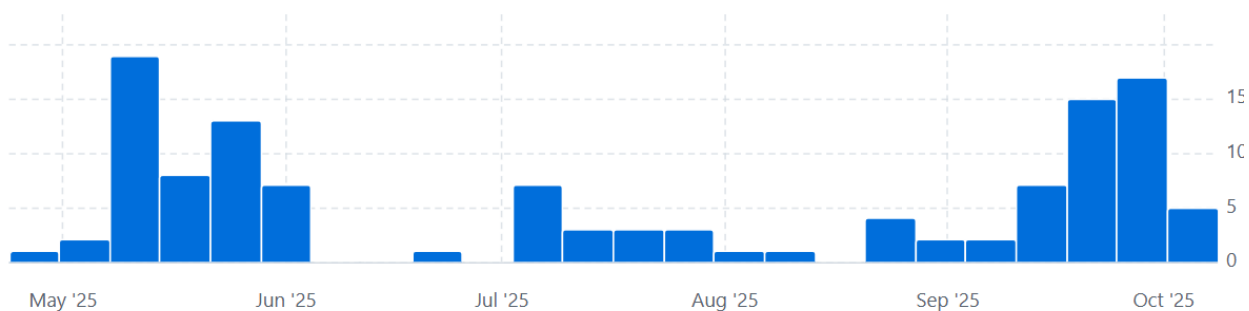
圖 4-3 11146012 邱鈺婷 上傳 GitHub 紀錄



圖 4-4 11146018 趙明威 上傳 GitHub 紀錄



Weekly from 2025年4月27日 to 2025年10月5日



第5章 需求模型

5-1 使用者需求

表 5-1-1 為使用者需求表，列出系統針對不同使用情境（Use Case）所對應的功能需求。內容涵蓋歷史影像查詢、即時通知、異常統計報表、登入機制、即時監控、風險分析報告及語音互動紀錄等，全面滿足照護者對於安全監控、行為判讀與情感互動的多元需求。

表 5-1-1 使用者需求表

Use Case	需求
查看互動報告	使用者可快速了解長者當週狀態總結，透過長者與系統的互動，統計長者的興趣及活動等資訊
即時突發異常通知	使用者可於長者發生跌倒等突發異常狀況時，立即透過 LINE Bot 接收通知與影片片段，及時掌握事件發生情況
登入/註冊帳號	使用者透過電話號碼登入及註冊 App
查看緊急聯絡人	使用者可查看長者的「緊急聯絡人」清單，並支援一鍵撥號，遇到突發狀況時能迅速通知
查看跌倒事件	使用者可利用時間區段查詢長者之「跌倒／疑似跌倒」發生前的步伐狀態，且了解跌倒事件時間發生地點，以便後續追蹤長輩狀態。
查看睡眠列表	使用者可查看長輩每天/每週的起床/睡覺時間，由偵測系統辨識長輩睡眠並回傳，以了解長輩的生活作息以及睡眠狀態
查看久坐資訊	使用者可查看長輩每天/每週的久坐時間，由偵測系統辨識久坐，紀錄以讓使用者進一步了解長輩身體狀況
查看活動量紀錄	使用者可查看長者每日／每週活動量趨勢圖表，比對前後期變化，以便掌握日常作息與異常變化
查看語音互動紀錄	使用者可查看當日長者與系統的語音互動紀錄，透過對話框形式呈現，掌握長者日常互動狀態與反應
FB 授權-播報子女貼文	使用者可於 App 端連結 Facebook 帳號，授權系統存取貼文內容，以供後續自動生成互動報告與語音播報，增進長者與家屬間的互動連結。

5-2 使用個案圖(Use case diagram)

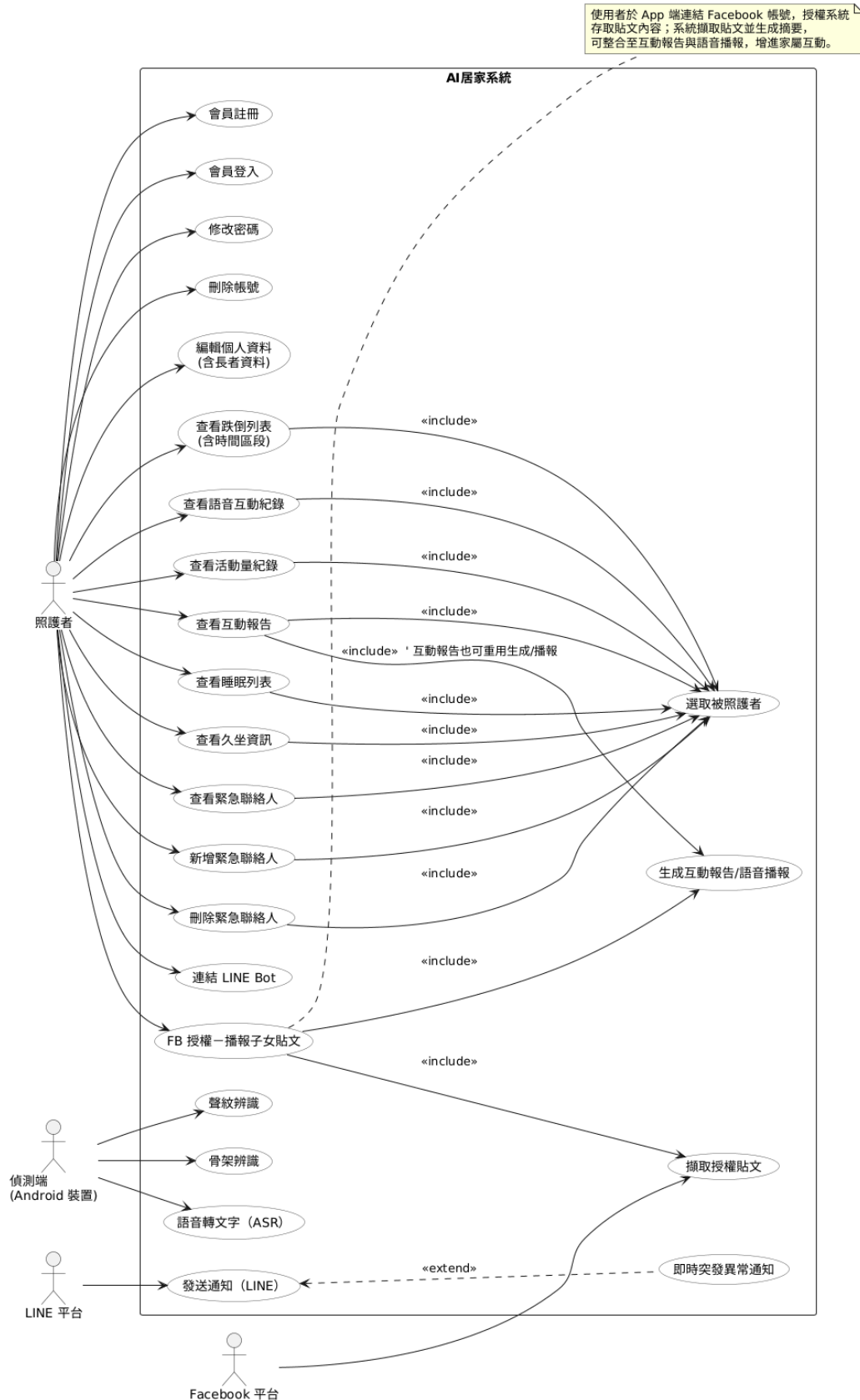


圖 5-1 使用個案圖

圖 5-2-1 為使用個案圖，主要使用者為「照護者」，可完成會員註冊/登入、修改密碼、刪除帳號與編輯個人與長者資料；在查詢面，照護者能查看跌倒列表（含時間區段）、語音互動紀錄、活動量、互動報告、睡眠與久坐資訊，這些查詢情境皆會先「選取被照護者」。聯絡人管理包含查看/新增/刪除緊急聯絡人。通知面則可連結 LINE Bot，當系統偵測到異常時，會「延伸」觸發發送 LINE 通知的流程。偵測端（Android 裝置）負責語音轉文字、聲紋與骨架辨識，持續產出判斷依據。新增的「FB 授權—播報子女貼文」讓照護者在 App 端連結 Facebook，系統「擷取授權貼文」並「生成互動報告/語音播報」，以強化長者與家屬的日常互動與陪伴。

5-3 使用個案描述

表 5-3-1 查看跌倒狀態列表

使用個案名稱：查看跌倒狀態列表	
行為者	使用者
前提	使用者已登入系統
結束狀態	了解長者跌倒狀態
事件路徑：	
Actor 動作	系統回應
1.在主選單下方點擊「回放圖示」	
2.下拉式選單選取「跌倒列表」事件	
3.選取查看時間	4.列出跌倒前狀態、跌倒地點及時間

表 5-3-2 即時突發異常狀況通知

使用個案名稱：即時突發異常狀況通知	
行為者	使用者
前提	使用者已登入系統
結束狀態	已接收到異常通知，並得以查看事發影片片段
事件路徑：	
Actor 動作	系統回應
無需主動操作（背景運作）	1.系統偵測長者可能跌倒的行為
	2.將異常通知透過 LINE Bot 傳送給使用者
	3.同步記錄異常事件與影片資訊至資料庫，供後續查詢使用

表 5-3-3 異常統計報表通知

使用個案名稱：異常統計報表通知	
行為者	使用者
前提	使用者已登入系統
結束狀態	已接收到統計異常的通知並檢視近七日的異常狀態報表
事件路徑：	
Actor 動作	系統回應
無需主動操作（系統背景監測）	1. 系統每日統計長者跌倒、差點跌倒、步態異常的次數
	2. 系統偵測近一週頻率高於以往平均值（即出現異常）
	3. 自動產出近 7 天異常狀態統計圖表
	4. 將統計圖表與提醒訊息透過 LINE Bot 傳送給使用者者

表 5-3-4 查看互動報告

使用個案名稱：查看跌倒風險報告	
行為者	使用者
前提	使用者已登入系統
結束狀態	知道長者當週與系統互動狀態
事件路徑：	
Actor 動作	系統回應
1. 在主選單下方點擊「查詢圖示」	
2. 下拉式選單選取「互動報告」事件	
3. 選取查看時間(選日期/最新)	4. 顯示當週長輩互動狀況

表 5-3-5 查看睡眠紀錄

使用個案名稱：查看睡眠紀錄	
行為者	使用者
前提	使用者已登入系統
結束狀態	知道長者每週/每天的起床及睡覺狀態
事件路徑：	

Actor 動作	系統回應
1.在主選單頁面點擊「查詢圖示」	
2.下拉式選單選取「睡眠列表」事件	
3.選取查看時間(天/週)	4.顯示當天/當週的睡眠紀錄

表 5-3-6 查看久坐紀錄

使用個案名稱：查看久坐紀錄	
行為者	使用者
前提	使用者已登入系統
結束狀態	知道長者每週/每天的久坐狀態
事件路徑：	
Actor 動作	系統回應
1. 在主選單頁面點擊「查詢圖示」	
2. 下拉式選單選取「久坐資訊」事件	
3. 選取查看時間(天/週)	4. 顯示當天/當週的久坐紀錄

表 5-3-7 查看語音互動紀錄

使用個案名稱：查看語音互動紀錄（長者與系統對話）	
行為者	使用者
前提	使用者已登入系統
結束狀態	已掌握長者當日與系統互動的語音內容與生活狀況反應
事件路徑：	
Actor 動作	系統回應
1.在主選單頁面點擊「對話圖示」	2.顯示當日與被照護者對話的對話框

表 5-3-8 撥打緊急聯絡人

使用個案名稱：撥打緊急聯絡人	
行為者	使用者
前提	使用者已登入系統
結束狀態	已聯絡到緊急聯絡人
事件路徑：	

Actor 動作	系統回應
1. 在主選單下方點擊「電話圖示」	2. 顯示長者綁定的緊急聯絡人
3. 點擊右側電話圖示	4. 跳轉到撥號頁面
5. 照護者進行通話	

表 5-3-9 查看報告圖表

使用個案名稱：查看報告圖表	
行為者	使用者
前提	使用者已登入系統
結束狀態	照護者了解長輩狀況
事件路徑：	
Actor 動作	系統回應
1. 進入 App 系統	2. 顯示長者跌倒、活動量圖表

表 5-3-10 FB 授權-播報子女貼文

使用個案名稱：FB 貼文授權	
行為者	使用者
前提	使用者登入 APP
結束狀態	已成功授權並將 access_token 儲存至系統
事件路徑：	
Actor 動作	系統回應
1. 進入 App 系統，選擇「Facebook 授權」功能	2. 系統導向 Facebook 登入與授權頁面
3. 使用者於 Facebook 頁面登入帳號	4. Facebook 驗證使用者身分並要求授權確認
5. 使用者點擊「允許授權」	6. Facebook 將授權碼 (code) 傳回後端
7. 系統伺服器使用授權碼向 Facebook 交換 access_token	8. Facebook 回傳 access_token
9. 系統儲存 access_token 至 MySQL 並綁定使用者 ID	10. 顯示「授權成功」訊息並回到主畫面
11. 若使用者按「取消」或驗證失敗	12. 顯示「授權失敗或已取消」訊息

5-4 分析類別圖(Analysis class diagram)

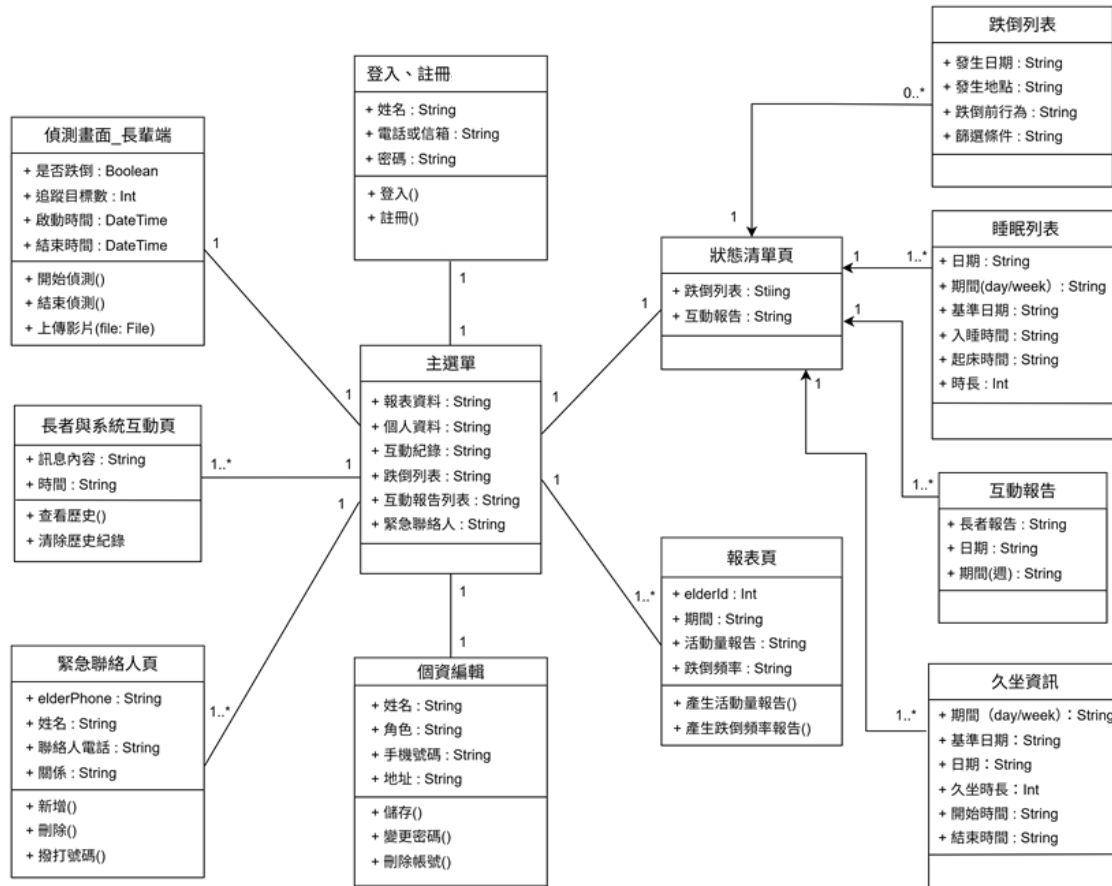


圖 5-2 分析類別圖

圖 5-4-1 為系統分析類別圖，描繪照護者端 App 的主要畫面類別與其關聯。中心為主選單（導航聚合類別），連結至登入／註冊、個資編輯、緊急聯絡人頁、長者與系統互動頁與偵測畫面__長輩端；其中狀態清單頁再細分關聯到跌倒列表、睡眠列表、久坐資訊、互動報告與報表頁等功能。各類別定義了核心屬性（如當前被照護者、時間區段、清單資料）與操作方法（查詢、篩選、切換被照護者、撥打電話、編輯／刪除聯絡人...），並以關聯線標示導覽與資料流向，確保所有列表與報表皆以「被照護者」作為查詢鍵值。此圖可作為後續開發的結構藍本，統一頁面間的資料傳遞與事件觸發規格。

第6章 設計模型

6-1 循序圖(Sequential diagram)或通訊圖(Communication diagram)

圖 6-1-1 為註冊會員的循序圖，描述使用者進行註冊時的流程。當使用者輸入資料後，系統會透過登入畫面與資料庫互動，驗證電話號碼是否已存在。若已註冊，系統回傳註冊失敗訊息；若尚未註冊，則會儲存使用者資料並回傳成功訊息。此圖清楚呈現不同條件下的邏輯分支與系統反應流程。

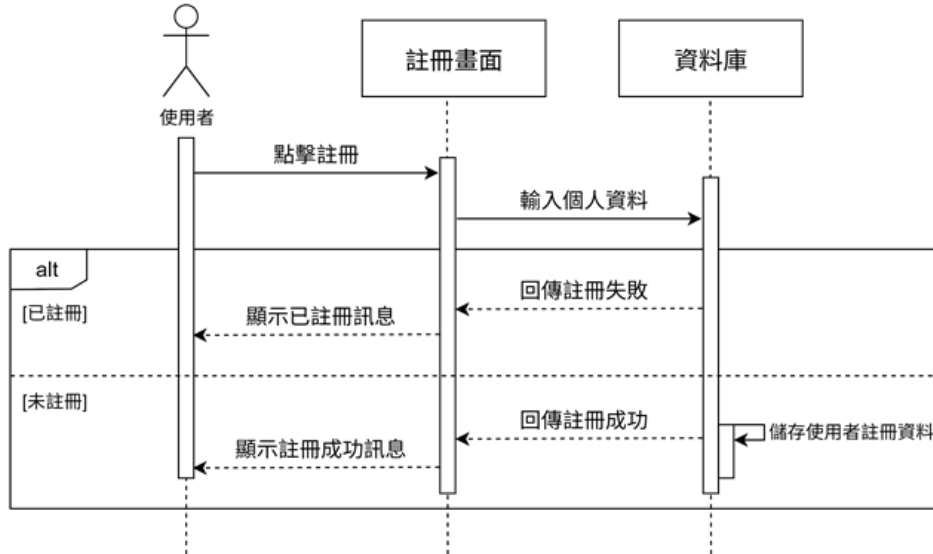


圖 6-1 循序圖-註冊會員

圖 6-1-2 為 登入流程的循序圖，說明使用者登入系統時的操作步驟與系統回應。當使用者輸入手機號碼與密碼後，系統會向資料庫進行驗證。若資訊正確，則顯示首頁；若密碼錯誤或手機號碼不存在，則回傳此號碼未註冊消息。

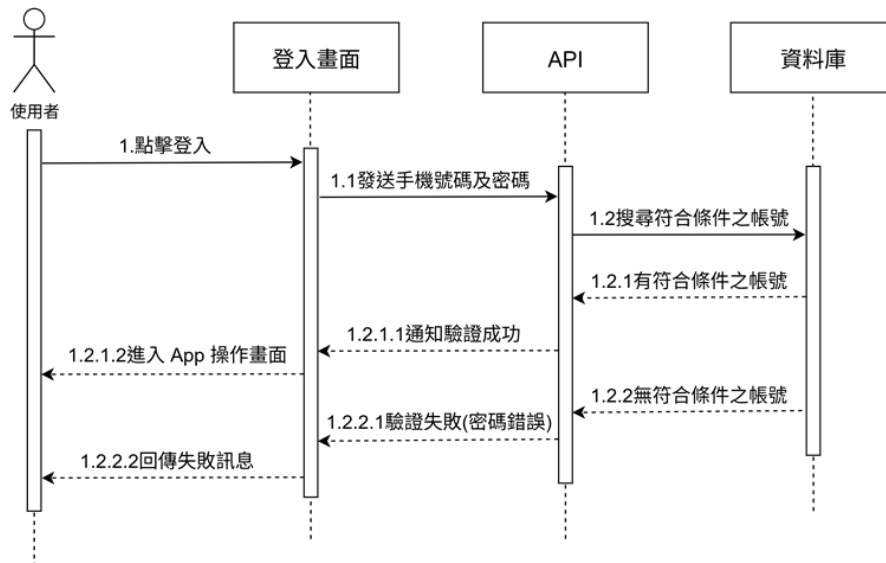


圖 6-2 循序圖-登入

圖 6-1-3 為更改密碼流程的循序圖，描述使用者在登入後進入個人頁面並選擇更改密碼的操作流程。使用者輸入當前密碼與新密碼後，系統會向資料庫驗證舊密碼是否正確，若驗證成功則更新新密碼並顯示成功訊息。此流程強調帳戶安全驗證，確保只有本人能修改密碼。

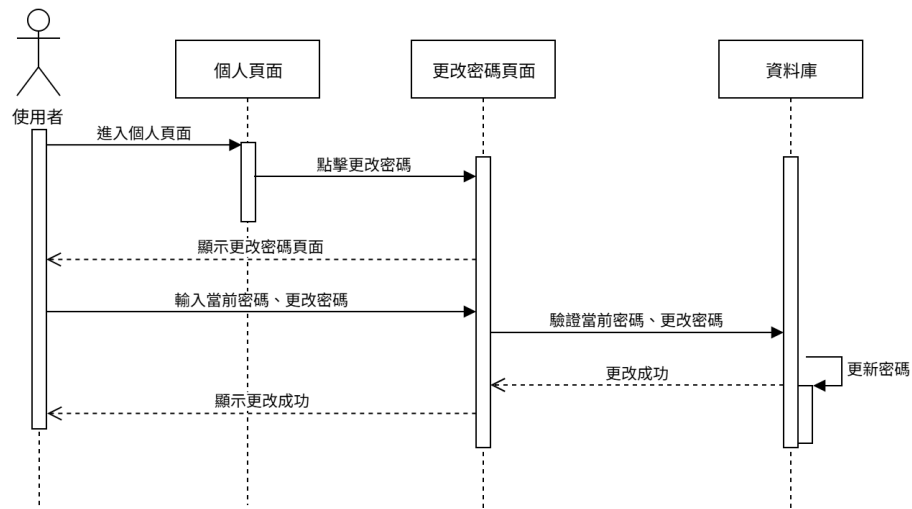


圖 6-3 循序圖-更改密碼

圖 6-1-4 為攝影鏡頭傳送至伺服器的循序圖，本圖說明使用者啟動偵測頁面後，系統透過 WebSocket 以每秒固定 10 fps 的頻率上傳骨架資料至伺服器。伺服器端在接收到足夠數量的幀資料後，執行 AI 模型推論（跌倒、走路、坐下、躺下），並根據判斷結果即時回傳分類結果給用戶端。若判定為非跌倒類（如坐下、躺下），則系統同時將該事件紀錄與狀態結果儲存至資料庫中。若資料不足或使用者 ID 錯誤，伺服器則回傳錯誤提示。此循序圖完整呈現了即時串流資料在 前端、伺服器與資料庫間的互動流程，展現本系統即時行為辨識與結果儲存的運作機制。

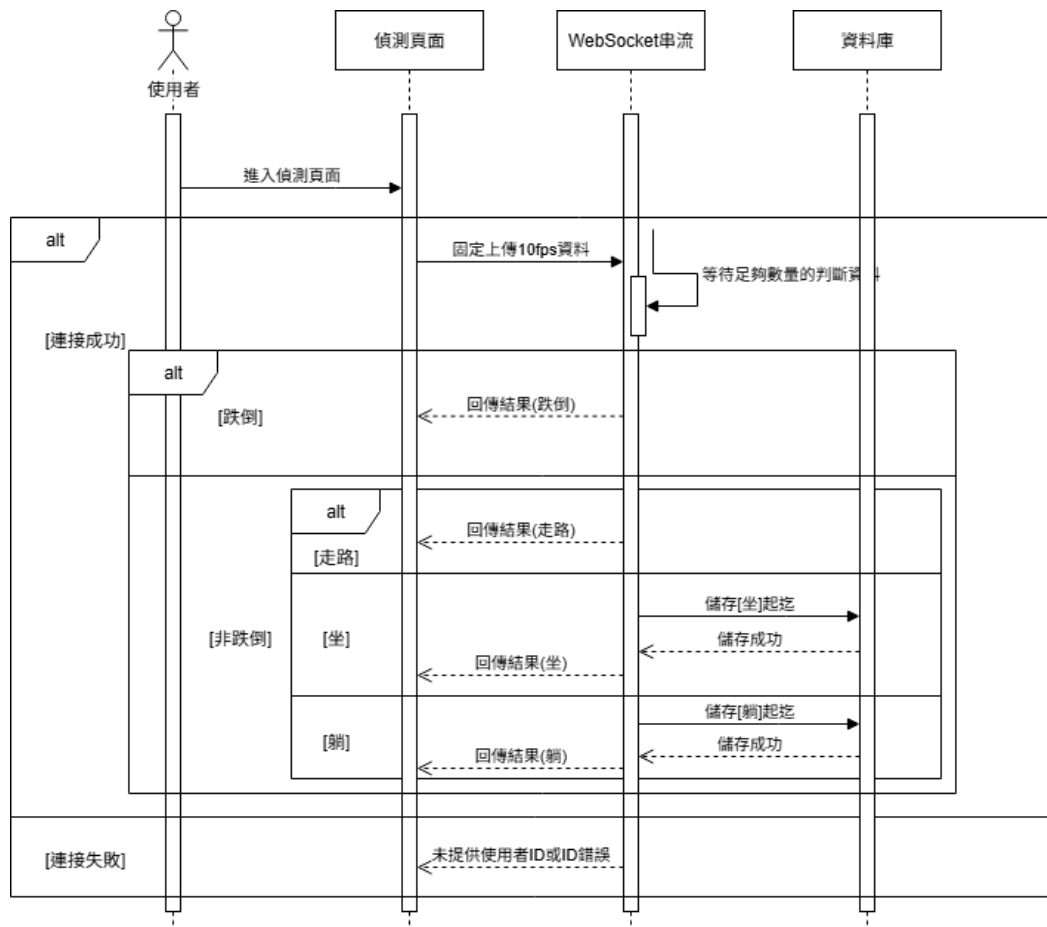


圖 6-4 循序圖-攝影機鏡頭傳送至伺服器

圖 6-1-5 長者跌倒列表的循序圖，使用者在「列表頁面」選擇跌倒狀態清單之後，並選擇查看時間區段，系統再透過 API 向資料庫查詢對應的紀錄，最終依查詢條件顯示跌倒狀態列表。

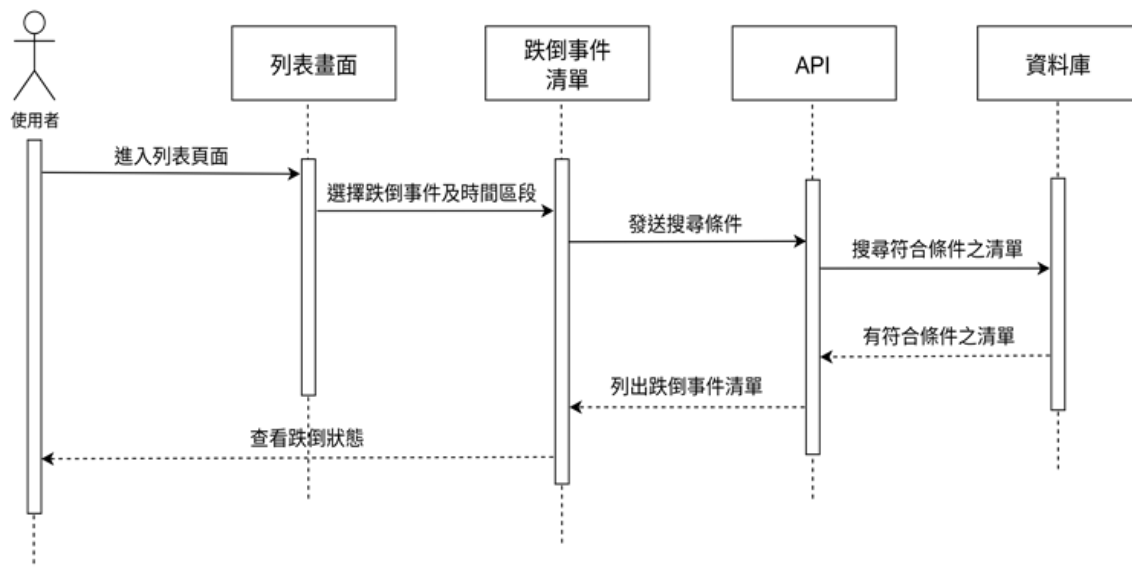


圖 6-5 循序圖-查詢跌倒列表

圖 6-1-6 為使用者進入查詢列表頁面，選取互動報告選單後送至 API；API 依條件向資料庫查詢並回傳資料，頁面則會列出當週的互動報告總結。

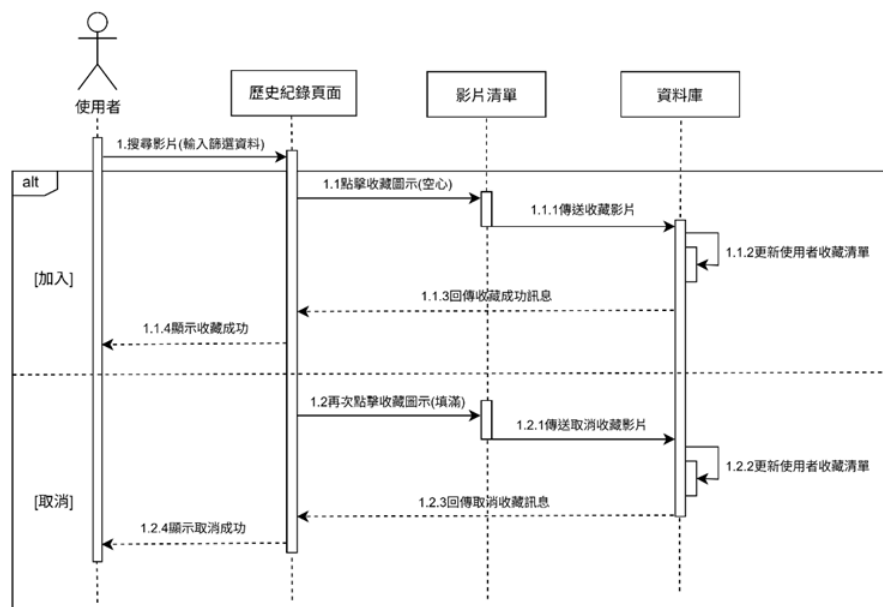


圖 6-6 循序圖-查詢互動報告

圖 6-1-7 為使用者開啟緊急聯絡人頁，送出長輩識別資訊至 API；API 於資料庫查詢綁定緊急聯絡人並回傳，頁面顯示姓名、關係與電話資訊，並提供一鍵撥打與複製號碼功能。

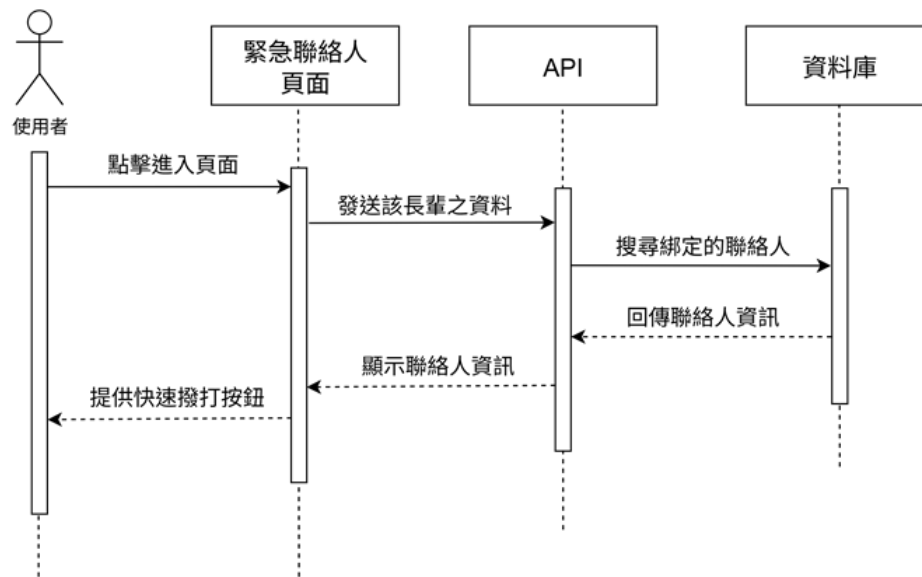


圖 6-7 循序圖-查詢緊急連絡人

圖 6-1-8 為使用者從列表頁選擇「睡眠列表」，前端選擇日期並將查詢條件送至 API。API 連結資料庫檢索符合條件的睡眠紀錄並回傳。頁面顯示每日入睡、起床與時長清單，可切換週期並查看當週／當日睡眠狀態與趨勢。

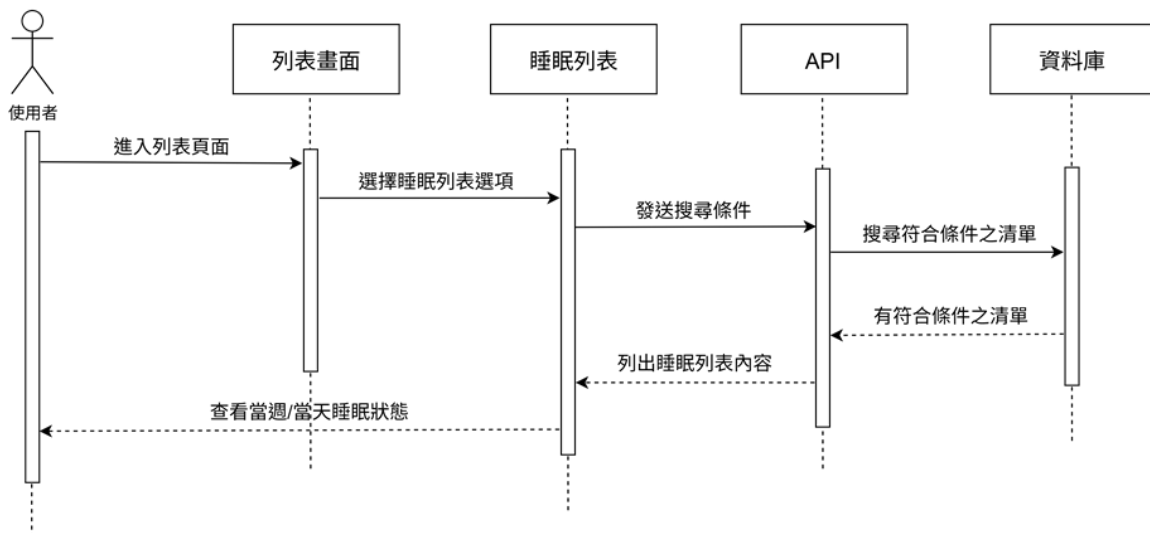


圖 6-8 循序圖-查看睡眠狀態列表

圖 6-1-9 為使用者於列表頁選擇「久坐資訊」，前端依日期將查詢條件送至 API。API 連結資料庫篩選出符合條件的久坐紀錄並回傳。畫面顯示每日久坐總時長，可切換週/日檢視，供照護者了解當週／當日久坐狀態與趨勢。

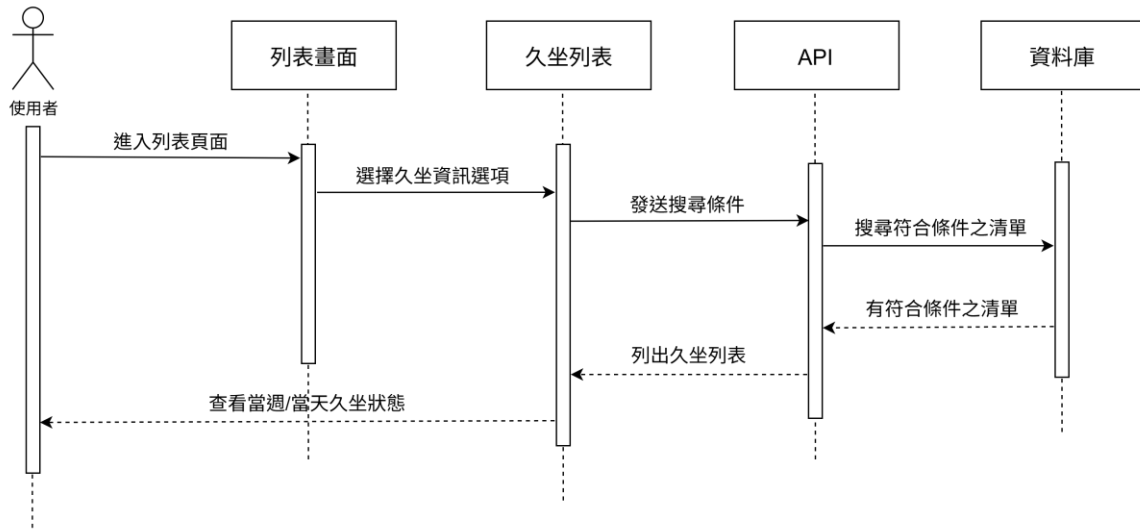


圖 6-9 循序圖-查看久坐狀態列表

使用者於系統點擊「Facebook 授權」後，系統開啟 Facebook 登入頁面，登入授權後，Facebook 伺服器隨即回傳授權碼（code）至 n8n Webhook，n8n 透過 Facebook Graph API 請求交換 access_token，並將取得的 access_token 與使用者綁定寫入資料庫。完成後，系統回傳成功訊息，App 顯示授權成功畫面，供照護者確認社群帳號綁定狀態，以便後續擷取家庭貼文與互動內容。

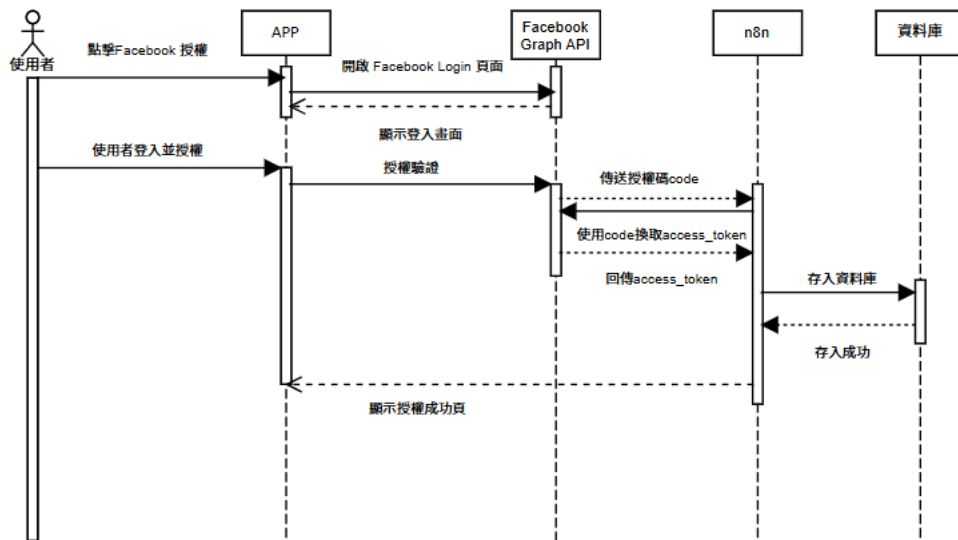


圖 6-10 循序圖-FB 授權流程

6-2 設計類別圖(Design class diagram)

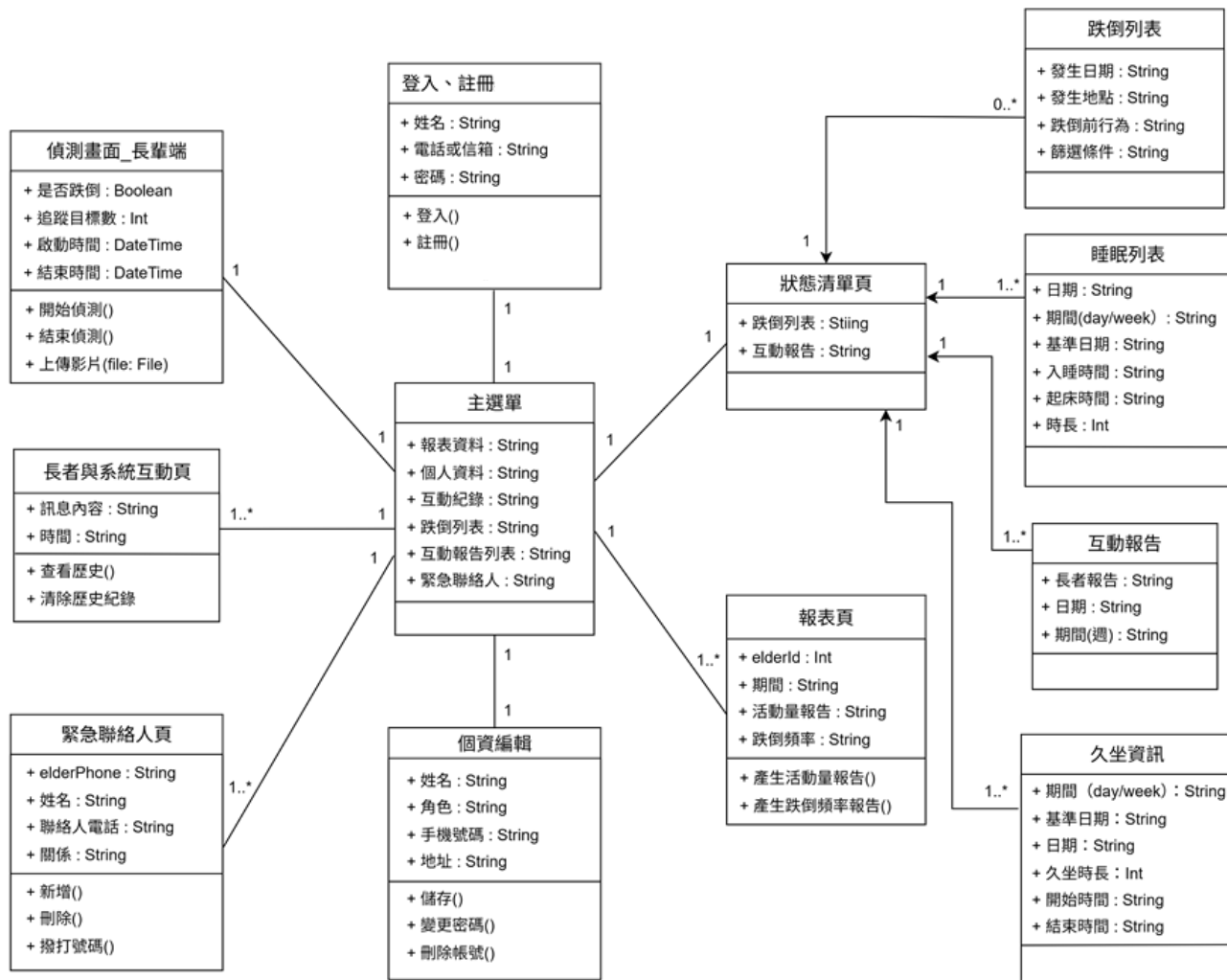


圖 6-11 設計類別圖

呈現照護者端 App 與偵測端之主要類別、屬性/方法與它們的資料與導覽關聯。以主選單為核心樞紐，連結到登入／註冊、個資編輯、緊急聯絡人頁、長者與系統互動頁與偵測畫面_長輩端等功能；其中狀態清單頁為列表容器，與跌倒列表、睡眠列表、久坐資訊、互動報告、報表頁建立 1..* 關聯，負責帶入查詢鍵值（如 elderId、日期/週）與篩選條件並顯示結果。各列表類別定義了必要屬性（事件時間、地點或指標數值等）與操作（查詢、篩選、載入更多），報表頁則依 elderId、時間區間與量測項目產生統計圖表。偵測畫面_長輩端負責即時感知與上傳結構化資料（如骨架與語音轉文字），長者與系統互動頁呈現對話紀錄；緊急聯絡人頁提供檢視/新增/刪除等操作。圖中的多重度（1、1..*）與關聯線清楚描述畫面間的邏輯連結與資料流向，可作為實作時的模組切分與資料整合依據。

第7章 實作模型

7-1 佈署圖(Deployment diagram)

圖 7-1-1 為本系統的佈署圖，說明各元件的運作位置。使用者透過 Android 手機執行 AI 底家，影像資料會上傳至 Azure 虛擬機，由 FlaskAPI 串接 MediaPipe 進行姿勢擷取，並透過 CNN-LSTM 模型進行跌倒判斷，再由 ChatGPT 產生語音提醒。資料最終儲存在獨立的 MySQL 資料庫中。

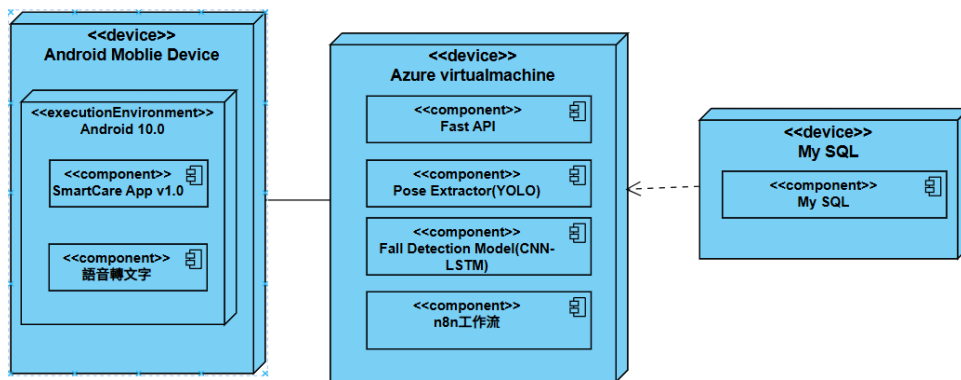


圖 7-1 佈署圖

7-2 套件圖(Package diagram)

圖 7-2-1 為本系統在資料視覺化與分析階段所使用的套件圖，tqdm（版本 4.67.1）用於顯示模型訓練與資料處理的進度條，方便監控執行進度；pytz（版本 2025.2）則用於時間與時區的轉換，確保不同裝置與地區之間的時間同步性。這些工具共同提升了系統在資料分析階段的效率與精準度，並有助於後續視覺化結果的正確呈現。

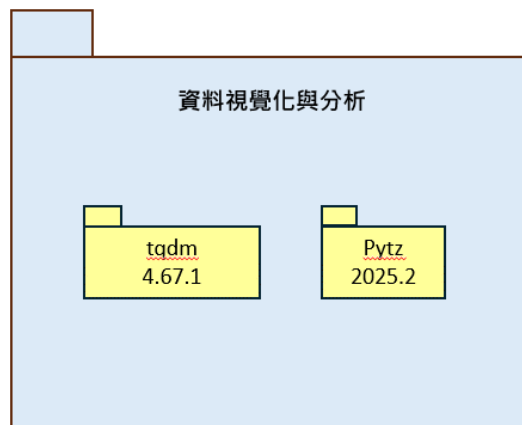


圖 7-2 套件圖-資料視覺化與分析

圖 7-2-2 為本系統在資料儲存至 MySQL 階段所使用的套件圖，aiomysql (0.2.0) 與 PyMySQL (1.1.2) 用於非同步與同步的資料庫連線與查詢操作，mysql-connector-python (9.4.0) 負責穩定的資料庫連接與指令執行，而 json 套件則協助進行資料格式轉換與結構化儲存。透過這些工具的整合，系統能有效管理資料庫連線、提升查詢效率，並確保資料交換的正確性與穩定性。

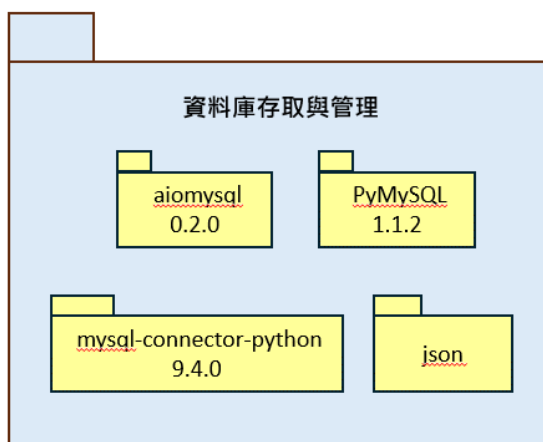


圖 7-3 套件圖-資料存入 SQL

圖 7-2-3 為本系統進行 AI 跌倒偵測所使用的套件圖，torch (2.8.0+cpu) 與 torchvision (0.23.0+cpu) 用於深度學習模型的建立與推論；numpy (2.2.6) 負責進行數值運算與資料矩陣處理；opencv-python (4.12.0.88) 用於影像擷取與前處理；pillow (11.0.0) 輔助影像轉換與格式處理；而 json 則用於骨架資料與模型輸出結果的序列化與交換。透過這些套件的整合，系統能即時完成影像處理、姿態辨識與跌倒偵測流程，實現高效且穩定的 AI 行為分析功能。

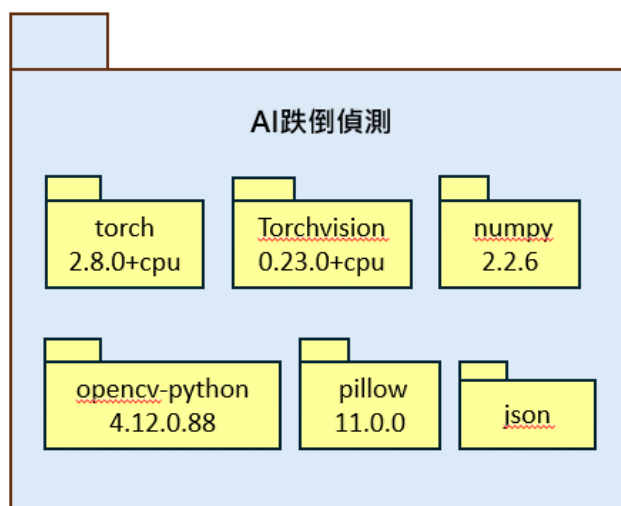


圖 7-4 套件圖-AI 跌倒偵測

圖 7-2-4 為本系統 WebAPI 服務所使用的套件圖，FastAPI（0.117.1）為主要的後端框架，用於建立高效能的 RESTful API；Uvicorn（0.36.0）作為 ASGI 伺服器，負責非同步請求處理與服務啟動；Jinja2（3.1.4）用於動態模板渲染；Websockets（15.0.1）支援即時資料傳輸與雙向通訊；Starlette（0.48.0）則提供 FastAPI 的底層非同步框架基礎。透過這些套件的整合，系統能實現穩定、高效且可擴充的 Web API 服務架構，支援前後端即時互動與 AI 推論資料交換。

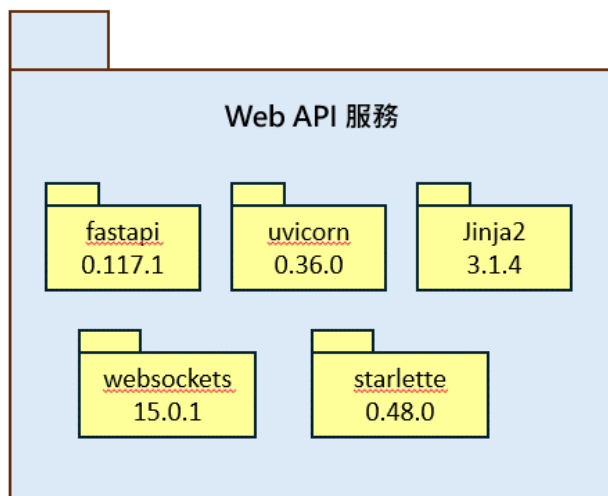


圖 7-5 套件圖-WebAPI 服務

圖 7-2-5 為本系統 LINE 整合通知所使用的套件圖，line-bot-sdk（3.19.1）為核心開發套件，用於與 LINE 官方帳號進行訊息傳遞與事件處理；httpx（0.28.1）與 aiohttp（3.12.15）提供同步與非同步的 HTTP 請求支援，用於與外部 API 溝通；requests（2.32.5）則用於簡化伺服器端的資料傳輸與回應流程。透過這些套件的協同運作，系統能即時傳送跌倒警報、互動回覆與週報通知，達成即時、穩定的 LINE 通訊整合功能。

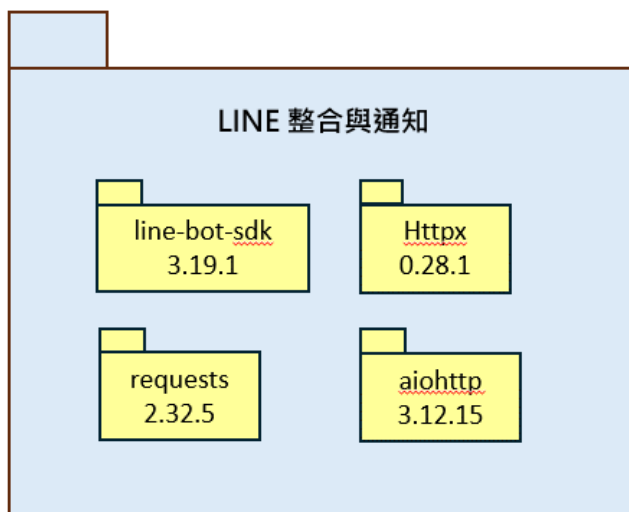


圖 7-6 套件圖-LINE 整合通知

圖 7-2-6 為本系統 N8N 自動化流程與推理模組的套件圖，整體部署於同一執行環境（如 Docker/VM），包含三個元件：① N8N 3.8 作為流程編排與排程中樞，負責接收 Webhook、定時觸發、資料整理與後續動作；② ollama 為本機模型推理服務，提供統一的 HTTP API；③ gemma3:4b 為部署在 ollama 上的語言模型權重，負責文字生成/分析。運作流程為：N8N 依流程呼叫 ollama 指定 gemma3:4b 進行推理，取得結果後再寫入資料庫或觸發通知。此設計將「編排」與「推理」解耦，便於日後替換模型或擴充更多自動化流程。

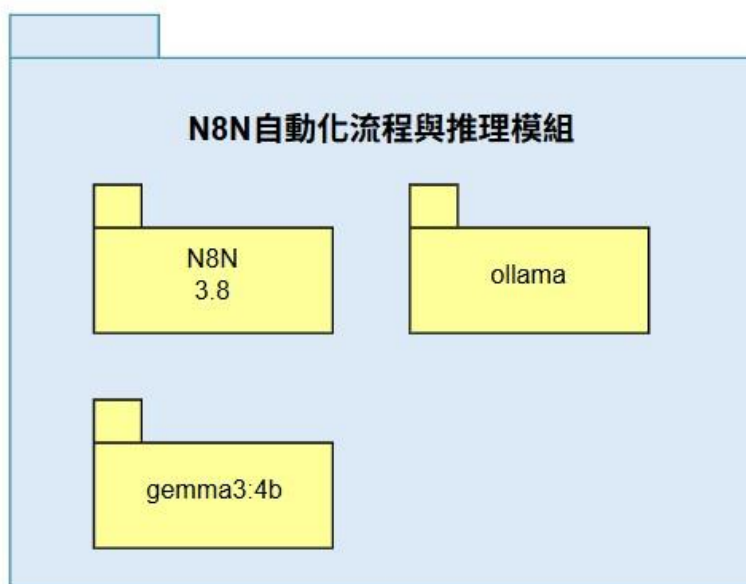


圖 7-7 套件圖-N8N 自動化流程與推理模組

7-3 元件圖(Component diagram)

圖 7-3-1 為本系統的元件圖，說明各功能模組的組成與層級關係。以「登入」為核心，左側為使用者管理：個人資訊（含編輯個資、修改密碼、刪除帳號）、FB 授權與帳號綁定。中間為服務與查詢：通知模組（異常動作警告、異常紀錄通知）、列表查詢（跌倒列表、睡眠列表、活動量紀錄、久坐資訊），搭配歷史影像回放。右側為對外互動：連結 LINE Bot、語音播報、語音互動紀錄與互動報告，以及聯絡人管理（查看／新增／刪除緊急聯絡人）。此結構將功能清楚模組化，便於分工開發、測試與後續擴充。

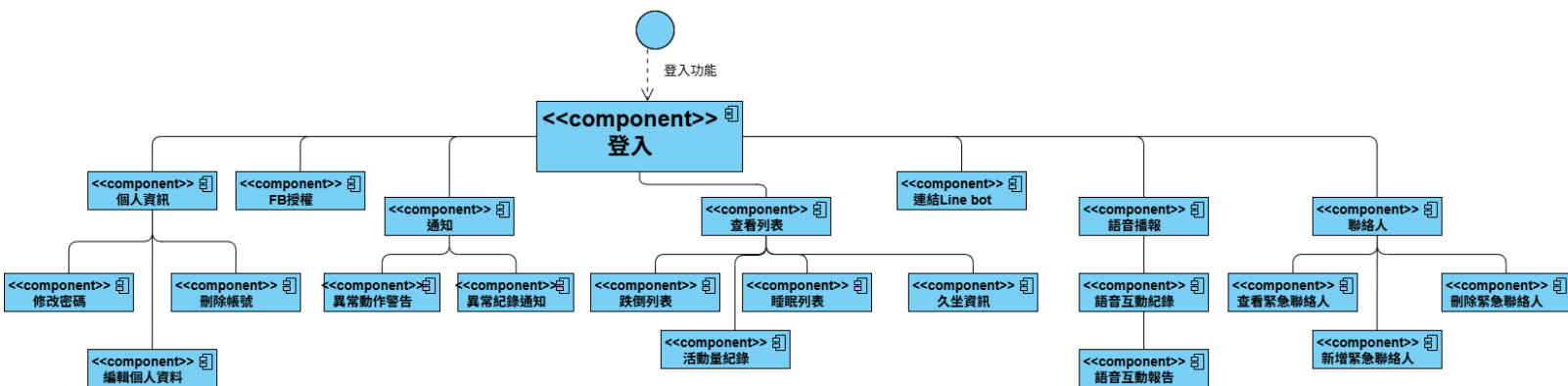


圖 7-8 元件圖

7-4 狀態機(State machine)

圖 7-4-1 為登入流程的狀態機，描述使用者從輸入帳號、電話、密碼到提交的過程中，各操作所經歷的狀態轉換。若驗證成功則進入「登入成功」狀態，若失敗則轉至「登入失敗」並可重新嘗試。此圖清楚呈現登入操作的動態流程。

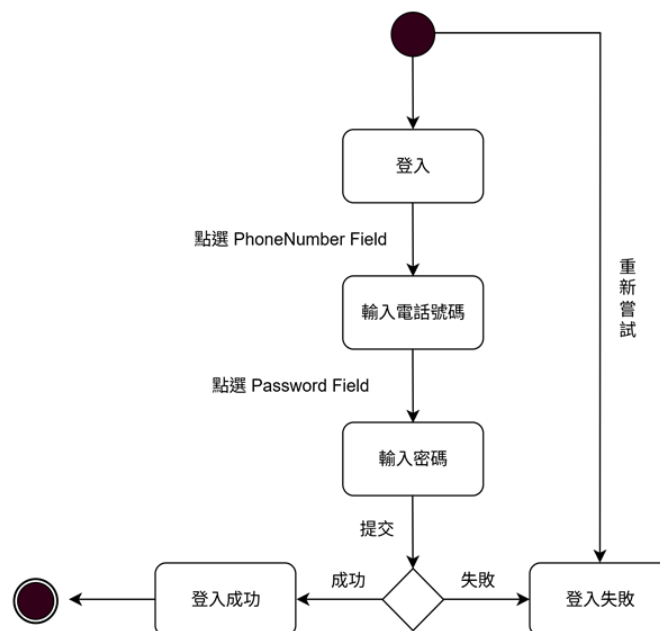


圖 7-9 狀態機-登入

圖 7-4-2 為帳號註冊流程的狀態機，描述使用者從點選註冊、依序輸入姓名、性別、電話、密碼與確認密碼，到提交資料的整體流程。若驗證成功則儲存資料並完成註冊，若失敗則顯示錯誤訊息並返回初始狀態。此圖清楚呈現註冊過程的狀態轉換與邏輯判斷。

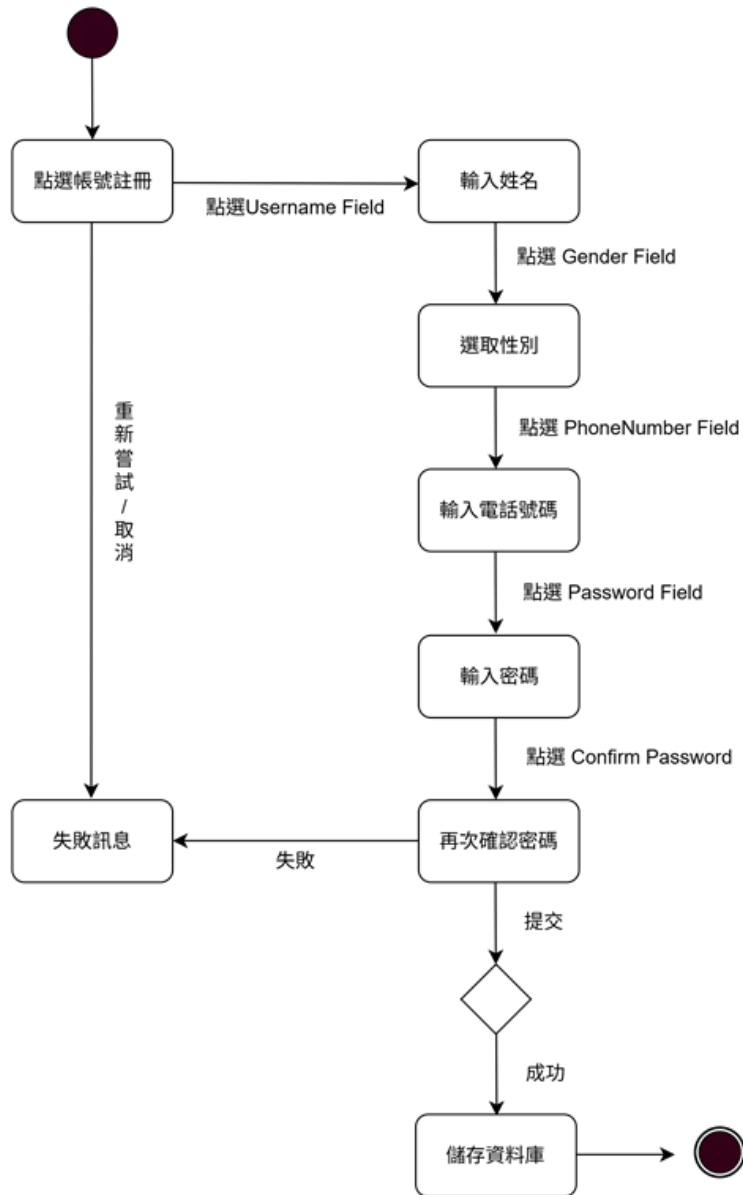


圖 7-10 狀態機-帳號註冊

圖 7-4-3 為系統偵測端的狀態機圖，描述使用者進入鏡頭畫面後，判斷長輩行為若有異常則上傳至伺服器，並將資料存在資料庫，最後會呈現在 App 端畫面供照護者看。

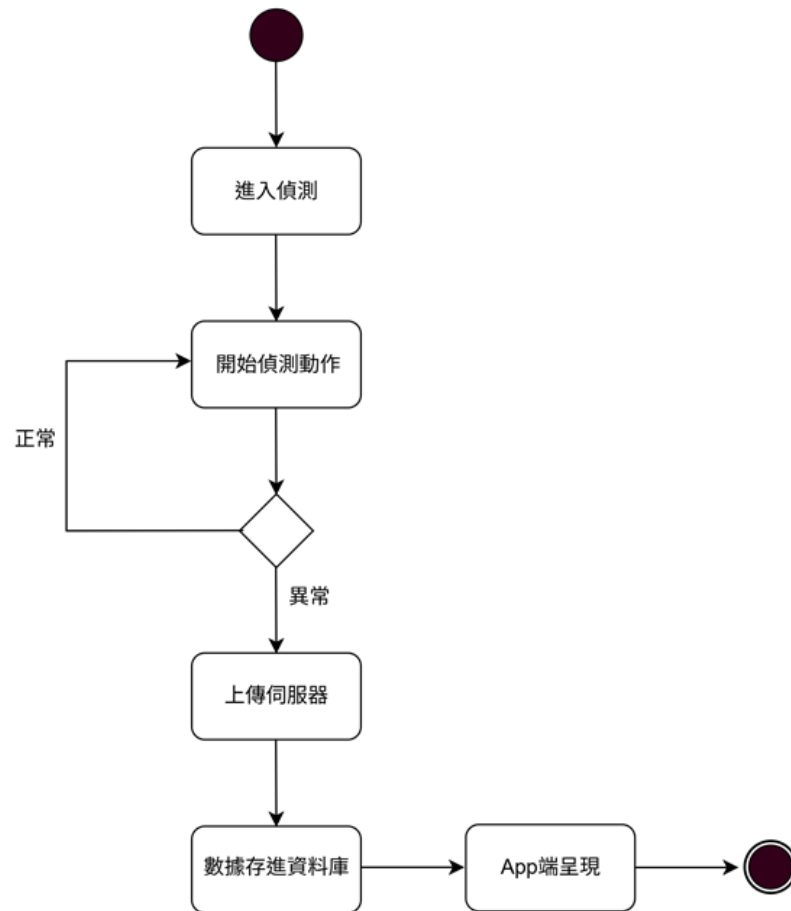


圖 7-11 狀態機-系統偵測端

圖 7-4-4 為更改密碼的狀態機，使用者依序輸入舊密碼、新密碼與確認新密碼。若舊密碼輸入正確，系統將檢查新密碼一致性與強度，不符則顯示錯誤訊息，驗證成功即更新資料庫並要求重新登入。

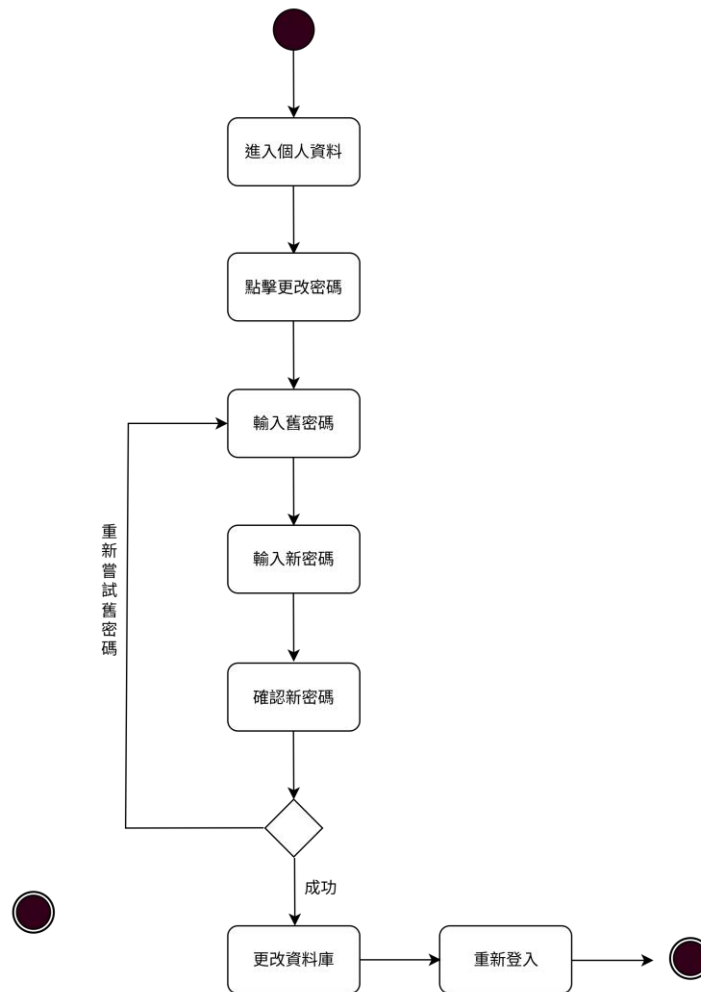


圖 7-12 狀態機-更改密碼

圖 7-4-5 為使用者進入清單頁，先選擇要查看的清單類別，再選擇查詢時間。有按鈕可以切換「按日」或「按週」模式，載入對應區間資料並顯示列表，讓照護者快速了解長輩的狀況。

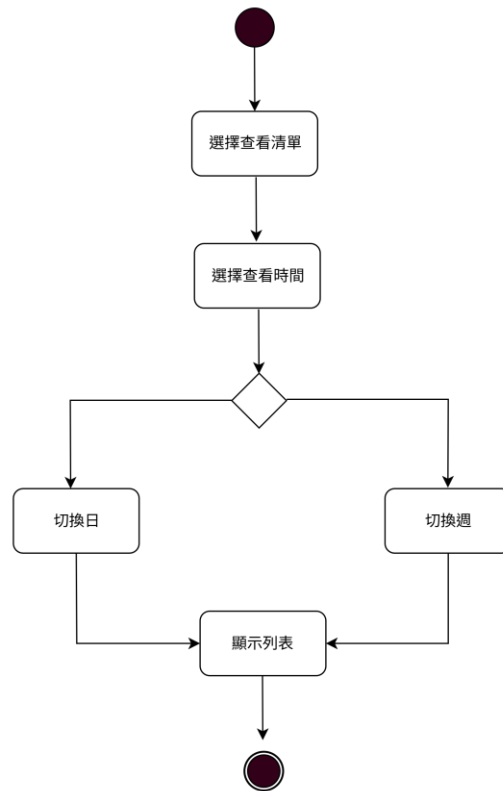


圖 7-13 狀態機-狀態清單查看

圖 7-4-6 系統啟動偵測後，先判別長輩是否開口說話；偵測到語音即啟動 STT 將聲音轉為文字，連同時間與身份傳至後端。後端解析、去雜訊與分類後寫入資料庫；App 端接收推送更新，在互動紀錄顯示文字與時間，並支援跨裝置同步。

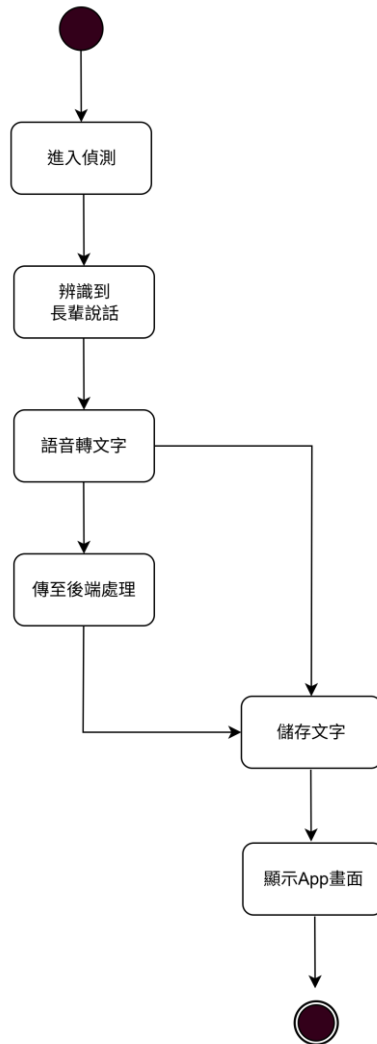


圖 7-14 狀態機-語音互動紀錄

圖 7-4-7 使用者進入緊急聯絡人頁面，可進行新增與刪除兩種操作。點擊新增，輸入關係與電話並送出，後端寫入成功即更新清單。刪除即左滑項目、再次確認後執行，資料庫移除並同步刷新列表。

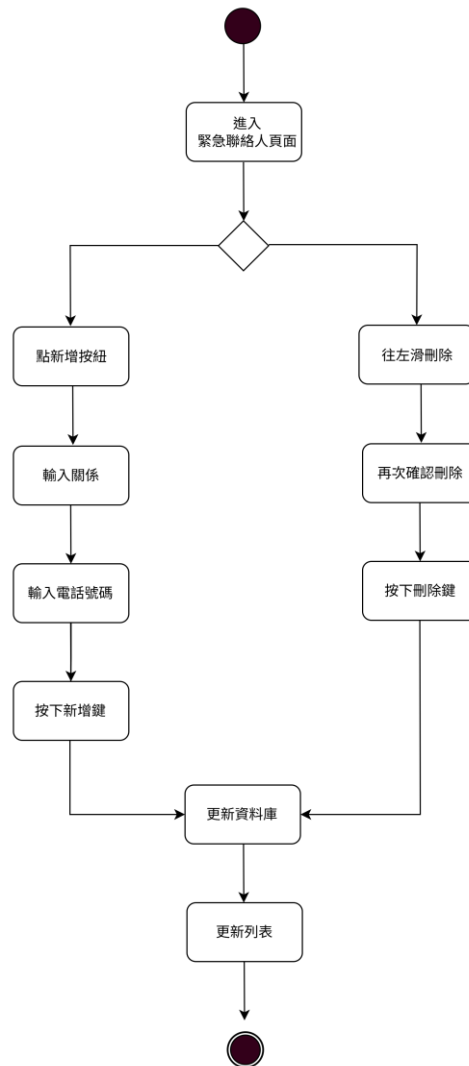


圖 7-15 狀態機-緊急連絡人新增及刪除

圖 7-4-8 為 Facebook 授權流程的狀態機圖，描述使用者於 App 端進行 Facebook 授權時的狀態變化。系統初始狀態為未授權，當使用者點擊「Facebook 授權」按鈕後，App 進入授權流程。若使用者尚未登入，系統會導向 Facebook 登入頁面；登入後使用者可選擇確認授權或取消授權。當使用者確認授權時，伺服器會進行驗證與 access token 交換，若驗證成功則轉為授權成功狀態，並完成授權綁定。若使用者取消授權或驗證過程發生錯誤，則轉為授權失敗或取消狀態，系統將回到未授權狀態等待下一次操作。

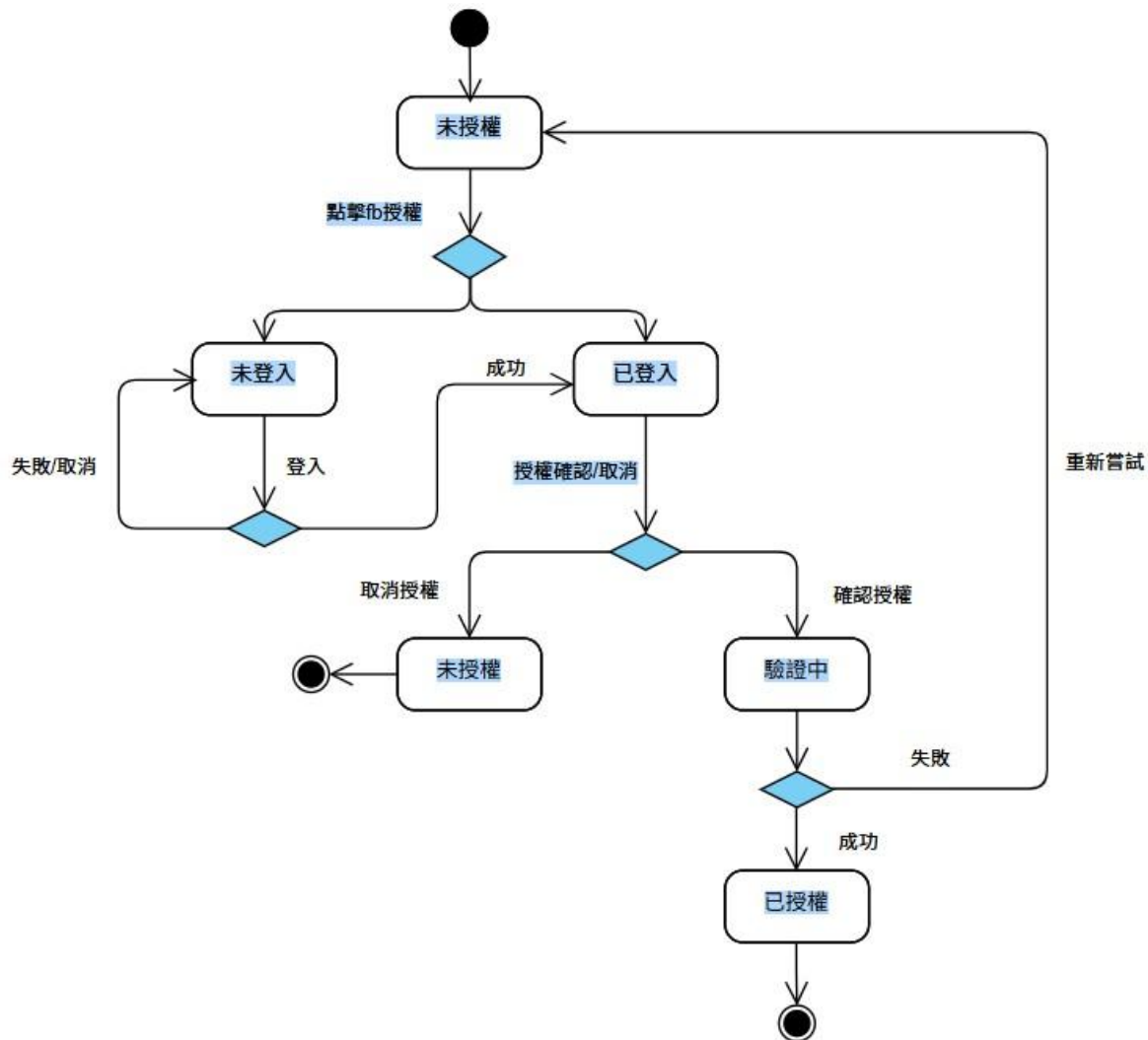


圖 7-16 狀態機-Facebook 授權

第8章 資料庫設計

8-1 資料庫關聯表

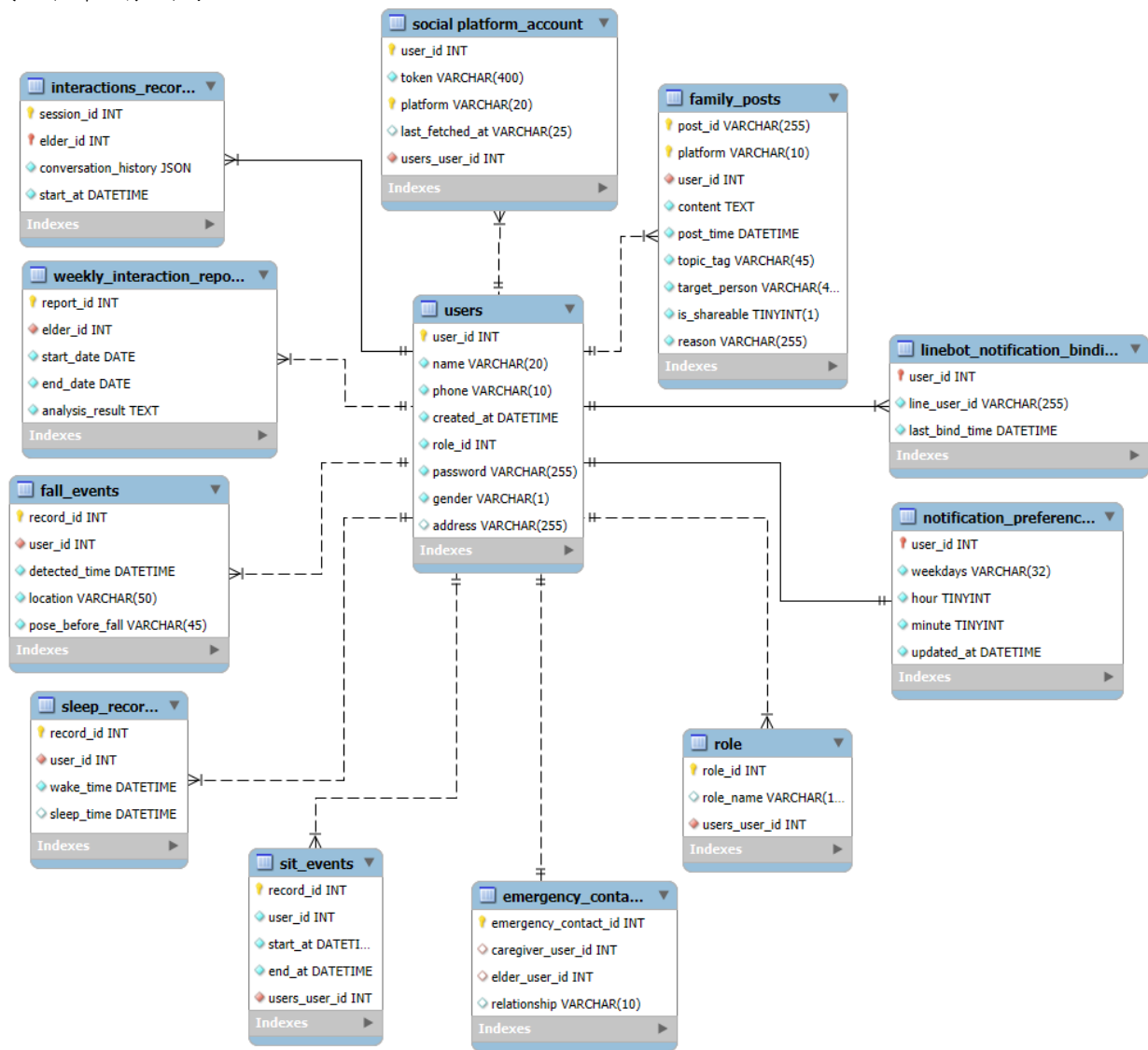


圖 8-1 MySQL 資料庫 ER 圖

圖 8-1-1 為本系統的 MySQL 資料庫 ER 圖，呈現各資料表的結構與關聯。整體以「照顧者／長者」為核心，透過使用者、角色定義身份與權限，並以照護者—長者關係（通知聯絡）供通知、內容、互動等模組做利用；事件行為模組（如跌倒事件、坐下行為紀錄、睡眠紀錄）、通知模組（Linebot 綁定）、內容模組（照護者社群貼文）、互動模組（AI 語音互動、週報）等，皆以外鍵連結至核心實體，此設計整合事件、互動、社群內容記錄，支援智慧照護系統的異常通知、陪伴與後續分析產生報告之需求。

8-2 表格及其 Meta data

表 8-2-1 資料表

資料表編號	資料表英文名稱	資料表中文名稱
01	users	使用者資料
02	role	角色資料
03	emergency_contacts	照護者聯絡資料
04	linebot_notification_binding	line 帳號綁定
05	notification_preferences	通知偏好設定
06	socialplatform_account	社交平台帳號資訊
07	family_posts	照護者社交貼文
08	fall_events	跌倒事件紀錄
09	sit_events	坐下行為紀錄
10	sleep_records	睡眠紀錄
11	interactions_records	AI 互動紀錄
12	weekly_interaction_reports	AI 互動報告

表 8-2-2 資料表描述-01 使用者資料

中文名稱	使用者資料		資料表編號		01	
英文名稱	users		主索引		user_id	
資料檔陳述	記錄系統中所有具有識別性的使用者					
欄位名稱	資料型態	長度	允許空值	中文	備註	預設值
user_id	INT		否	使用者編號	PK, AI	
role_id	INT		否	角色編號	FK	
name	VARCHAR	20	否	使用者姓名		
phone	VARCHAR	10	否	使用者電話	UNIQUE	
gender	VARCHAR	1	否	使用者性別		
address	VARCHAR	255	否	使用者地址		

password	VARCHAR	255	否	使用者密碼		
created_at	DATETIME		否	註冊時間		CURRENT_TIMESTAMP

表 8-2-3 資料表描述-02 角色資料

中文名稱	角色資料		資料表編號		02	
英文名稱	role		主索引		role_id	
資料檔陳述	定義系統中所有可辨識的使用者角色身分					
欄位名稱	資料型態	長度	允許空值	中文	備註	預設值
role_id	INT		否	角色編號	PK	
role_name	VARCHAR	10		角色名稱		

表 8-2-4 資料表描述-03 通知聯絡資料

中文名稱	通知聯絡資料		資料表編號		03	
英文名稱	emergency_contacts		主索引		emergency_contact_id	
資料檔陳述	紀錄長者的通知聯絡對象					
欄位名稱	資料型態	長度	允許空值	中文	備註	預設值
emergency_contact_id	INT		否	自建編號	PK、AI	
elder_user_id	INT			長者 ID	FK、 UNIQUE(elder_user_id, caregiver_user_id)	
caregiver_user_id	INT			照護者 ID	FK、 UNIQUE(elder_user_id, caregiver_user_id)	
relationship	VARCHAR	10		關係名稱		

表 8-2-5 資料表描述-04 line 帳號綁定

中文名稱	linebot 推播帳號綁定		資料表編號		04	
英文名稱	linebot_notification_binding		主索引		user_id	
資料檔陳述	紀錄照護者與 LINEBOT 通知服務的綁定資訊					
欄位名稱	資料型態	長度	允許	中文	備註	預設值

			空值			
user_id	INT		否	照護者 ID	PK、FK	
line_user_id	VARCHAR	255	否	LINE ID		
last_bind_time	DATETIME		否	最後綁定時間		

表 8-2-6 資料表描述-05 通知偏好設定

中文名稱	通知偏好設定		資料表編號		04	
英文名稱	notification_preferences		主索引		user_id	
資料檔陳述	紀錄照護者與 LINEBOT 通知服務的綁定資訊					
欄位名稱	資料型態	長度	允許空值	中文	備註	預設值
user_id	INT		否	照護者 ID	PK、FK	
weekdays	VARCHAR	32		星期		
hour	TINYINT			小時		
minute	TINYINT			分鐘		
updated_at	DATETIME		否	最新設定時間		CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

表 8-2-7 資料表描述-06 社交平台帳號資訊

中文名稱	社交平台帳號資訊		資料表編號		05	
英文名稱	socialplatform_account		主索引		(user_id, platform)	
資料檔陳述	記錄系統中照護者社交平台帳號資訊					
欄位名稱	資料型態	長度	允許空 值	中文	備註	預設值
user_id	INT		否	照護者 ID	PK	
platform	VARCHAR	20	否	社交平台	PK	
token	VARCHAR	400				

last_fetched_at	VARCHAR	25		擷取最新貼文時間		
-----------------	---------	----	--	----------	--	--

表 8-2-8 資料表描述-07 照護者社交貼文

中文名稱	照護者社交貼文		資料表編號		06	
英文名稱	family_posts		主索引		(post_id ,platform)	
資料檔陳述	紀錄照護者的社群平台貼文內容					
欄位名稱	資料型態	長度	允許空值	中文	備註	預設值
post_id	VARCHAR	255	否	貼文編號	PK	
platform	VARCHAR	10	否	社交平台	PK	
user_id	VARCHAR	45	否	照護者 ID	FK	
content	TEXT		否	貼文內容		
post_time	DATETIME		否	發文時間		
topic_tag	VARCHAR	45	否	貼文主題		
target_person	VARCHAR	45	否	描述對象		
is_shareable	TINYINT	1	否	適合分享與否		
reason	VARCHAR	255	否	不適合原因		

表 8-2-9 資料表描述-08 跌倒事件紀錄

中文名稱	跌倒事件紀錄		資料表編號		07	
英文名稱	fall_events		主索引		record_id	
資料檔陳述	紀錄長者跌倒事件					
欄位名稱	資料型態	長度	允許空值	中文	備註	預設值
record_id	INT		否	事件編號	PK、AI	
user_id	INT		否	長者 ID	FK	
detected_time	DATETIME		否	偵測時間		
location	VARCHAR	50	否	發生地點		
pose_before_fall	VARCHAR	45	否	跌倒前姿勢描述		

表 8-2-10 資料表描述-09 坐下行為紀錄

中文名稱	坐下行為紀錄		資料表編號		08	
英文名稱	sit_events		主索引		record_id	
資料檔陳述	長者坐下紀錄					
欄位名稱	資料型態	長度	允許空值	中文	備註	預設值

record_id	INT		否	紀錄編號	PK、AI	
user_id	INT		否	長者 ID	FK	
start_at	DATETIME		否	開始時間		
end_at	DATETIME	50		結束時間		null

表 8-2-11 資料表描述-10 睡眠紀錄

中文名稱	睡眠紀錄		資料表編號		09	
英文名稱	sleep_records		主索引		record_id	
資料檔陳述	紀錄長者起床事件					
欄位名稱	資料型態	長度	允許空值	中文	備註	預設值
record_id	INT		否	事件編號	PK、AI	
user_id	INT		否	長者 ID	FK	
wake_time	DATETIME		否	起床時間		
sleep_time	DATETIME			睡覺時間		null

表 8-2-12 資料表描述-11 AI 互動紀錄

中文名稱	AI 互動紀錄		資料表編號		10	
英文名稱	interactions_records		主索引		interaction_id	
資料檔陳述	紀錄長者與 AI 互動之紀錄					
欄位名稱	資料型態	長度	允許空 值	中文	備註	預設 值
session_id	INT		否	互動編號	PK、AI	
elder_id	INT	10	否	長者 ID	FK	
conversation_history	JSON	10	否	互動內容		
start_at	DATETIME		否	開始時間		

表 8-2-13 資料表描述-12 AI 互動報告

中文名稱	AI 互動報告		資料表編號		11	
英文名稱	weekly_interaction_reports		主索引		report_id	
資料檔陳述	紀錄長者與 AI 互動之紀錄					
欄位名稱	資料型態	長度	允許空 值	中文	備註	預設 值
report_id	INT		否	報告編號	PK、AI	
elder_id	INT		否	長者 ID	FK	
start_date	DATETIME		否	開始時間		
end_date	DATETIME		否	結束時間		
analysis_result	TEXT		否	報告內容		

第9章 程式

9-1 元件清單及其規格描述

表 9-1-1 元件清單與規格描述表(後端)-資料庫(MySQL)

Json 檔案(後端)-資料庫(MySQL)		
編號	檔案名稱	功能
	OAuth_Authorization.json	FB 社群授權
	SocialPosts_Parentworkflow.json SocialPosts_subworklow.json	FB 社群貼文抓取與處理
	Interaction_Analytics.json	互動紀錄分析報告

表 9-1-2 元件清單與規格描述表(後端)- API (Routes)

Python 檔案 (後端) — API (Routes)		
編號	檔案名稱	功能
2-2-1	auth_routes.py	登入/驗證、取得使用者資訊
2-2-2	user_routes.py	使用者基本資料 CRUD
2-2-3	line_routes.py	LINE 綁定、Quick Reply、Webhook 互動
2-2-4	emergency_contacts_routes.py	緊急聯絡人 CRUD
2-2-5	fall_event_routes.py	跌倒事件查詢/新增 (含回放連結)
2-2-6	sit_event_route.py	坐下/起身等姿態事件 API
2-2-7	gait_routes.py	步態/姿勢分析相關 API
2-2-8	pose_routes.py	姿態資料上傳/測試
2-2-9	sleep_records_routes.py	睡眠紀錄查詢/上傳
2-2-10	weekly_interaction_reports_routes.py	每週互動報告查詢
2-2-11	news_voice_routes.py	產生新聞語音/播報相關 API
2-2-12	reels_routes.py	影片/短片列表與播放資訊
2-2-13	ws_test_routes.py	WebSocket 測試端點
2-2-14	debug_routes.py	一般除錯與內部測試端點
2-2-15	debug_borrowers.py	特定場景的除錯/假資料端點

表 9-1-3 元件清單與規格描述表(後端)- Service (業務層)

Python 檔案 (後端) — Service (業務層)		
編號	檔案名稱	功能
2-3-1	user_service.py	使用者邏輯、權限檢查
2-3-2	role_id_service.py	角色 ID 與權限對應

2-3-3	elder_context_service.py	照護者↔長者關係查詢與快取
2-3-4	emergency_contacts_service.py	緊急聯絡人業務流程
2-3-5	fall_event_service.py	跌倒事件寫入、查詢與關聯處理
2-3-6	sit_event_service.py	坐下/起身事件整合與門檻設定
2-3-7	sleep_records_service.py	睡眠紀錄彙整與查詢
2-3-8	weekly_interaction_reports_service.py	每週互動報告彙整產製
2-3-9	weekly_report_push_service.py	週報推播排程與送出
2-3-10	notify_prefs_service.py	推播偏好（weekday/時間）維護
2-3-11	notify_line_service.py	LINE 推送訊息（文字/卡片）
2-3-12	line_auth_service.py	LINE OAuth/驗證流程
2-3-13	line_binding_service.py	LINE 使用者綁定/解綁
2-3-14	line_richmenu_service.py	LINE Rich Menu 管理

表 9-1-4 元件清單與規格描述表(後端)- 資料層（DAO / DB）

Python 檔案（後端）－資料層（DAO / DB）		
編號	檔案名稱	功能
2-4-1	db.py	連線池設定、交易與連線管理
2-4-2	caregiver_elder_dao.py	照護者-長者關係查詢
2-4-3	users_dao.py	使用者資料表 CRUD
2-4-4	role_dao.py	角色表查詢
2-4-5	emergency_contacts_dao.py	緊急聯絡人資料存取
2-4-6	fall_events_dao.py	跌倒事件資料寫入/查詢
2-4-7	sit_event_dao.py	坐下/姿態事件資料存取
2-4-8	sleep_records_dao.py	睡眠紀錄存取
2-4-9	weekly_interaction_reports_dao.py	互動報告資料存取
2-4-10	weekly_report_push_dao.py	週報推播佇列/紀錄
2-4-11	line_binding_dao.py	LINE 綁定映射 (user_id↔line_user_id)
2-4-12	notify_line_dao.py	LINE 推播訊息紀錄

2-4-13	notify_prefs_dao.py	推播偏好表 CRUD
2-4-14	exceptions.py	統一例外型別/錯誤碼定義

表 9-1-5 元件清單與規格描述表(後端)- 即時推論與 WS (核心 Utils)

Python 檔案 (後端) — 即時推論與 WS (核心 Utils)		
編號	檔案名稱	功能
2-5-1	stream_infer_manager.py	串流推論協調：收幀→前處理→二階推論→事件時間線→Hook
2-5-2	binary_cnn_lstm_loader.py	二元 (fall / non_fall) 模型載入與推論
2-5-3	multi_cnn_lstm_loader.py	多類別 (sit/lie/walk...) 模型載入與推論
2-5-4	ws_connection_manager.py	以 user_id 管理 WS 連線、廣播與錯誤處理
2-5-5	ws_message_dispatcher.py	WS 訊息分類與路由 (inbound/outbound)

表 9-1-6 元件清單與規格描述表(後端)- 通用工具 (Utils)

Python 檔案 (後端) — 通用工具 (Utils)		
編號	檔案名稱	功能
2-6-1	cnn_lstm_utils.py	relation-map 產生、motion(9) 計算、mask
2-6-2	kalman_filter.py	姿態關鍵點平滑與低 FPS 升頻策略
2-6-3	news_utils.py	新聞抓取/整理與語音化輔助
2-6-4	response_util.py	統一 API 回應格式 (code/msg/data)
2-6-5	time_utils.py	UTC↔Asia/Taipei 轉換、時間字串工具
2-6-6	paths.py	專案常用路徑常數 (SC_ROOT/SC_MODELS...)

表 9-1-7 元件清單與規格描述表 (模型資源)

Python 檔案 (後端) — (模型資源)		
編號	檔案名稱	功能

2-7-1	models/binary/best.pt	二元模型權重
2-7-2	models/binary/classes.json	二元類別清單（順序/名稱）
2-7-3	models/multi/best.pt	多類別模型權重
2-7-4	models/multi/classes.json	多類別類別清單

表 9-1-8 元件清單與規格描述表（啟動與環境）

Python 檔案（後端）－（啟動與環境）		
編號	檔案名稱	功能
2-8-1	start.sh	啟動腳本（匯入環境、啟動 Uvicorn）
2-8-2	run.py	應用程式入口、路由掛載與初始化
2-8-3	.env	環境變數（DB、LINE、推論門檻等；文件以遮敏版示例）

表 9-1-9 訓練與資料前處理（離線）

Python 檔案（後端）－（離線）		
編號	檔案名稱	功能
2-9-1	binary_cnn_lstm_trainer.py	訓練二元跌倒偵測模型，輸出 models/binary/best.pt
2-9-2	multi_cnn_lstm_trainer.py	訓練多類別動作辨識模型，輸出 models/multi/best.pt

表 9-1-10 Json 檔案(後端)-邊緣運算

Json 檔案(後端)-邊緣運算		
編號	檔案名稱	功能
1-1-1	Flow.json	根據輸入格式決定子工作流
1-1-2	Voice_Agent.json	語音助手
1-1-3	Analyze.json	骨架偵測
1-1-4	supabaseDB.json	SQL 資料存入 supabaseDB

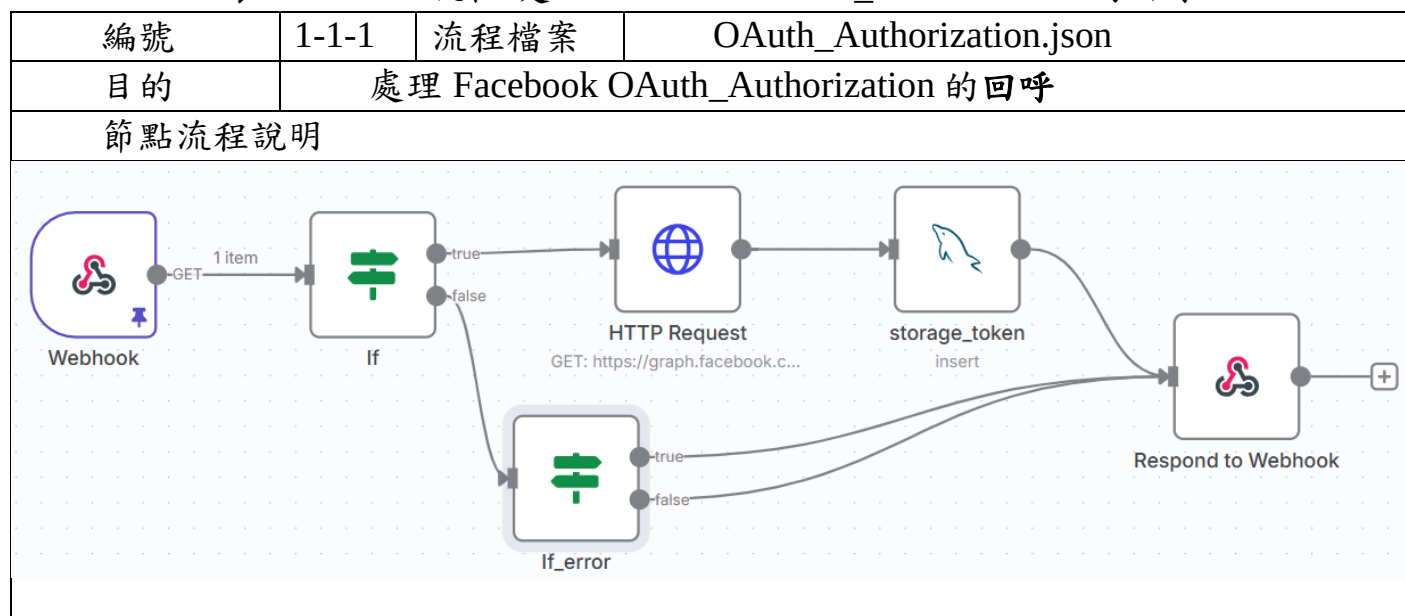
表 9-1-11 元件清單與規格描述表(前端)

Android 檔案(前端)-App 介面與邏輯			
編號	Kotlin	xml	功能
1-2-1	LoginActivity.kt RegisterActivity.kt	activity_login.xml activity_register.xml	登入及註冊
1-2-2	DashboardActivity.kt	activity_dashboard.xml	系統首頁

1-2-3	EditProfileActivity	activity_edit_profile.xml	個人頁面，進入能夠更改/刪除密碼、編輯資料及加入 Line 接收通知；FB 授權子女貼文播報
1-2-4	ChangePasswordActivity.kt	activity_change_password.xml	
1-2-5	無	dialog_select_elder.xml	選擇長輩卡片元件
1-2-6	VideoListActivity.kt	activity_video_list.xml	狀態列表頁面，可以查看長輩跌倒、互動、睡眠及久坐清單
1-2-7	VoiceResultActivity.kt	activity_voice_result.xml	語音互動紀錄頁面
1-2-8	EmergencyContactsActivity.kt	activity_emergency_contacts.xml	緊急聯絡人頁面

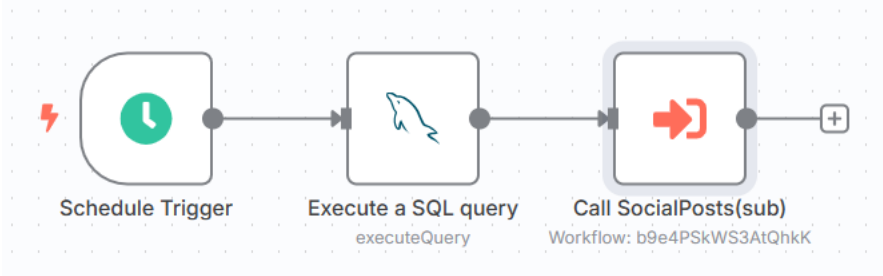
9-2 其他附屬之各種元件

表 9-2-1 n8n 流程-處理 Facebook OAuth_Authorization 的回呼



1. Webhook (redirect_uri) - 於 redirect_uri (Webhook) 使用 GET Method 接收 code/error - Respond : Using 'Respond to Webhook' Node
If (是否可嘗試換 token) 為 true
- 條件：有 code 且沒有 error HTTP Request - 使用 code 換取 access_token - 設定： Method : GET URL : https://graph.facebook.com/v23.0/oauth/access_token Send Query Parameters : client_id、client_secret、redirect_uri、code storage_token - 保存 access_token 於 mysql
If (是否可嘗試換 token) 為 false
If_error 條件：error 為 access_denied，true 表示授權取消；false 表示其他錯誤
Respond to Webhook - 以 html 將授權結果回應瀏覽器的頁面 - 設定： Response Code : 200 Response Headers : 1.Content-Type: text/html; charset=utf-8 2.Referrer-Policy: no-referrer 3.Cache-Control: no-store Response Body : html

表 9-2-2 n8n 流程- FB 社群貼文抓取與處理貼文

編號	1-1-2	流程檔案	SocialPosts_Parentworkflow.json SocialPosts_subworklow.json
目的	FB 社群貼文抓取與處理貼文		
節點流程說明			
Parent-workflow			
 <pre>graph LR; A[Schedule Trigger] --> B[Execute a SQL query executeQuery]; B --> C[Call SocialPosts(sub) Workflow: b9e4PSkWS3AtQhkk]; C --> D[+];</pre>			

1.Schedule Trigger

- 設定固定每天 00:00 啟動此 workflow 流程

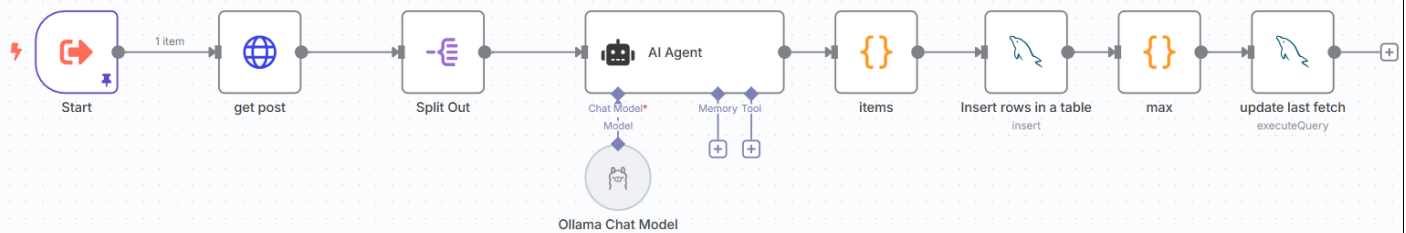
2.Execute a SQL query (executeQuery)

- 到資料庫抓所有照顧者社群帳號名單
- SQL：SELECT * FROM `114-402`.`social platform\_account`；
- 輸出：多筆 item，每筆包含 user_id / platform / access_token / last_fetched_at。

3.Call 處理貼文 (Call Sub-workflow)

- 把上一步取得的每一筆 item 丟到 subworkflow 個別處理。

Sub-workflow



1.Start

接收 parentworkflow 丟來的一筆 item (user_id/platform/access_token/last_fetched_at)。

2.get post (HTTP Request)

- 呼叫 FB API 依 access_token 與 last_fetched_at 抓取貼文。

3.Split Out

- 把前一節點的貼文陣列拆成個別 item，方便逐筆處理。

4.AI Agent (Ollama Chat Model)

- 對每則貼文做前處理
- 輸入：前一節點貼文
- 輸出：JSON 格式

```
{
  "user_id": "{{ $('Start').item.json.user_id }}",
  "platform": "{{ $('Start').item.json.platform }}",
  "topic_tag": "",
  "target_person": "",
  "is_shareable": 1,
  "reason": "",
  "context": "{{ $json.message }}",
  "created_time_tw":
    "{{ DateTime.fromISO($json.created_time).setZone('Asia/Taipei').toFormat('yyyy-LL-dd HH:mm:ss') }}",
  "post_time": "{{ $json.created_time }}"
  "platform_post_id": "{{ $json.id }}"
}
```

}

1. items (Function)

- 來源是 item.json.output(前一節點)，內容常被 LLM 包在 ``json ... ``。
- 步驟：
 1. 移除 ``json 與 ``。
 2. JSON.parse 轉成物件。
 3. 回傳 { json: parsed }，讓後續 Insert/HTTP 直接取用欄位。

2. Insert rows in a table

- 每筆前處理結果寫入 mysql

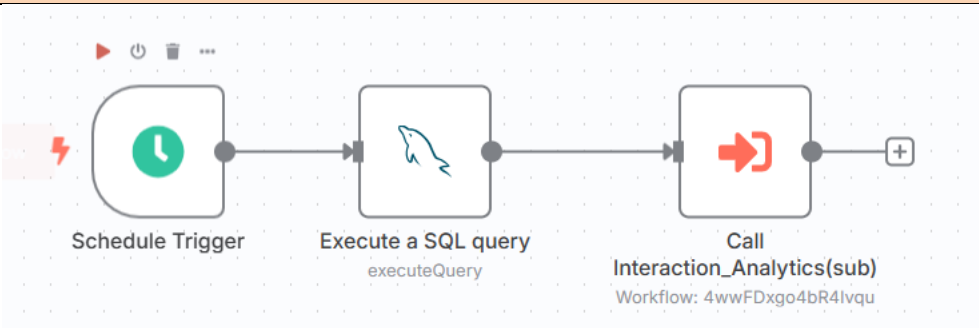
3. max (Function)

- 從上一個 items 的節點抓到的多筆資料中，找出最大的 post_time，然後只回傳一筆包含這個最大時間的 item，給後續更新 last_fetched_at 用

4. update last_fetched_at (executeQuery)

- 更新此人社群帳號的 last_fetched_at，以便下次只抓取新貼文

表 9-2-3 n8n 流程- 分析互動紀錄產生週報

編號	1-1-3	流程檔案	Interaction_Analytics_Parentworkflow.json Interaction_Analytics_Subworkflow.json
目的	分析互動紀錄產生週報		
節點流程說明			
Parent-workflow			
			

1. Schedule Trigger

- 設定每周固定時間 00:00 啟動此 workflow 流程

2. Execute a SQL query

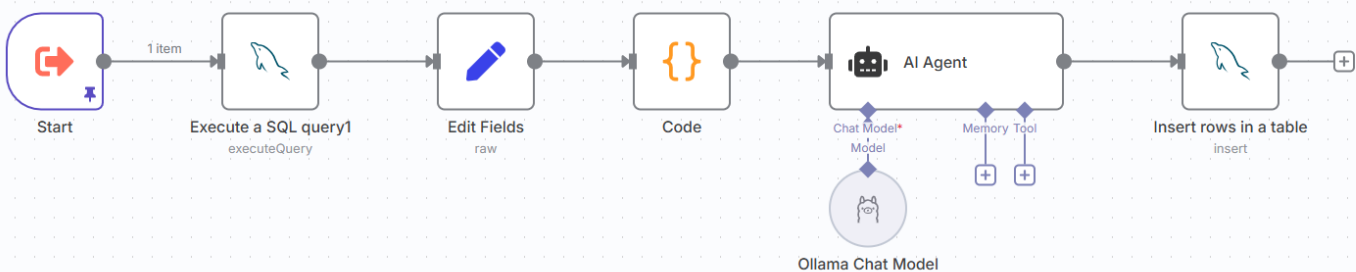
- 撈出「最近 7 天內有互動紀錄的長者清單」，並同時帶出查詢時間區間
- SQL:

```
SELECT DISTINCT elder_id,  
DATE_SUB(CURDATE(), INTERVAL 7 DAY) AS from_ts,  
CURDATE() AS to_ts  
FROM `114-402`.`interactions_records`  
WHERE start_at >= DATE_SUB(CURDATE(), INTERVAL 7 DAY)  
AND start_at < CURDATE();
```

3. Call Interaction_Analytics(sub)

- 前一節點每筆 item 各自跑子流程 Interaction_Analytics(sub)

Sub-workflow



1. Start

- 接收 parentworkflow 丟來的一筆 item (elder_id / from_ts / to_ts)。

2. Execute a SQL query1

- 取得個別長者 elder_id 在前 7 天的互動紀錄
- SQL :

```
SELECT *  
FROM `114-402`.`interactions_records`  
WHERE elder_id = {{ $json.elder_id }}  
AND start_at >= '{{ $json.from_ts }}'  
AND start_at < '{{ $json.to_ts }}'  
ORDER BY start_at;
```

3. Edit Fields

- 將原始欄位中的對話紀錄 conversation_history 中元素{A,Q}正規化為 messages = {Q,A}
- 輸入：上一節點的每筆 row，conversation_history 為陣列或 JSON 字串，元素是 {Q,A}

- 處理：若是字串就 JSON.parse，兼容 Q、A，清掉空白，輸出固定 {Q,A} 結構

4. Code (Function)

- 將多筆 session 合併成一筆 sessions，並依時間排序，方便後續 ollama 分析

- 輸入：多筆 items（至少有 elder_id / session_id / start_at / messages）。

- 處理：

1. normalizeMessages()：支援三種來源型態，統一成 {Q,A} 陣列。
2. 依時間 start_at 排序
3. 將每筆 row 視為一個 session，抽出 session_id / session_start / messages。
4. 組合整體 elder_id 與 range = {start, end}（用第一筆與最後一筆時間）。

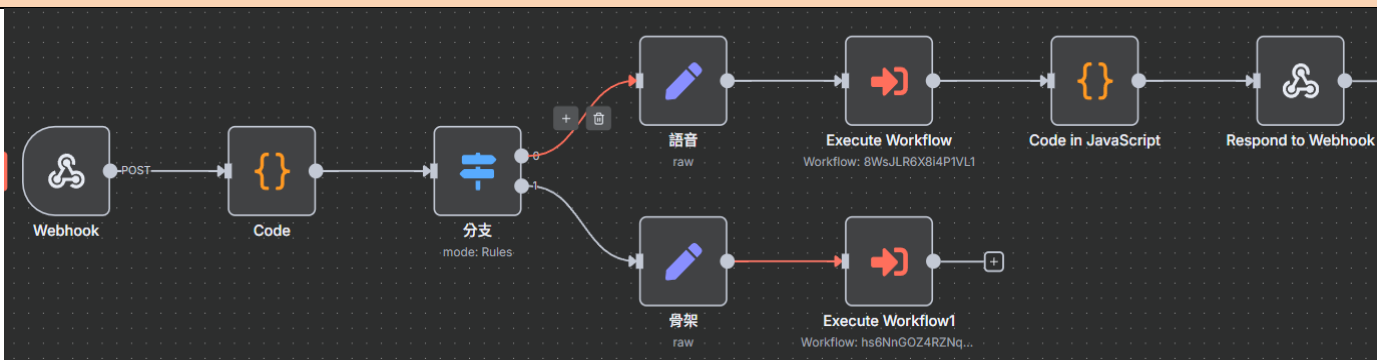
5. AI Agent (Ollama Chat Model)

- 對 7 天互動內容作分析
- 輸入：前一節點
- 輸出：以 JSON 輸出 3-5 句自然、溫和的中文語句，摘要長者一週的整體狀態與建議，只需總體報告禁止展開說明

6. Insert rows in a table

- 每筆分析報告結果寫入 mysql

表 9-2-4 n8n 流程- 將收到的 json 依據格式不同執行不同子工作流

編號	1-1-4	流程檔案	Flow.json
目的	將收到的 json 依據格式不同執行不同子工作流		
節點流程說明			
Flow			
 <pre>graph LR; Webhook[Webhook] -- POST --> Code[Code]; Code --> Branch[分支 mode: Rules]; Branch -- "+" --> Voice[語音 raw]; Branch -- "1" --> Skeleton[骨架 raw]; Voice --> ExecuteWorkflow[Execute Workflow Workflow: 8WsJLR6X8i4P1VL1]; Skeleton --> ExecuteWorkflow1[Execute Workflow1 Workflow: hs6NnGOZ4RZNq...]; ExecuteWorkflow --> CodeJS[Code in JavaScript]; ExecuteWorkflow1 --> CodeJS; CodeJS --> Respond[Respond to Webhook];</pre>			

1. **webhook**

- 接收來自前端的 TTS(語音轉文字) 或骨架 json

範例：{

```
"body": {  
  "elder_id": "1",  
  "session_id": "1",  
  "text": "早安"}  
}
```

2. **code**

- 根據資料輸入格式來設定 method=talk 或 detect，並輸出 json

3. **分支 Switch**

- 根據上個節點輸出的 method 走對應的流程

甲、語音流

i. Set field(語音)

1. 格式化輸出 json：{"elder_id": "1","session_id": "17","text": "我兒子的近況"}

ii. Execute Workflow

1. 執行「語音助手」工作流，並輸出給下一個節點
2. 格式：

```
{  
  "conversation_history": ,  
  "current_qa": {"Q": " ", "A": },  
  "encoded_user_text": ,  
  "encoded_ai_text":  
}
```

iii. Code in JavaScript

1. 確保收到的音訊檔案資料完整，並為其設定標準的檔案名稱和格式，供前端使用
2. 防子流程錯誤

iv. Respond to Webhook

1. 回傳結果

乙、骨架流

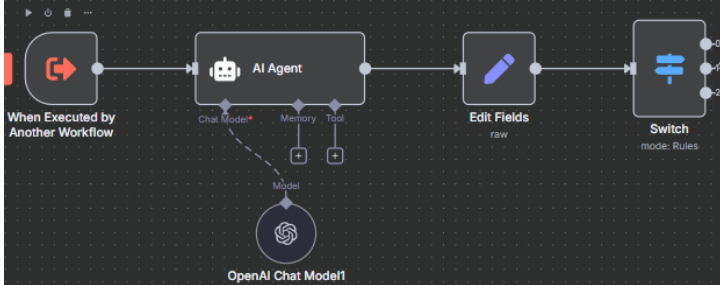
i. Set Field(骨架)

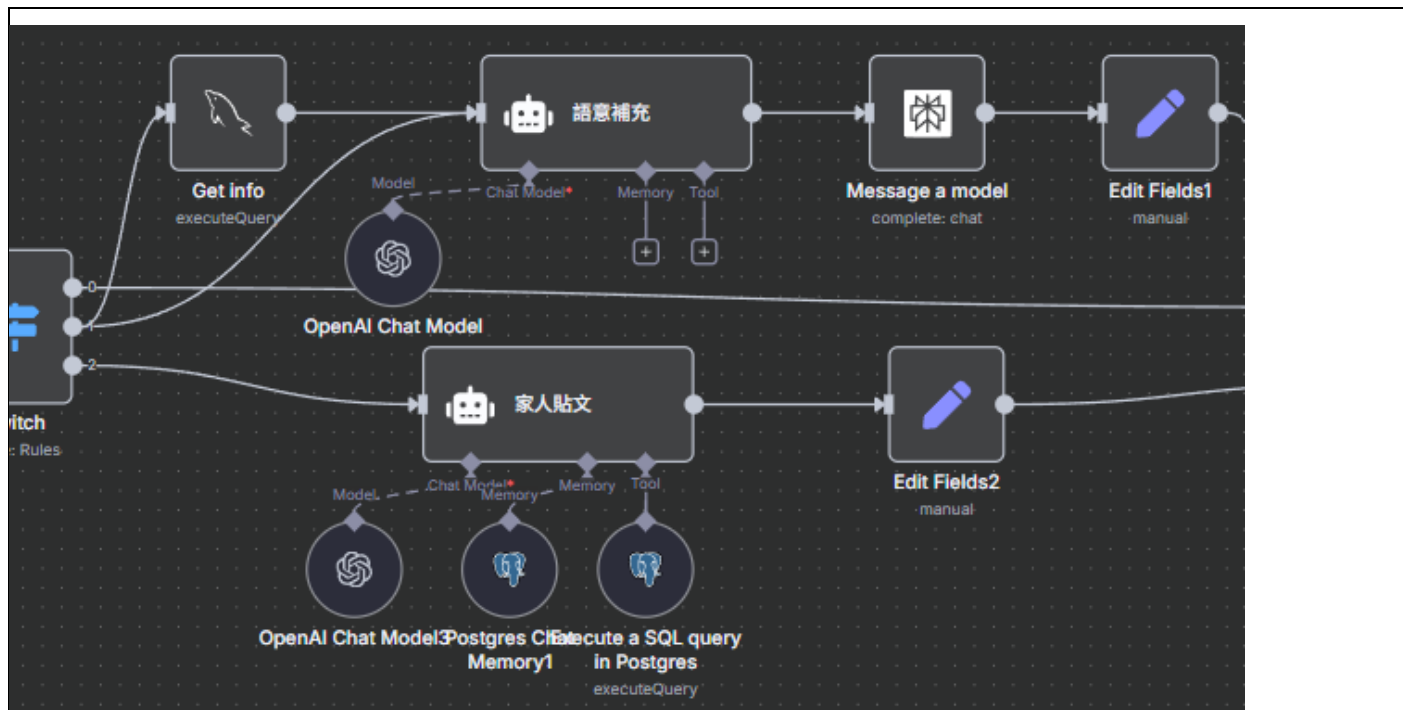
1. 從上游 Code 節點傳來的資料中，只挑選出 elder_id, session_id, frames, start, end, dets，並放入新的 JSON 物件中

ii. Execute Workflow1

1. 執行「骨架分析 v4」子工作流

表 9-2-5 n8n 流程- 自然語言理解

編號	1-1-5	流程檔案	Voice_Agent.json
目的	自然語言理解、情境判斷和擬人化的口語輸出，並將所有互動記錄下來，以便未來進行分析		
節點流程說明			
Voice_Agent part1			
			
一、 關鍵字識別及分派 1.When Executed by Another Workflow - 接收包含使用者文字 (text)、使用者 ID (elder_id) 和對話 ID (session_id) 的請求 2.AI Agent - 分析使用者的 text，並強制輸出 search、chat 或 family_post 三個關鍵字之一 - 模型: gpt-4o 3.Switch - 根據 AI Agent 輸出的意圖關鍵字，決定接下來要走哪一條處理路徑 (0: chat, 1: search, 2: family_post)			
節點流程說明			
Voice_Agent part2			



二、使用者問題核心處理

1. 路徑 1: search (資訊查詢)

二.1.1 Get info

- 從 MySQL 資料庫中查詢使用者的姓名和預設地址 (如 "台北市士林區")

二.1.2 AI Agent (語意補充)

- 結合使用者問題和其地址，生成一句精確的搜尋指令
- 例如，將 "附近有什麼好吃的" 轉換為 "台北市士林區附近有什麼好吃的"

二.1.3 Message a model (Perplexity)

- 執行網路搜尋任務

二.1.4 Edit Field1

- 格式化輸出
- JSON 格式 :

`{"elder_id": , "session_id": , "text": , "A": , "output": }`

2. 路徑 2: family_post (家庭動態)

二.2.1 AI Agent (家人貼文)

- 將使用者的口語化查詢 (如 "我弟最近發了什麼") 轉換為精確的 SQL 查詢語句
- Execute a SQL query in Postgres (Postgres 工具): 執行 AI 生成的 SQL 語句，從 family_posts 表中撈取資料。
- Postgres Memory：讀取存在 postgresDB 的 family_post 作為參照

二.2.2 Edit Field2

- 格式化輸出

- JSON 格式 :

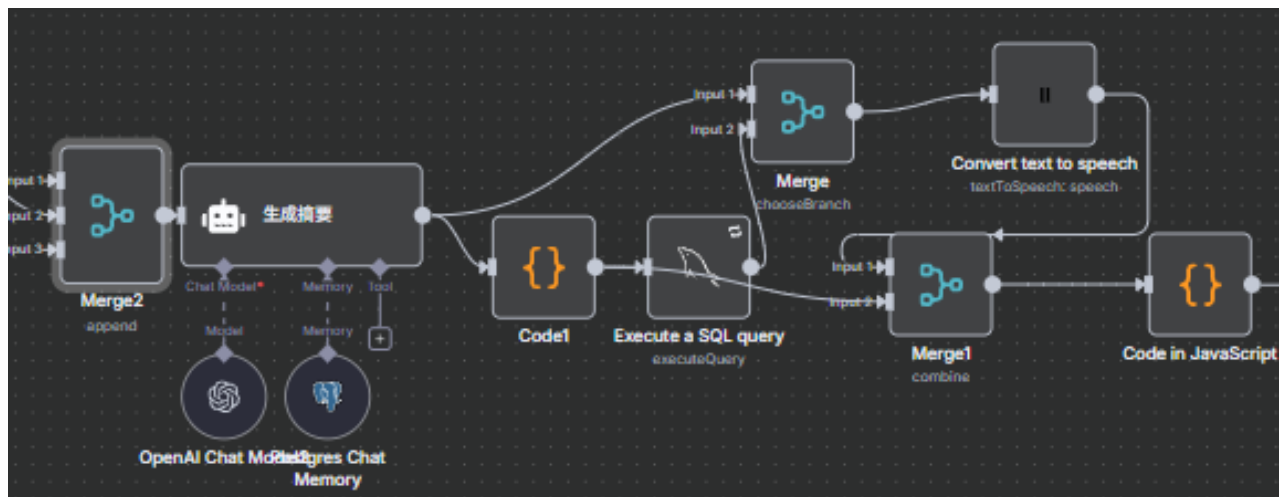
```
{"elder_id": , "session_id": , "text": , "A": , "output": }
```

3. 路徑 3: chat (閒聊)

二.3.1 直接將使用者的輸入傳遞到下一階段

節點流程說明

Voice_Agent part3



三、 整合回應與記錄

1. Merge2

- 將三條路徑的處理結果匯集到一個統一的流程中(無論哪條路徑被執行)

2. Postgres Chat Memory

- 從 postgresDB 裡尋找跟使用者(elder_id)相關的最近幾次對話歷史

3. AI Agent (生成摘要)-核心

- 設定了詳細的角色扮演指令 (prompt)，要求它扮演一位名為「小金」的智慧夥伴
- 模型: gpt-4o (使用了較高的 temperature 以增加創意和人性化)

4. Code1

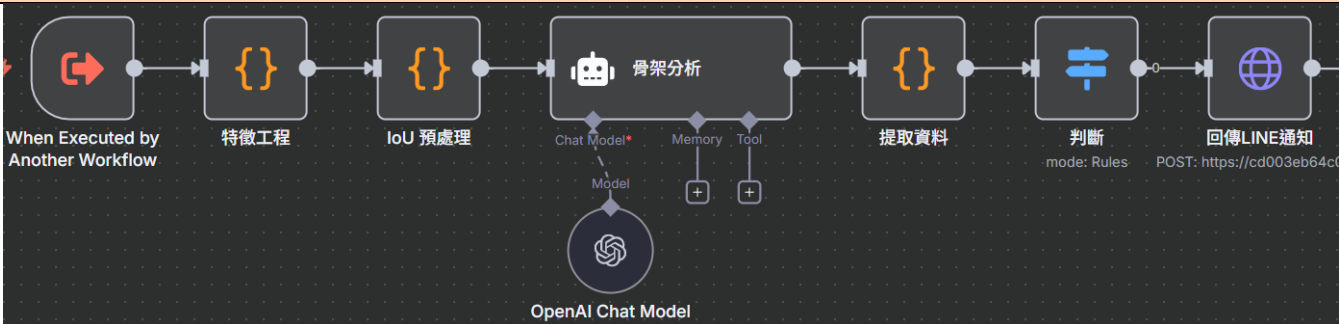
- 從「生成摘要」節點獲取 AI 的最終回覆 (A) 和使用者的原始問題 (Q)，並將它們與 elder_id、session_id 等資訊打包成一個符合資料庫格式的 JSON 物件

5. Execute a SQL query

- 如果是一個新的 session_id，就在 interactions_records 表中創建一筆新的對話記錄

- 如果 session_id 已存在，則將本次的 Q&A 附加到 conversation_history 欄位中
6. Convert text to speech
 - 調用 ElevenLabs 的服務，將「生成摘要」節點產出的最終文字稿，轉換為 MP3 格式的語音檔
 7. Code in JavaScript
 - 確保 MP3 檔案 (binary data) 的格式正確無誤（如檔名、MIME 類型），並將所有需要回傳給前端的資訊（如使用者問題、AI 回覆的文字、語音檔案等）整理成最終的輸出格式

表 9-2-6 將收到的 json 依據格式不同執行不同子工作流

編號	1-1-6	流程檔案	Analyze.json
目的	將收到的 json 依據格式不同執行不同子工作流		
節點流程說明			
Flow			
			
1. When Executed by Another Workflow • 接收來自主流程的 frames 陣列 範例：{ "type": "pose", "tag": "start/end", "frame_id": 42, "ts_ms": 1759508034919, "bbox": [100,100,100,100], "keypoints": [[257,288],[256,283],[254,281],[242,280],[235,278],[235,312],[211,314], [231,365],[199,375],[242,405],[220,423],[222,413],[205,413],[223,485], [200,485],[193,526],[163,529]] } 2. Code (特徵工程) • 提取首尾幀：找出代表動作開始與結束的骨架幀 • 計算基準尺度 (S0)：以起始幀的肩寬作為後續計算的相對單位			

- 計算姿態特徵
 - 計算軀幹角度變化 (dtheta)
 - 計算質心位移 (com_dx_s, com_dy_s)
 - 計算手腕、腳踝等關節的最大位移等 feature

3. Code (IoU 預處理)

- 提取人體邊界框 (Person BBox)：根據結束幀的骨架關節點，計算出能完整包覆人體的最小矩形邊界框
- 提取環境物件：從輸入資料中，找出所有已標記的、感興趣的環境物件（如 chair, bed, sofa, table 等）及其邊界框
- 計算空間關係特徵：
 - 最大交並比 (iou_max)：計算「人體邊界框」與所有「環境物件邊界框」之間的最大重疊率 (Intersection over Union)。一個高的 IoU 值通常意味著人正躺在或坐在該物件上
 - 鄰近狀態 (near_chair, near_bed ...)；
 - 定義一個動態的「鄰近」閾值，此閾值會根據人體在畫面中的大小進行調整
 - 判斷人體是否與椅子、床等關鍵物件「鄰近」(IoU 超過閾值)

4. AI agent(骨架分析)

- 接收所有特徵：姿勢特徵 + 環境關係特徵
- 執行綜合判斷：AI 根據其 System Prompt 中定義的規則進行決策
 - com_dy_s 很大，但是 iou_max 也很高 且 near_bed 為 true → 不是跌倒，而是躺在床上
 - com_dy_s 很大，且 iou_max 很低 → 很可能是跌倒在地板上
 - AI 按照指定的 JSON 格式輸出，例如：{"elder_id":"123","status":"跌倒"}
 - 使用 chatgpt-4o 模型 (gemma:4b 運算速度過慢)

5. Code (提取資料)

- 接收 AI 輸出的純文字，並將其解析為 JSON 物件

6. Switch(判斷)

- Status 是否為跌倒，如果是，執行下一個節點。不是，就什麼都不做

7. http request(回傳 line 通知)

- 向 Line 推播服務的 API (/line/notify_line) 發送一個 POST 請求，請求中包含 elder_id 和 status，從而觸發對照護者的即時警報

表 9-2-7 FB 社群貼文抓取與處理貼文

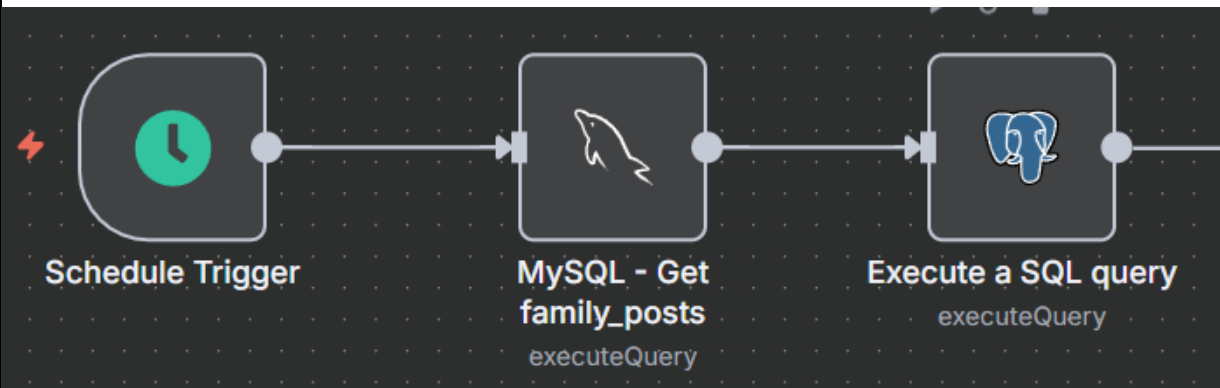
編號	1-1-7	流程檔案	supabaseDB.json
目的	FB 社群貼文抓取與處理貼文		
節點流程說明			
Parent-workflow			
			
Schedule Trigger - 設定固定每一個小時觸發一次 workflow 流程 Execute a SQL query (executeQuery) - 執行一條 SELECT SQL 語句，從 family_posts 表中選取所有欄位的資料 - Execute a SQL query (Postgres) - 將 MySQL 傳入的資料嘉進 postgresDB - 在 INSERT 語句的末尾，使用 ON CONFLICT(platform_post_id)DO NOTHING。如果 platform_post_id 這個欄位的值在目標資料表中已經存在，則什麼都不做 (DO NOTHING)			

表 9-2-8 虛擬碼—db.py

編號	1-2-1	程式名稱	db.py
目的	建立 MySQL 連線池，提供安全借用與健康檢查。		
虛擬碼			
1. 初始化與設定 • 載入模組與庫 dotenv 讀取環境變數；aiomysql 建立連線池；asyncio、uuid、traceback 用於非同步與追蹤。 • 核心參數（來自 .env） ◦DB_POOL_MIN_SIZE / DB_POOL_MAX_SIZE：連線池最小/最大連線數（預設保守，如 2 / 8）。 ◦DB_POOL_RECYCLE：連線最長存活秒數（例：300s，需小於MySQL wait_timeout）。 ◦DB_CONNECT_TIMEOUT：初始連線逾時。			

- USE_POOL_TIMEOUT + POOL_TIMEOUT：借用連線是否啟用逾時與秒數。

- DB_PRE_PING + DB_PING_TIMEOUT：借出前是否 ping() 健檢與逾時秒數。

- DB_HOLD_WARN_MS：連線持有超過此毫秒數即印出警示（例：800ms）。

- **重試錯誤碼**

- _RETRY_ERRCODES = {2006, 2013}（server gone / lost connection during query）。

- **監控狀態**

- **ACTIVE_BORROWERS**：記錄目前被借用的連線／呼叫堆疊，協助排查漏釋放或長時間持有。

2. 類別與資料結構

- **_RetryingCursor（透明重試的 Cursor 包裝器）**

- 支援 async with；在 execute() / executemany() 遇到 2006/2013 時，自動重建連線並重試一次。

- 其餘屬性直接代理到底層 aiomysql.Cursor。

- **_RetryingConnection（透明重試的 Connection 包裝器）**

- 取得新連線時執行 **Pre-Ping**，失敗則換新。

- cursor() 回傳 _RetryingCursor；_reacquire_new() 可在執行中快速換線。

- close()/ensure_closed() 封裝安全關閉。

- **Database（連線池門面）**

- init_pool() 建立池；close_pool() 關閉池；_acquire_from_pool() 含最多 3 次借用重試與（可選）逾時控制。

- get_connection() 取得 _RetryingConnection。

- connection() **非同步情境管理器**：登記→產出連線→紀錄持有時間→超標告警→釋放歸還。

- debug_status()/borrowers_snapshot()：除錯用狀態檢視。

3. 函數與功能（流程化描述）

- **初始化連線池 init_pool()**

1. 讀取 .env 與池參數 → 2) 呼叫 create_pool(...)
（autocommit=True、charset=utf8mb4、pool_recycle=DB_POOL_RECYCLE）

2. 成功後印出「Connection pool initialized (min=?, max=?, recycle=?s)」。

- **借用原始連線 _acquire_from_pool()**

1. 最多重試 3 次；若 `USE_POOL_TIMEOUT`，使用 `asyncio.wait_for(pool.acquire(), POOL_TIMEOUT)`
2. 逾時或失敗則短暫 `sleep` 退避後再試 → 3) 仍失敗則丟出錯誤。

- **取得可重試連線 `get_connection()`**

1. 建立 `_RetryingConnection` → 2) 內部 `_acquire_initial()` 先從池借到一條 → 3) **Pre-Ping** 驗證，失敗即換線。

- **釋放連線 `release_connection()`**

1. 偵測包裝連線的底層 `_raw` → 2) 呼叫 `pool.release(raw)` 歸還 → 3) 若釋放失敗則強制關閉。

- **情境管理器 `connection()` (建議 DAO 用法)**

1. 借用前：在 `ACTIVE_BORROWERS` 登記 (含呼叫堆疊)
2. `yield conn` 供 DAO 執行 SQL
3. 結束時：計算持有毫秒；若 `> DB_HOLD_WARN_MS` 印出告警行
4. 一定釋放連線回池。

- **除錯介面**

1. `debug_status()`：列出池大小、已用、可用、min/max；如 `freysize==0` 顯示警告。
2. `borrowers_snapshot()`：輸出目前借用清單 (持有最久優先)。

4. 錯誤處理與重試策略

- **游標層重試**：`execute()` / `executemany()` 捕捉 `OperationalError`，若錯誤碼在 `{2006,2013}` → **立即換線並重試一次**。

- **Pre-Ping 保護**：借用與新建游標前都會在 `PING_TIMEOUT` 內 `ping()`，失敗則快速換線。

- **借用逾時**：若啟用 `USE_POOL_TIMEOUT`，超過 `POOL_TIMEOUT` 秒會重試並最終回報逾時。

- **持有過久告警**：情境管理器自動印出 `[DB] held xxx.ms by -` 以利找出慢查詢／漏釋放。

5. 與其他模組關聯

- **DAO／Service 用法 (示意)：**

- `async with Database.connection() as conn:`
`async with conn.cursor(DictCursor) as cur: await cur.execute(SQL, params)`

- **時機**：所有路由 (Routes) 進入 Service／DAO 前透過本模組提供的安全借用；WS 事件回呼寫 DB 時亦一致。

表 9-2-9 虛擬碼—ws_connection_manager.py

編號	1-2-2	程式名稱	ws_connection_manager.py
目的	管理 WebSocket 連線（依 user_id），提供傳送與清理。		
虛擬碼			
1.初始化與設定 • 載入模組： typing.Dict, List：型別標註。 fastapi.WebSocket、starlette.websockets.WebSocketDisconnect：WS 物件與斷線例外。 datetime：時間戳；traceback：錯誤堆疊印出。 • 初始化屬性： active_connections: Dict[str, List[WebSocket]] = {}：以 使用者 ID 為 key 的連線清單。 • 工具方法： ts()：回傳 YYYY-MM-DD HH:MM:SS 時間字串。 _ws_id(ws)：組合 id(host:port) 供日誌辨識。 _count(user_id)：取得該使用者目前連線數。 2.連線生命週期 • connect(user_id, websocket)： 印出「嘗試連線」日誌。 await websocket.accept() 接受握手。 將 websocket 附加到 active_connections[user_id]。 印出「連線成功、目前連線數」日誌。 • disconnect(user_id, websocket)： 印出「準備斷線」日誌。 從 active_connections[user_id] 移除該連線；若清單為空，刪除該 key。 印出「斷線完成、剩餘連線數」日誌。 3.傳送訊息（單一使用者） • send_personal_message(message, user_id)（純文字）： 複製連線清單（避免迭代時被修改）。 逐一 await ws.send_text(message) 傳送。 若捕捉到 WebSocketDisconnect：印出錯誤碼、呼叫 disconnect() 清理。 其他例外：印出 [SEND_TEXT][ERROR] 與堆疊，並清理該連線。 • send_json(user_id, data)（JSON）： 先印出「Sending data to user ...」日誌（含 payload 摘要）。 逐一 await ws.send_json(data) 傳送。 斷線/例外處理邏輯同上，並附加「cleaned connection」日誌。 4.廣播訊息（所有使用者）			

- broadcast(message) (純文字) :
走訪 active_connections.items()，逐一對各連線 send_text()。
捕捉 WebSocketDisconnect 或其他例外並清理。
- broadcast_json(data) (JSON) :
走訪所有使用者的連線清單，逐一 send_json()。
斷線/例外處理與清理流程一致。

5. 除錯輔助

- get_user_connections(user_id)：回傳指定使用者的連線清單。
- get_all_user_ids()：回傳目前有連線的所有使用者 ID。
- debug_dump()：印出 使用中使用者數、各使用者連線數與 host:port 明細。

6. 錯誤處理與穩定性設計

- 對 斷線 使用 WebSocketDisconnect 分流處理，擷取 code 便於追蹤。
- 其餘 非預期例外 會印出 [...] [ERROR] 與 traceback，隨即清理連線，避免殭屍連線。
- 以 list(...) 先複製連線清單，避免在迭代同時被 disconnect() 修改導致錯誤。
- _ws_id() 允許 client.host/port 不存在時回退為？，提升日誌健壯性。

表 9-2-10 虛擬碼—ws_message_dispatcher.py

編號	1-2-3	程式名稱	ws_message_dispatcher.py
目的	分流 WebSocket 訊息至推論協調器；斷線時釋放使用者狀態。		
虛擬碼			
1.初始化與設定			
•載入模組： ◦json：解析文字訊息。 ◦typing.Optional：型別註記（程式中可選用）。 ◦stream_infer_manager：推論協調器（同目錄匯入）。 •全域狀態：無；以函式形式提供服務。			
2.函數與功能			
•async def handle_ws_text(user_id: str, text: str) ◦輸入：user_id（使用者識別）、text（WS 傳入的文字 JSON）。 ◦步驟： 1. 嘗試 json.loads(text) 解析為 data。 2. 解析成功 → await stream_infer_manager.ingest(user_id, data) 交由推論協調器處理。 3. 解析失敗 → 直接返回（不回應、不拋錯）。 ◦輸出/副作用：無回傳值；可能觸發協調器進行緩衝、成片段與推論。 •def notify_user_disconnected(user_id: str)			

- 目的：使用者 WS 斷線時釋放其在協調器中的暫存狀態。
- 動作：stream_infer_manager.drop_user(user_id)。

3.主要流程（事件順序）

- WS 收到 文字訊息 → handle_ws_text()
- 成功解析 JSON → 交由 **StreamInferManager** ingest()
- 使用者 斷線 → 路由呼叫 notify_user_disconnected() → 協調器清除該 user 緩衝。

4.錯誤處理

- **JSON 解析失敗**：以 try/except 捕捉，靜默忽略並返回（不回覆、不記 log）。
- **協調器執行例外**：本檔未攔截；由上層 WS 路由或協調器內部處理。

表 9-2-11 虛擬碼—kalman_filter.py

編號	1-2-4	程式名稱	kalman_filter.py
目的	平滑 17 個關鍵點 (x,y) 座標，降低抖動並穩定序列。		
虛擬碼			
1.初始化與設定 • 載入：numpy as np • 狀態維度：34（17 關鍵點 × 2） • 內建矩陣與雜訊： ◦ transition_matrix = I(34) ◦ measurement_matrix = I(34) ◦ process_noise = I(34) * 0.01 ◦ measurement_noise = I(34) * 0.1 • 初始狀態：state = None, covariance = None 2.類別與資料結構 • class KalmanFilter: ◦ 欄位：state: np.ndarray None、covariance: np.ndarray None ◦ 參數矩陣：transition_matrix, measurement_matrix, process_noise, measurement_noise 3.函數與功能（流程化描述） • predict() 1. 若 state is None → 返回 None（尚未初始化） 2. 狀態預測：state = F @ state 3. 協方差預測：cov = F @ cov @ F.T + Q 4. 回傳 state • update(measurement) 1. 轉為一維量測向量：z = np.array(measurement).flatten()			

2. 若首次更新 (state is None) → 直接初始化: state = z, cov = I(34)
3. 計算殘差協方差: $S = H @ cov @ H.T + R$
4. 卡爾曼增益: $K = cov @ H.T @ inv(S)$
5. 狀態更新: $state = state + K @ (z - H @ state)$
6. 協方差更新: $cov = (I - K @ H) @ cov$
7. 回傳 state

• predict_and_update(measurement)

1. 若為首次 → 初始化同 update(), 並輸出提示字串
2. 否則: 先 predict() → 再 update(measurement)
3. 例外時捕捉錯誤並回傳當前 state

4. 錯誤處理

- 首次收到量測時自動初始化, 避免未定義狀態造成運算錯誤
- predict_and_update() 以 try/except 包覆全流程, 錯誤時印出訊息並回傳舊狀態
- 量測長度不符 (非 34 維) 可能在 np.linalg.inv(S) 或矩陣乘法時觸發例外

表 9-2-12 虛擬碼—cnn_lstm_utils.py

編號	1-2-5	程式名稱	cnn_lstm_utils.py
目的	定義 CNN+LSTM 與時序匯聚頭（支援 mask 與 motion 特徵）。		
虛擬碼			
1.初始化與設定 • 載入：torch, torch.nn as nn, typing.Optional • 主要張量介面： ◦ 影像序列 x: (B, T, C, H, W) ◦ 動作特徵 motion: (B, T, D)（可選） ◦ 有效幀遮罩 mask: (B, T)，元素$\in\{0,1\}$ 2.類別與資料結構 • class SpaceCNN(nn.Module) ◦ 卷積骨架：Conv2d $\rightarrow$ ReLU $\rightarrow$ Conv2d $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv2d $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv2d $\rightarrow$ ReLU $\rightarrow$ AdaptiveAvgPool(1) ◦ 全連接壓縮：fc: Linear(256, out_dim) ◦ forward(x)：輸入 (B*T, C, H, W) $\rightarrow$ 吐出 (B*T, out_dim) • class TemporalHead(nn.Module)（時序匯聚） ◦ 模式："last" "mean" "attn"（預設 "attn"；可搭配 dropout） ◦ "attn"：attn: Linear(in_dim, 1) 產生每幀權重；可與 mask 結合 ◦ forward(seq_feats, mask=None)：			

- seq_feats: (B, T, D) → 依模式匯聚成 (B, D) → fc: Linear(D, num_classes)
- 有 mask 時：mean/attn 會忽略無效幀
- class CNNLSTM(nn.Module)
 - 結構：SpaceCNN 萃取空間特徵 → (可選) 串接 motion → LSTM → TemporalHead
 - 參數重點：in_ch, num_classes, cnn_out, lstm_h, lstm_layers, bidirectional, temporal_pool, dropout, motion_dim
 - forward(x, motion=None, mask=None)：
 1. x reshape 成 (B*T, C, H, W) → SpaceCNN → 還原 (B, T, cnn_out)
 2. 若 motion_dim>0 且 motion≠None：cat([cnn_feat, motion], dim=-1)
 3. LSTM 得 (B, T, H)
 4. TemporalHead(out, mask) 匯聚 → 類別分數 (B, num_classes)
- 3. 函數與功能（流程化描述）
 - 空間特徵抽取：對每幀影像做卷積與自適應平均池化，獲得定長特徵。
 - 時序建模：將每幀特徵（可含 motion）送入 LSTM，保留時序依賴。
 - 時序匯聚：
 - last：取最後一幀輸出。
 - mean：平均（可用 mask 排除無效幀）。
 - attn：以學習到的注意力權重對幀做加權（mask 將無效幀權重置為 -inf）。
 - 分類頭：Linear 輸出各類別 logits。
- 4. 錯誤處理
 - 維度不符：x 需為 5 維；motion.shape[:2] 必須與 x[:2] 對齊；mask.shape 需為 (B, T)。
 - motion_dim 與實際 motion 最後一維不一致時，會在 cat 觸發維度錯誤。
 - mask 值域：非 0/1 會導致注意力 masked_fill 行為異常（應先二值化/裁切）。

表 9-2-13 虛擬碼－response_util.py

編號	1-2-6	程式名稱	response_util.py
目的	統一 API 回傳格式。		
虛擬碼			
1.初始化與設定 • 載入：fastapi.responses.JSONResponse、fastapi.encoders.jsonable_encoder • 基本結構：{"success", "code", "message", "data"} 2.類別與資料結構 • 無（導出單一非同步函式） 3.函數與功能（流程化描述） • async def make_json_response(data=None, code=200, message="", success=None) 1. 若 success is None → 依 code 自動判斷 $success = 200 \leq code < 300$ 2. 組合 payload = {"success": success, "code": code, "message": message, "data": data} 3. 以 jsonable_encoder(payload) 確保可序列化 4. 回傳 JSONResponse(content=..., status_code=int(code)) 4.主要流程（呼叫端視角） • 成功案例：await make_json_response(data=物件, code=200, message="OK") • 失敗案例：await make_json_response(data=None, code=400, message="Bad Request", success=False) 5.錯誤處理 • success 省略時自動推斷，避免呼叫端遺漏一致性 • code 以 int(code) 保險轉型，避免非整數型引發框架錯誤 • jsonable_encoder 對 datetime/自定物件做安全轉換，避免序列化例外			

表 9-2-14 虛擬碼—notify_prefs_service.py

編號	1-2-7	程式名稱	notify_prefs_service.py
目的	維護使用者推播偏好（weekday/hour/minute），含 LINE 綁定查找與人性化提示。		
虛擬碼			
1.初始化與設定 - 載入：typing、Database、upsert_prefs、get_prefs_by_user_id、get_user_by_line_user_id - 週名常數：WEEK_NAMES = ["日","一","二","三","四","五","六"]（0=週日...6=週六） 2.類別與資料結構 - 無（以函式為主） 3.函數與功能（流程化描述） - parse_sel(sel: str) -> List[int] 1. 若空字串 → 回傳 [] 2. 以,分割後過濾空白 → 轉 int 清單 - toggle_day(sel: List[int], d: int) -> List[int] 1. 將 sel 轉集合 s 2. 若 d 已存在 → 移除；否則加入 3. 回傳排序後清單 - human_days(days: List[int]) -> str 1. 若為空 → 回傳（未選） 2. 否則以 週{WEEK_NAMES[d]} 串接（、 分隔） - human_time(h: int, m: int) -> str 1. 以零補齊格式化 HH:MM - async def save_prefs_by_line_uid(line_user_id, days, hour, minute) -> Tuple[bool, str] 1. 以 line_user_id 查找對應使用者（未綁定則回 False, "尚未綁定..."） 2. 將 days 以逗號串成 weekdays 3. 呼叫 upsert_prefs(conn, user_id, weekdays, hour, minute) 4. 成功訊息：設定完成 ☒ + human_days/human_time 組字 5. 例外：印錯並回 False, "設定失敗：..." - async def get_prefs_for_line_uid(line_user_id) -> Optional[Dict] 1. 以 line_user_id 查找使用者（無則 None） 2. 讀取 get_prefs_by_user_id(conn, user_id)（無則 None） 3. 回傳 {user_id, weekdays:[int], hour, minute}（weekdays 由逗號字串轉整數陣列） 4.主要流程（呼叫端視角）			

- 寫入：LINE 使用者回報偏好 → save_prefs_by_line_uid() → upsert 成功 → 回覆人性化文案
- 讀取：LINE 使用者查詢偏好 → get_prefs_for_line_uid() → 回傳結構化設定值

5. 錯誤處理

- 未綁定 LINE 帳號：回 False 與友善提示
- 任何例外：列印錯誤訊息並回傳 False/None，避免泡沫化例外中斷流程

表 9-2-15 虛擬碼—notify_line_service.py

	1-2-8	程式名稱	notify_line_service.py
目的	發送 LINE 訊息並記錄收發結果。		
虛擬碼			
1.初始化與設定 • 載 入 ： os, asyncio, httpx, datetime, typing , Database , list_caregiver_line_uids_by_elder , list_line_uids_by_role • 變 數 ： LINE_CHANNEL_ACCESS_TOKEN （ 環 境 變 數 ） 、 LINE_PUSH_URL="https://api.line.me/v2/bot/message/push" 2.類別與資料結構 • 無（以函式為主） 3.函數與功能（流程化描述） • async def push_line_message(to_uid: str, message: str) -> int 1. 組 headers（含 Bearer token）與 payload（文字訊息） 2. httpx.AsyncClient POST 至 LINE API 3. 非 200 印出錯誤摘要，回傳 status_code • async def push_text_bulk(recipients: Iterable[str], message: str) -> Dict[str, List[str]] 1. 將收件者轉清單 2. asyncio.gather 併發呼叫 push_line_message 3. 依 status_code 分組：sent（200）／failed（非 200） • def build_message(*, status, message, detected_at) -> str 1. 若 message 已給 → 直接回傳 2. 正規化 status，預設 detected_at=now() 3. 依狀態模板回傳字串（跌倒／離床／返回／一般事件） • async def notify_from_payload(*, elder_id, status, message, detected_at) -> Dict[str, object] 1. 以 build_message() 產生文案 2. Database.connection() 取得收件者： ▪ 有 elder_id → list_caregiver_line_uids_by_elder(conn, elder_id) ▪ 無 elder id → list line uids by role(conn, 2)（廣播給照護者）			

3. 若無收件者 → 回 {status:"ok", message, source, sent:[], failed:[], note:"無收件者"}

4. 否則 push_text_bulk(uids, msg) , 回 {status:"ok", message, source, sent, failed}

4.主要流程（呼叫端視角）

- 組事件 payload（可含 elder_id | status | message | detected_at）
- 呼叫 notify_from_payload() → 自動尋找 LINE 收件者 → 產生文案 → 併發推送 → 回傳 sent/failed 清單

5.錯誤處理

- 單筆推送非 200：列印 [LINE PUSH] to=... code=... body=...，不拋出、由批次彙整
- 併發批次中個別失敗不影響其他收件者；結果以 sent/failed 匯總
- DB 查詢若未得任何收件者，回傳 note:"無收件者" 以利路由端處理

表 9-2-16 虛擬碼—weekly_report_push_service.py

編號	1-2-9	程式名稱	weekly_report_push_service.py
目的	定時或批次推播互動週報給照護者。		
虛擬碼			
1. 初始化與設定 - 載入：typing, datetime, asyncio - 資料庫：Database - DAO：select_latest_report_by_elder_id、list_caregiver_line_uids_by_elder、get_users_for_weekly_push_by_time、get_all_elders_with_recent_reports - 通知：push_text_bulk（LINE 批次推送） - 例外：NotFoundError, DatabaseError 2. 類別與資料結構 - 無（以函式為主） 3. 函數與功能（流程化描述） - async def push_weekly_reports_by_time(hour=None, minute=None, weekday=None) -> Dict 1. 取現在時間，缺省參數以當下補齊；將週日（7）轉為 0（配合 DB）。 2. get_users_for_weekly_push_by_time(conn, h, m, d) 取在該時間需推播的使用者。 3. 依 elder_id 分組，避免同一長者多次推播。 4. 對每組：			

- 取最新週報 `select_latest_report_by_elder_id(conn, elder_id)`。
- 組文案 `format_weekly_report_message(report)`。
- 收件人為該組 `line_user_id` 清單；呼叫 `push_text_bulk(uids, msg)`。
- 彙整結果：`status/sent/failed/report_period/target_users`。
- 5. 計算總結（成功長者數、sent/failed 總數）並回傳。
- 6. 例外時回傳 `{"status": "error", ...}` 與目標時間。
- `async def push_weekly_reports_batch() -> Dict`
 1. 以 `get_all_elders_with_recent_reports(conn)` 取近一週有週報的長者清單。
 2. 為每位長者建立 `push_weekly_report_to_elder(elder_id)` 任務並 `asyncio.gather()` 併發執行。
 3. 彙整各任務回傳，計算成功/失敗總數並回傳總結。
 4. 例外時回傳 `{"status": "error", ...}`。

備註：檔案內存在重複定義 `push_weekly_reports_batch` 的區塊，實際以後者覆蓋。
- `async def push_weekly_report_to_elder(elder_id: int) -> Dict`
 1. 取該長者最新週報 `select_latest_report_by_elder_id(conn, elder_id)`。
 2. 取照護者 LINE UUIDs：`list_caregiver_line_uids_by_elder(conn, elder_id)`。
 3. 若無收件者 → 回 `no_recipients`。
 4. `format_weekly_report_message(report)` → `push_text_bulk(uids, msg)` → 回傳包含 sent/failed 的結果。
 5. `NotFoundError` → 回 `no_report`；其他例外 → 回 `error`。
- `async def format_weekly_report_message(report: Dict, elder_name: str=None) -> str`
 1. 讀取 `start_date/end_date/analysis_result`，`analysis_result` 長度 > 1300 會截斷並加「(完整報告請至 App 查看)」。
 2. 長者顯示名稱：優先 `elder_name`，否則 長者 (ID: ...)。
 3. 回傳多行文字（含期間與分析摘要）。
- 4. 主要流程（呼叫端視角）
 - 定時推播：CRON → `push_weekly_reports_by_time(h, m, d)` → 依長者彙總推送。
 - 一次全量：管理端 → `push_weekly_reports_batch()` → 併發對所有長者推送。
 - 個別補送：事件或人工 → `push_weekly_report_to_elder(elder_id)`。
- 5. 錯誤處理

- 找不到週報：回 {"status":"no_report"} 並將該組收件者列入 failed。
- 無收件者：回 {"status":"no_recipients"}。
- DB/網路等其他例外：回 {"status":"error"}；批次推送不影響其他任務。
- 檔案中多餘 SQL 區塊（獲取長者列表）與重複定義函式，建議整併以免混淆執行路徑。

表 9-2-17 虛擬碼—line_binding_service.py

編號	1-2-10	程式名稱	line_binding_service.py
目的	管理 LINE 綁定查詢與解綁。		
虛擬碼	- 初始化與設定 - 載入：typing (Optional, Dict, Any)、Database - DAO：unbind_by_line_user_id、get_line_user_id_by_user、get_user_by_line_user_id - 所有操作皆以 async with Database.connection() 取得連線 - 類別與資料結構 - 無（以函式為主） - 函數與功能（流程化描述） - async def unbind_line_user(line_user_id: str) -> int - 開啟連線 - 呼叫 unbind_by_line_user_id(conn, line_user_id) - 回傳受影響列數（0/1） - async def get_line_uid_by_user(user_id: int) - 開啟連線 - 呼叫 get_line_user_id_by_user(conn, user_id) - 回傳對應 Uxxxx（若無則 None） - async def get_user_by_line_uid(line_user_id: str) -> Optional[Dict[str, Any]] - 開啟連線 - 呼叫 get_user_by_line_user_id(conn, line_user_id) - 回傳使用者資料（若無則 None） - 主要流程（呼叫端視角） - 解綁：收到管理或取消綁定事件 → unbind_line_user(Uxxxx) → 回 0/1 - 查 LINE UID：已知 user_id → get_line_uid_by_user(user_id) - 查使用者：已知 Uxxxx → get_user_by_line_uid(Uxxxx) - 錯誤處理		

- DB 例外交由呼叫端統一處理（本層僅確保連線取得/釋放）
- 查無資料時回傳 None（呼叫端需判斷未綁定情況）

表 9-2-18 虛擬碼—binary_cnn_lstm_trainer.py

編號	1-2-11	程式名稱	binary_cnn_lstm_trainer.py
目的	訓練二元跌倒偵測模型並產出 best.pt / last.pt / topk.json 與 classes.json、motion_norm.json		
虛擬碼			
1. 初始化與設定 - 載入：torch/nn/DataLoader, numpy, cv2, tqdm, sklearn.metrics。 - 基本常數：_KP_CONF_TH, _SIGMA_KP, _CNN_OUT, _FOCAL_GAMMA, _EARLY_STOP_PATIENCE 等。 - Config：資料路徑（pose_root/obj_root）、視窗（window/stride）、RelationMap 尺寸（H,W）、是否含骨線/物件通道、LSTM 與 TemporalHead 參數、訓練超參、二元模式（binary_mode=True、rare_class_name="fall"、binary_class_names=("non_fall","fall"））。 - 輸出：out_dir（會建立 best.pt / last.pt / ep{n}_score{:.4f}.pt / topk.json / classes.json / motion_norm.json）。 2. 類別與資料結構 RelationMapConfig：控制通道組成（bbox(1)+dist(1)+kp(17)+bone(12 可選)+object(N)+coord(2)）。 資料集 PoseObjectDataset - 掃描類別資料夾；若 binary_mode=True，將 rare_class_name 標為 1，其餘類別併成 0。 - 以 window/stride 產生序列切片；可要求首幀/全幀骨架完整。 - 讀取每幀 pose/objects → 產生 RelationMap；計算 motion(9) 與有效幀 mask；（可選）KF 半視窗平滑關節點。 - 估計 motion 的 mean/std 供 z-score 正規化並保存。 模型 - SpaceCNN：卷積抽空間特徵到固定維度。 - CNNLSTM：CNN 特徵（可拼接 motion(9)）→ LSTM → TemporalHead("attn" "mean" "last"）（支援 mask）。 Loss - FocalLoss(alpha, gamma) 或 CrossEntropyLoss，不平衡時使用動態 alpha。 3. 函數與功能（流程化描述）			

- rasterize_frame(...)：依 bbox/kp/bone/objects/coord 生成多通道 RelationMap。
- compute_motion_feats_with_mask_from_parsed(parsed)：輸出 (T,9) 與 (T,) mask。
- kalman_smooth_kps(window_kps, half_slide, ...)：半視窗 KF 平滑，不改動原始輸入檔。
- build_loaders(cfg, ds, idx_tr, idx_va)：建立 train/val DataLoader；可選 **WeightedRandomSampler** 平衡類別。
- evaluate(model, loader, device, use_motion=True)：回傳 acc, macro_f1。
- train(cfg)：
 1. 固定亂數；建立 PoseObjectDataset → 切分 8:2；建立 DataLoader。
 2. 建模 (in_ch = 2+17+bone+objects+2；motion_dim=9) 與優化器/AMP/損失。
 3. 每 Epoch：model.train() 前傳 → 回傳 → GradScaler → (可 clip_grad_norm_) → 累積 train loss/acc。
 4. evaluate() 取 va_acc/macro_f1 作 **排序分數**。
 5. 保存檔案：
 - last.pt (含 model/optimizer、class_names、motion_norm、config)。
 - 若分數最佳 → 覆蓋 **best.pt**，並同步寫入 **classes.json / motion_norm.json**。
 - 永久保存 ep{epoch}_score{:.4f}.pt，並維護 **Top-3** (多餘自動清理，索引寫入 topk.json)。
 6. **Early Stopping**：macro_f1 連續未提升達門檻即停止。
 7. 結尾載入 best.pt 產出最終 acc/macro_f1、**混淆矩陣**與分類報告。
- 4. 主要流程 (呼叫端視角)
 - CLI (範例)：python binary_cnn_lstm_trainer.py (直接使用 Config 內設定)。
 - 流程：資料建置 → 訓練/驗證 → 儲存 best.pt/last.pt/topk.json → 同步 classes.json/motion_norm.json → 報告。
- 5. 錯誤處理
 - 缺資料夾/無可用視窗：丟出 FileNotFoundError/RuntimeError。
 - 非法 JSON/欄位缺失：讀取時跳過；RelationMap/骨線繪製例外以 try/except 忽略當前點位。

- 計算中 NaN/Inf：由損失計算與 AMP 條件式處理；早停避免長時間無效訓練。

第10章 測試模型

10-1 測試計畫

本系統執行所需的硬體為作業系統 Android 8.0 以上的智慧型手機。在系統程式開發設計完成後，我們將程式存為 APK 檔後於手機上進行測試，確認每項功能是否能夠執行成功。

- (1) 會員註冊：確認使用者能夠註冊。
- (2) 會員登入：確認使用者能夠登入到首頁。
- (3) 更改密碼：修改使用者密碼。
- (4) 刪除帳號：確認使用者帳號可被刪除並登出。
- (5) 編輯個人資料（含長輩資料）：
確認使用者可修改並保存使用者/長輩基本資料。
- (6) 查看跌倒列表：確認使用者可查詢「跌倒／疑似跌倒」事件。
- (7) 查看長者與系統互動對話紀錄：確認使用者可瀏覽當日對話紀錄。
- (8) 查看緊急聯絡人：清單正確顯示、可觸發撥號。
- (9) 新增緊急聯絡人：確認使用者能成功新增。
- (10) 刪除緊急聯絡人：確認使用者能成功刪除。
- (11) 查看活動量：確認使用者能成功查看活動量。
- (12) 查看睡眠列表：確認使用者能成功查看睡眠列表。
- (13) 查看語音互動報告：確認使用者能成功查看語音互動報告。
- (14) 查看久坐資訊：確認使用者能成功查看久坐資訊。

10-2 測試個案與測試結果資料

表 10-2-1 功能測試-會員註冊

功能名稱	會員註冊
測試目標	確認使用者能夠註冊
測試作業	(1)輸入電話號碼與密碼 (2)驗證資料庫是否有符合的會員資料 (3)若無重複的會員資料則直接註冊該筆會員資料
測試結果	成功

表 10-2-2 功能測試-會員登入

功能名稱	會員登入
測試目標	確認使用者能夠登入到首頁
測試作業	(1)輸入電話號碼與密碼 (2)驗證資料庫是否有符合的會員資料 (3)若查無符合的會員資料無法登入，需先註冊帳號
測試結果	成功

表 10-2-3 功能測試-更改密碼

功能名稱	更改密碼
測試目標	修改使用者密碼
測試作業	(1)點選【個人】頁面 (2)驗證資料庫是否有符合的會員資料 (3)測試是否能用新密碼登入
測試結果	成功

表 10-2-4 功能測試-刪除帳號

功能名稱	刪除帳號
測試目標	確認使用者能夠註冊
測試作業	(1)點選【個人】頁面 (2)驗證資料庫是否有符合的會員資料 (3)測試是否能用新密碼登入
測試結果	成功

表 10-2-5 功能測試-編輯個人資料（含長輩資料）

功能名稱	編輯個人資料（含長輩資料）
測試目標	確認使用者能修改並保存使用者/長輩基本資料
測試作業	(1)點選【個人檔案】頁面 (2)修改姓名、角色、手機號碼、地址並儲存 (3)顯示最新資料，欄位驗證正常
測試結果	成功

表 10-2-6 功能測試-查看跌倒列表

功能名稱	查看跌倒列表
測試目標	確認使用者能查詢「跌倒／疑似跌倒」事件
測試作業	(1)點選【查詢】頁面 (2)類別選取狀態清單 (3)顯示「跌倒／疑似跌倒」事件
測試結果	成功

表 10-2-7 功能測試-查看長者與系統互動對話紀錄

功能名稱	查看長者與系統互動對話紀錄
測試目標	瀏覽當日對話紀錄
測試作業	(1)點選【對話】頁面 (2)使用者可察看當日系統與被照護者語音對話紀錄
測試結果	成功

表 10-2-8 功能測試-查看緊急聯絡人

功能名稱	查看緊急聯絡人
測試目標	瀏覽目前已設定之緊急連絡人
測試作業	(1)點選【聯絡人】頁面 (2)使用者可察看目前已設定之緊急連絡人
測試結果	成功

表 10-2-9 功能測試-新增緊急聯絡人

功能名稱	新增緊急聯絡人
測試目標	確認使用者可新增緊急聯絡人
測試作業	(1)點選【聯絡人】頁面 (2)點選+號 (3)輸入電話號碼及關係 (4)點選新增
測試結果	成功

表 10-2-10 功能測試-刪除緊急聯絡人

功能名稱	刪除緊急聯絡人
測試目標	確認使用者可刪除緊急聯絡人
測試作業	(1)點選【聯絡人】頁面 (2)在該刪除了聯絡人地方向左滑動 (3)點選刪除
測試結果	成功

表 10-2-11 功能測試-查看活動量

功能名稱	查看活動量
測試目標	確認使用者可查看活動量
測試作業	(1)點選【查詢】頁面 (2)類別選取活動量 (3)顯示活動量
測試結果	成功

表 10-2-12 功能測試-查看睡眠列表

功能名稱	查看睡眠列表
測試目標	確認使用者可查看睡眠列表
測試作業	(1)點選【查詢】頁面 (2)類別選取睡眠列表 (3)顯示睡眠列表
測試結果	成功

表 10-2-13 功能測試-查看語音互動報告

功能名稱	查看語音互動報告
測試目標	確認使用者可查看語音互動報告
測試作業	(1)點選【查詢】頁面 (2)類別選取互動報告 (3)顯示語音互動報告
測試結果	成功

表 10-2-14 功能測試-查看久坐資訊

功能名稱	查看久坐資訊
測試目標	確認使用者可查看久坐資訊
測試作業	(1) 點選【查詢】頁面 (2) 類別選取久坐資訊 (3) 顯示久坐資訊
測試結果	成功

第11章 操作手冊

11-1 系統元件

表 11-1-1 系統安裝元件資訊

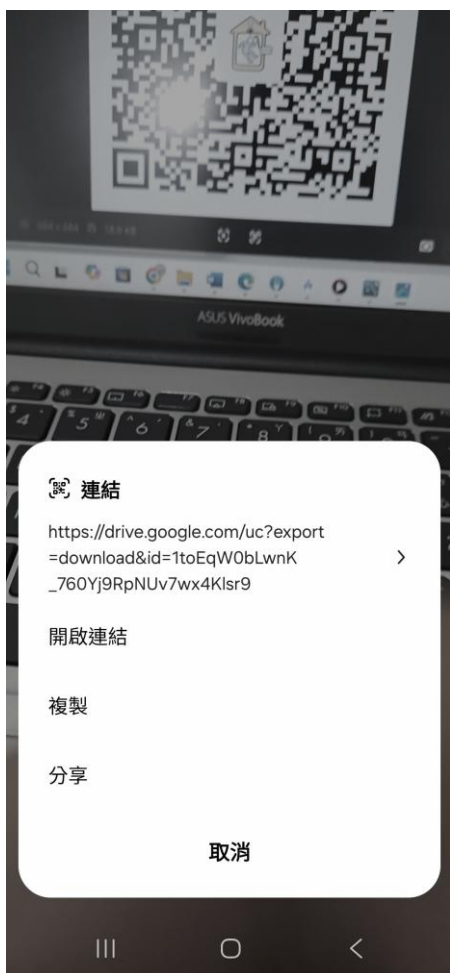
元件名稱	AI 底家
檔案大小	203MB
費用	免費
版本需求	Android 7.0
網路需求	Wifi / 4G / 5G 網路

11-2 系統下載及安裝

- 掃描 QR code 即可下載【AI 底家】App



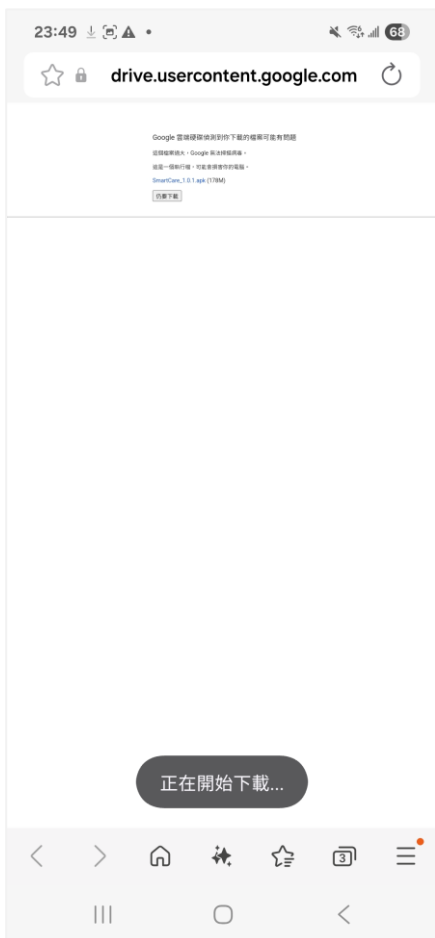
圖 11-1 【AI 底家】QR code



▲圖 11-2 步驟一

掃描【AI 底家】QR code 會跳出前往頁面，點擊開啟連結進入下載頁面。

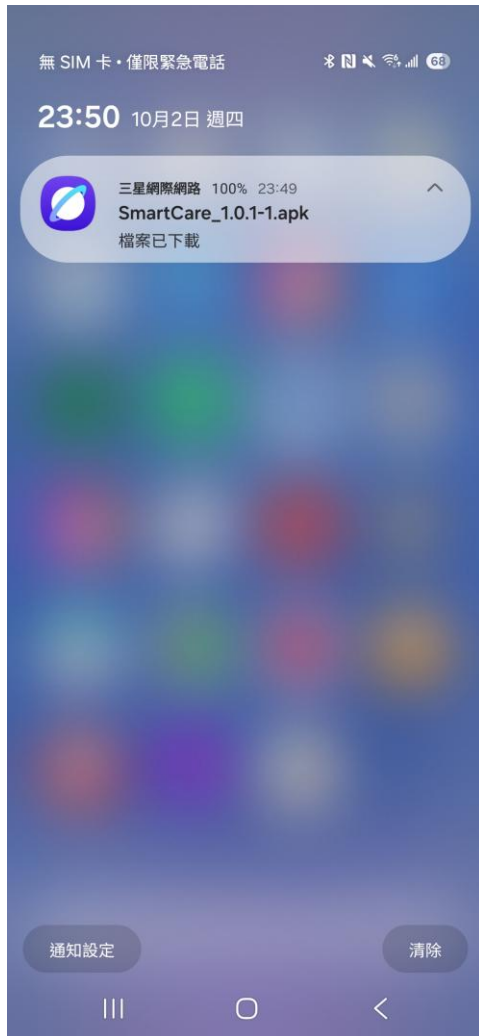
步驟二



點擊下載後等待 apk 檔下載完成。

▲圖 11-3 步驟二

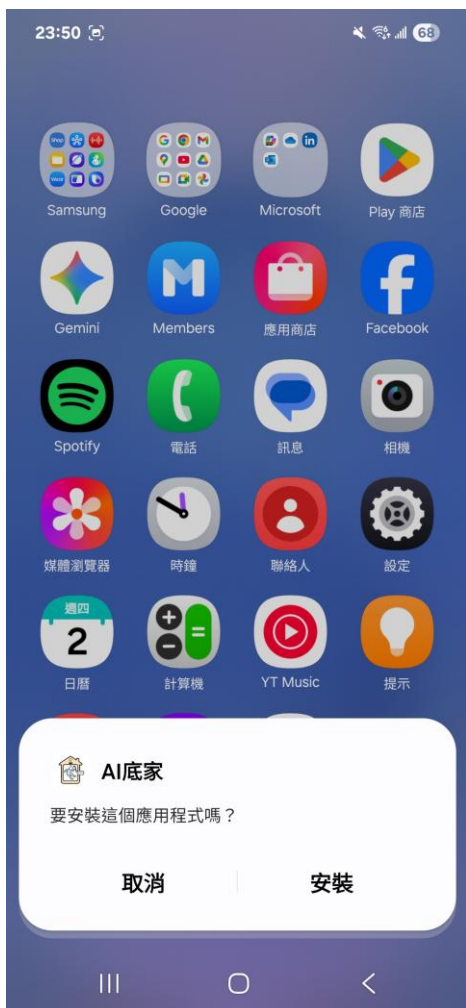
步驟三



apk 檔案下載後，點擊即可跳轉至安裝 App 畫面。

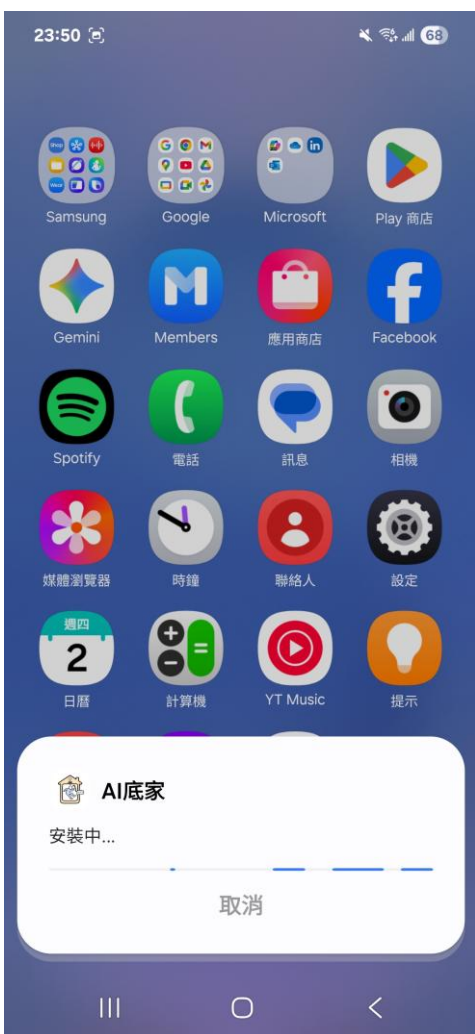
▲圖 11-4 步驟三

步驟四



點擊安裝後，即可將 App 下載至手機。

▲圖 11-5 步驟四



▲圖 11-6 步驟五

等待安裝完成後即可開始使用 App。

第12章 使用畫面

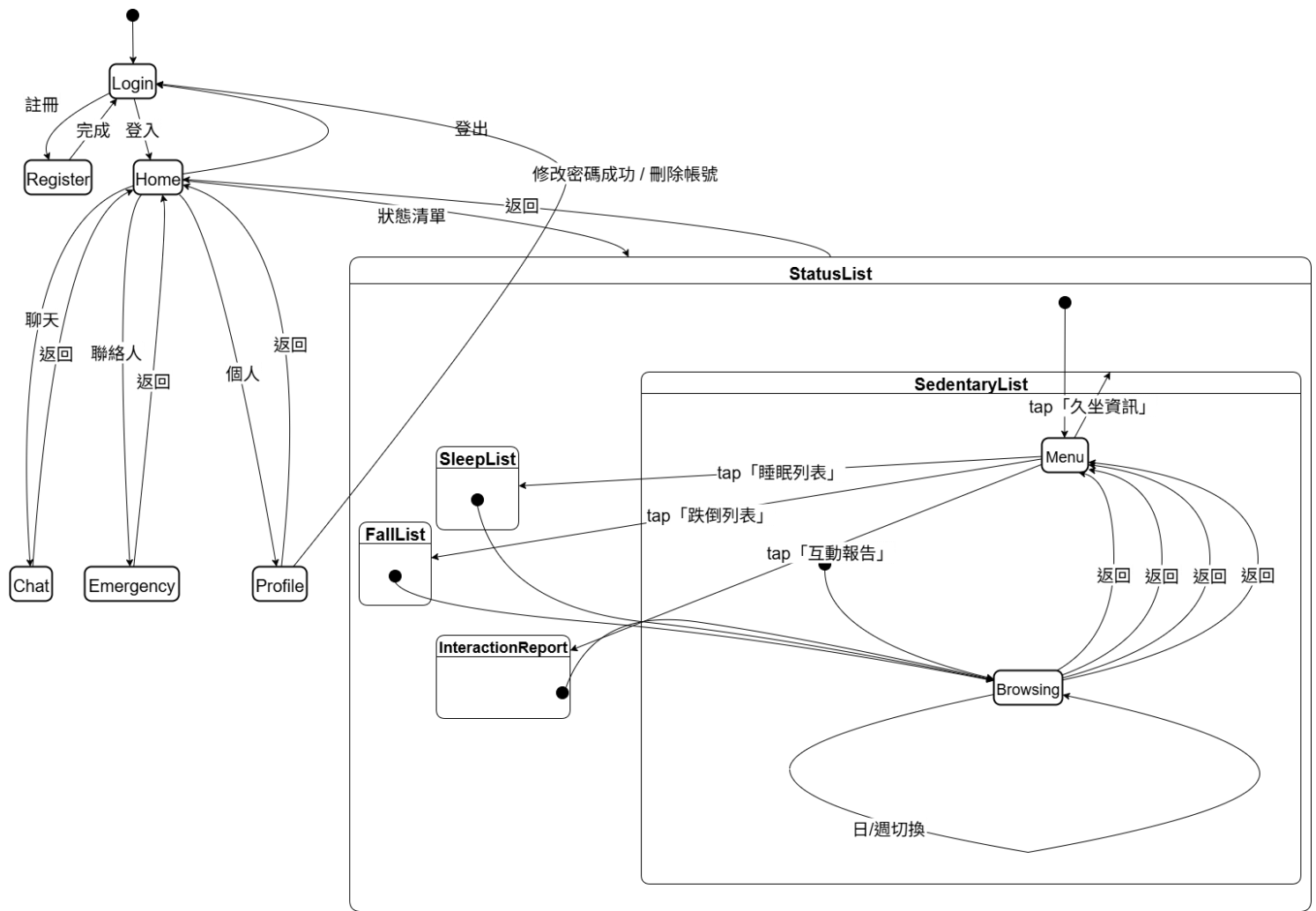


圖 12-1 系統主要操作流程狀態轉移圖-1

登入頁面



The image shows a mobile app login screen for 'AI 底家'. At the top, there is a status bar with the time 13:01 and various icons. The app logo, which consists of a house icon with a person inside and the text 'AI 底家', is centered. Below the logo, there are two input fields: 'Phone Number' with a telephone icon and 'Password' with a lock icon. Both fields have placeholder text '輸入電話號碼' and '輸入密碼' respectively. Below the password field, there is a link for '帳號註冊' (Account Registration) with a person icon. At the bottom, there is a large black button with the white text '登入' (Login).

使用者尚未登入前，App 頁面有登入及註冊選項。登入需輸入手機號碼以及密碼，按下登入後即可進入 App。

▲圖 12-3 登入頁面

註冊頁面



A mobile app registration form titled "帳號註冊" (Account Registration). At the top, there is a status bar with the time "13:01" and various icons. Below the status bar is a navigation bar with a back arrow and the text "返回" (Return). The form itself is a white card with rounded corners. It contains the following fields: "姓名" (Name) with a text input field labeled "請輸入姓名"; "性別" (Gender) with two radio buttons labeled "女性" (Female) and "男性" (Male); "手機號碼" (Mobile Number) with a text input field labeled "請輸入號碼"; "密碼" (Password) with a text input field labeled "請輸入密碼"; and "確認密碼" (Confirm Password) with a text input field labeled "再次輸入密碼". At the bottom of the card is a dark blue button labeled "註冊" (Register).

使用者尚未登入前，需點選登入頁面中的帳號註冊選項，進入後，需輸入姓名、性別、手機號碼、密碼及確認密碼之後，即可註冊帳號。

▲圖 12-4 註冊頁面

首頁頁面



使用者登入後會直接進入首頁，所有功能都在首頁中即可點擊操作。(狀態報告功能會在複審時呈現)

▲圖 12-5 登入頁面

個人資料頁面

▲圖 12-6 編輯頁面

▲圖 12-7 Linebot

使用者進入頁面之後，按下編輯按鈕即可更改基本資料。使用者點擊下方加入 Line 官方帳號後即可設定推播消息；點擊 Facebook 授權認證後即可綁定 FB 帳號以推播子女消息給長輩。



▲圖 12-8 FB 授權

個人資料頁面 (更改密碼、刪除帳號)



▲圖 12-9 修改密碼



▲圖 12-10 刪除密碼

使用者可於個人頁面進入左圖修改密碼，輸入舊密碼、新密碼及確認新密碼後，按下確認修改，即跳回登入頁面重新登入；點擊下方刪除帳號，會跳出視窗再次確認是否刪除帳號，按下確定後即跳到登入頁面。

選擇長輩畫面



▲圖 12-11 選擇被照護者

使用者於收首頁上方下拉式選單即可選擇要查看的長輩(除目前偵測畫面是一對一)。

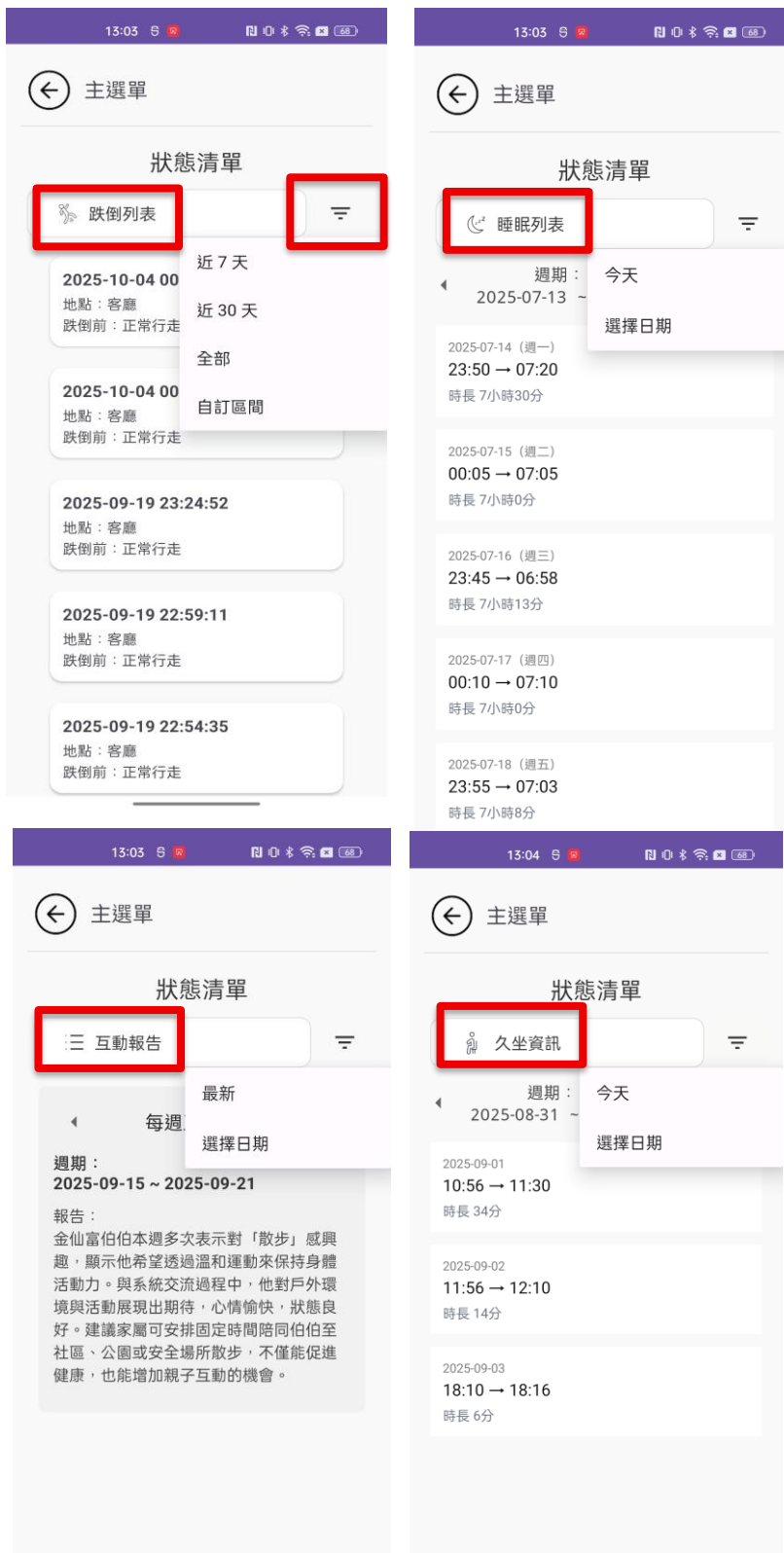
狀態清單頁面



使用者於首頁下方點擊放大鏡圖示即可進入狀態清單，有跌倒列表、睡眠列表、互動報告及久坐資訊供使用者查看。

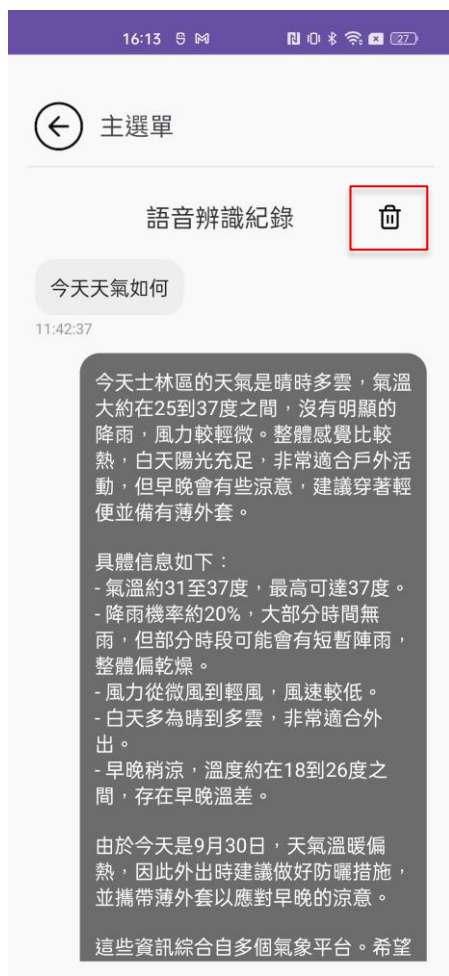
▲圖 12-12 狀態清單

狀態清單頁面(跌倒、睡眠、互動、久坐)



使用者可於狀態清單中選取查看細項，選擇條件皆包含天/週做選擇，要換成週列表可於右方按鈕做切換。

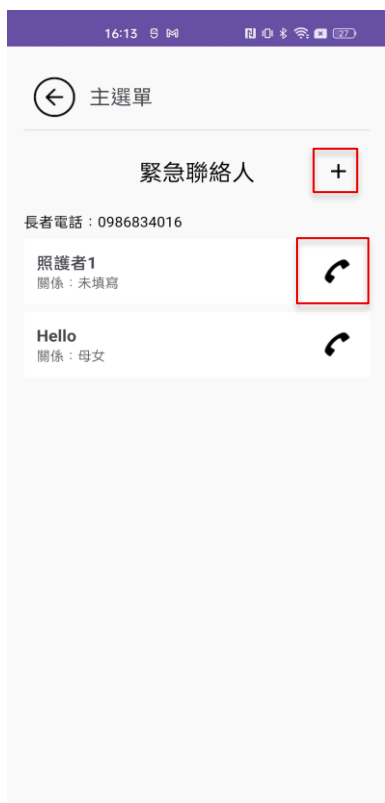
▲圖 12-13 狀態清單頁面(跌倒、睡眠、互動、久坐)



使用者於下方點選聊天圖示即可進入此頁面，可以查看長輩與系統的對話，右上角有刪除按鍵可清除紀錄。

▲圖 12-14 語音互動紀錄

緊急聯絡人頁面



▲圖 12-15 語音互動紀錄



▲圖 12-16 新增緊急聯絡人

使用者於首頁點及下方電話圖示即可進入頁面，按左圖的加號可加入聯絡人，並輸入電話及關係之後，按下新增即可出現在列表，要刪除聯絡人則在列表往左滑動即可刪除，要撥打則直接按下右邊電話圖示即可跳轉到撥號頁面。

第13章 感想

➤ 11146005 沈宛宣

從一開始的找指導老師，我們一個一個老師去問，到最後很榮幸能與蒯思齊老師合作，老師總是能一步步地帶領我們，並且給我們很多的資源。在專題中我是負責文件的同學及組長的角色，原先對自己在組內的定位感到很模糊，覺得自己既不負責前端也不負責後端，但老師的一句話：「在小組中，寫文件的同學也很重要」，這句話打醒了我。剛開始寫文件，以為是像在寫故事那樣，過多的壟言贅字，但老師一步步糾正，告訴我寫文件的技巧，精簡文字、善用圖表、抓住評審的重點與視線。這些都是過去沒有接觸過的能力；身為組長的我也需要了解整體進度進行到哪邊，要瞭解前端的設計理念，後端模型開發進度，目前的資料庫設計如何，覺得自己每周都在學習新的東西。謝謝負責各領域的同學能夠有耐心地與我講解他們各自負責領域的專業知識，讓我更加了解，並將這些東西寫進文件中。在這一年的專題實作中，好幾次都因為對專業知識的不夠了解，導致想放棄，但想到其他組員也依舊努力著，老師也很用心地指導我們，我就會重新打起精神，提醒自己要堅持到底。

➤ 11146012 邱鈺婷

在最先開始得知要開始做專題時，緊張的心情不禁湧了上來，從最先開始的分組、找指導老師，感覺每分每秒都是一場爭奪戰，到後來的訂定主題、開始製作，都是一場值得挑戰的關卡，專題並不是像平常上課所學的東西，多練習幾遍就會，而是要不斷的從嘗試到失敗最後再站起來，摸著不熟悉的程式碼，密密麻麻的字要想辦法理解，這段期間心情也跟著起起伏伏，但幸好有組員的幫忙以及蒯思齊老師的指導下，讓我對這份挑戰越來越有信心，也相信自己能做出一個從來沒做過的專案。在專題中我是擔任前端的角色，說難也不是最難，說簡單倒是蠻不簡單的，從一開始的版面設計，需要不斷去網路上參考他人的作品，以及觀察我們現在最常用的 APP 所規劃的版面，到後來的串接 API，過程中困難重重，看著失敗訊息一直跳出，又想不到原因時心態層層被搞砸，雖然說挫折感有些重，但在成功的那一刻成就感又讓我覺得離成功更進一步了，或許就是這份感覺才讓我有動力繼續做下去吧！

➤ 11146018 趙明威

這次專題的主題是「老人照護 APP」。我負責後端資料庫 API 與姿態判斷處理 API，也就是幾乎涵蓋大部分伺服器程式設計的部分。後端採 DAO／Service／Route 分層，讓各模組的職責更清楚、維護也更容易。

姿態判斷的資料傳輸採用 WS (WebSocket) 雙向溝通，主要是利用其低延遲達到即時性。伺服器在接收資料後，會先做前處理（骨架平滑、補幀、遮罩），接著以 relation maps 的方式把骨架資訊轉成 CNN 可理解的空間特徵，再由 CNN+LSTM：先讓 CNN 判斷整體空間特徵，再由 LSTM 捕捉時序關係。判斷流程是二階段：先用二分類模型判斷是否為跌倒；若非跌倒，再用多分類細分動作。這樣能有效降低訓練難度。同時，在輸出端我也加入「連續觸發」與「恢復門檻」的策略，降低結果抖動與誤判。

最具挑戰的是蒐集與處理訓練資料。我需要先找到影片，再自行產生骨架與相關資訊。影片型資料在公開資料集中其實不容易找（例如 Kaggle 多被歸到 other 類別）。就算找到了，還得確認影片是否為單人；雖然 YOLO 能抓多人的骨架與框，但我目前的前處理尚未把多人分離並追蹤成「單一人序列」，因此很多資料集因此被排除。之後還要檢查長度是否足夠；太短的片段模型學不到東西只能放棄。最後還有類別不均的問題：二分類理論上可容忍一定不平衡，但當我把所有「非跌倒」都丟進去後，跌倒樣本比例過低，模型一開始幾乎學不起來。也因為這些過程，我對資料品質、取樣策略與訓練行為的關係有更深刻的理解。

行動端也遇到實務差異。專用手機上跑得順，但換成一般手機時，模型推論變慢，導致無法以預期的 FPS 傳資料給伺服器。為此，前端我降低取幀頻率；伺服器端則強化低幀率情境下的補幀與平滑，穩住整體效果。

總而言之，這次專題讓我真正理解了模型訓練與部署的全流程，也體會了「理論」與「實作」之間的落差—很多設計要在真實環境中不斷調整。專題的成功仰賴團隊成員的專長與互補；透過持續溝通與協作，我們達成預期成果。感謝蒯思齊老師與組員們的指導與幫助，這段經歷帶給我豐富的知識與寶貴的實戰經驗。

➤ 11146033 陳欣怡

剛開始我主要負責專題的「資料庫」部分。透過這次實作，我對資料庫的運行與操作有了更深入的理解。建置時不僅需要先規劃可能使用的資料表，還要思考其邏輯架構，以及主鍵、外鍵的設計方式。這些細節在課堂上未必能學到，但在實務操作中卻非常重要。雖然過程中仍會懷疑設計是否完善，但我也體會到資料庫並沒有絕對固定的設計方式，而是需要依照需求彈性調整。

專題初期因各自作業，資料表設計一度遭到質疑，再加上當時能應用到的資料表有限，讓我對自己的角色感到不太確定。不過，從暑假開始，我除了資料庫外，也接手了 n8n 的部分。一開始對此十分陌生，需要花許多時間摸索設定與流程設計的方法。但在每週的進度報告推動下，我漸漸克服困難，也成功完成了功能實作。

最後，我要特別感謝指導老師，即使行程繁忙，他仍會定期關心我們的進度，並慷慨提供資源，引導我們找到解決問題的方向，也常常給予鼓勵與信心。同時也感謝組員們，能在過程中互相協助、共同承擔責任，沒有紛爭，每個人都付出了努力。這次專題不僅讓我增進了實作上的能力，更讓我深刻體會到團隊合作的不易。能有今天的成果，真的要感謝老師與組員們的支持與付出。

➤ 11146039 金峻瑋

在專題最一開始時，大家都很迷茫，提出了很多想法，像是跟交通有關的假日車流管理、跟 Ubike 有關的租借系統、跟銀髮族相關的照護。經過了幾個禮拜的討論後，我們最後選擇了高齡化趨勢帶來的銀髮族照護。初評前，我負責的部分較雜，有一部分牽扯到 line 通知，有一部分是語音助手。但是因為當時我還沒有接觸 n8n 這個非常好用的自動化工作流工具，所以我在寫伺服器程式時顯得比較吃力，因為我個人程式能力沒有到很好。

初評過後，老師讓我負責「n8n 決策流+linebot」的部分，因此，我暑假花了好幾天把 n8n 搞懂，並嘗試實作了各種不同的語音流、不同語言模型下的 AI agent，最後做出了分流器、語音助手、骨架分析這些工作流。在接觸 n8n 時，我也接觸到了 preplexity、elevenlabs 等 AI 工具，也不經感嘆才短短 2 年，AI 的發展已經如此迅速。

我很高興能夠被蒯思齊老師指導，他不會因為我們寫程式的能力差而給我們壓力，而是一步步地帶著我們，從選題目、資料準備、實作，直到完成。我也很高興能跟善於溝通的隊友們一同奮鬥，大家都各司其職，一步步的將整個專案聚沙成塔，最後一同享受勝利的果實。雖然過程很艱難，但是有老師以及同學們的共同努力，我從這次專題中學到了許多。

第14章 參考資料

1. n8n 手把手完整教學：https://www.youtube.com/watch?v=sYWCxgEF_yY
2. n8n 官方文件：<https://docs.n8n.io/>
3. n8n Sub-Workflow 教學：<https://lazyneverland.com/n8n-sub-workflow-execute-workflow-tutorial/>
4. MySQL 資料庫全攻略：<https://www.youtube.com/watch?v=3zzszKQ8Kk4>
5. 輕鬆學會 Android Kotlin 實作開發精心設計 24 個 Lab 讓你快速上手第三版-博碩
6. The ULTIMATE n8n Agentic RAG System <https://youtu.be/BTxghC1qHbw>
7. Turn Your AI Agent Into a Voice Assistant in Minutes
<https://www.youtube.com/watch?v=qJRFu88HUio&t=989s>
8. 使用 LINE Bot 聊天機器人替代 LINE Notify 服務
<https://hackmd.io/@flagmaker/SJLKW4lV1e>
<https://hackmd.io/@chrish0729/H1bzDiWCn>
9. Multiple Cameras Fall Dataset :
<https://www.kaggle.com/datasets/soumicksarker/multiple-cameras-fall-dataset/data>
10. Human Activity Recognition (HAR - Video Dataset):
<https://www.kaggle.com/datasets/sharjeelmazhar/human-activity-recognition-video-dataset/data>
11. MSRDailyActivity3D (RGB Videos only):
<https://www.kaggle.com/datasets/mdmofazzalhossain789/msrdailyactivity3d-rgb-videos-only?select=lie+down+on+sofa>

12. HumanEva Dataset:
http://humaneva.is.tue.mpg.de/datasets_human_1
13. Fall Detection Dataset:
https://search-data.ubfc.fr/FR-13002091000019-2024-04-09_Fall-Detection-Dataset.html

附錄 會議記錄

第 4 次專題會議紀錄(主題討論)

一、基本資訊

會議日期：2025 年 2 月 20 日

會議時間：16:15 - 18:05

會議地點：北商大資研討室 1

參與人員：指導老師、學生(全到)

指導老師：蒯思齊

組長：沈宛宣

組員：邱鈺婷、趙明威、陳欣怡、金峻瑋



1. 專案進度回報

- 沈宛宣：

(針對銀髮族智慧動作辨識技術去說明可以運用在我們專題上的可行性，與市面上不同的是此技術運用，無須搭配配戴裝置等，只須結合家中監視器，即可做到長輩動作智能監測，預防跌倒，即危險動作發生、偵測到跌倒時，做及時的通知。)

- 邱鈺婷：

(針對(配戴智能手錶偵測使用者當下身心狀態是否適合工作)分析如何獲利及商業模式，可提供訂閱服務，那身心狀態評估，如何去衡量；老師認為大家要培養一下，包含你設計方法，1 個流程或 1 個方法，要真正能夠解決到某個問題的話，你必須要針對他的特性去設計，方法必須針對問題特性去回答。)

- 趙明威：

(切換主攻能，提出影片中有不同主題，如何讓使用者輸入影片，辨識影片人聲說話語調，去判斷該段適合放甚麼字體、特效、音效；老師認為訓練成本高，資料量需求非常大，但容易與剪映相疊)

- 陳欣怡：

(針對上周提出的 youbike 尖峰時段沒車，數據方式調整調度去回答。找到原始可使用的資料，但資料很可惜，沒有腳踏車租借、還的時間，及商業模式如何定義，需要交互評估，像是台北市建 UBIKE 來補充路網，重點不是在於 UBKE 本身能賺多少錢，而是 UBIKE 的推廣能因此來節省台北市政府補貼多少公車的錢，或是能節省多少交通阻塞帶來的成本。)

- 金峻瑋：

(針對上周提出工作狀態偵測分析，在甚麼情況下能判斷該員工是否在偷懶，維持一個動作不動嗎？若是在課堂上，面部表情的判斷是否更重要，由以上兩點運用動作偵測的技術，可與銀髮族智慧動作偵測結合，商業模式更明確。)

2. 問題討論與解決

- 問題 1：銀髮族智慧動作偵測，用於甚麼場所

討論內容：計畫規劃於家中使用，小範圍的應用在客廳、臥室等

0714 第 19 次專題會議記錄(線上會議)

日期：2025 年 7 月 14 日

參與人員：蒯思齊老師、趙明威、沈宛宣、金峻瑋、陳欣怡

缺席人員：邱鈺婷（已錄製影片報告）

專案進度報告

邱鈺婷

播放預錄影片，說明兩項討論：

子女貼文如何挑選並播放給長輩聽：清晨先爬取子女社群貼文，隔天早上播放。GPT 提出三種方法：人工清單審核、NLP 分析、關鍵字排除。老師建議改用語言模型判斷語意，先抓貼文整理 JSON，再丟 GPT 判斷。

單獨處理長輩聲音：語音喚醒（類似小助手）及聲紋辨識（需訓練樣本，成本較高）。老師補充語音特徵提取比以往容易。

沈宛宣

流程架構圖討論：依 GPT 協助分為影像處理輸入、工作流程協調、外部服務、最終互動結果、輔助資料來源。老師提醒應將 Edge 邊緣端獨立框住，系統分為 Edge / Server / Cloud 三層。畫圖需有程式思維，明確標示輸入輸出。

趙明威

1. 已將 YOLO 產出的骨架與框架資料丟給 LSTM 學習。
2. 討論 YOLOv8 POS 與 DETECT 模型差異，家具偵測需額外啟用。
3. 下一步計劃：繼續將骨架、家具框、人物框同步輸出，手機端用 Tiny 模型執行，透過 WebSocket 傳給伺服器。
4. 老師提供語意分割（如 SAM）可作未來優化方向，目前先完成 YOLO 方案。

陳欣怡

1. 已將資料庫連到 N8N，用 View 撈取最近 7 天資料並做簡易統計，透過 Schedule Trigger 定時執行。
2. 老師建議：將統計結果丟給 OLAMA/GPT，自動生成報告存回資料庫，前端直接讀取顯示，形成閉環。

金峻瑋

1. 用 N8N 測試聊天機器人，設計 Switch Node 區分「天氣」「新聞」「未知」。
2. 目前使用爬蟲抓 ETtoday，新版將改用 Google News API。
3. 老師提醒：Prompt 需給多同義例句，強化 NLP 判斷；本地模型需明確指示避免誤解。
4. 下一步計劃：串接 Eleven Labs 做語音轉文字與 TTS，完整示範手機播放語音。

會議結尾

老師總結：大家進展不錯，已與 GPT / OLAMA 串接，接下來繼續保持並完成閉環整合。

0909 第 26 次專題會議記錄

日期：2025 年 9 月 9 日

參與人員：全體同學、蒯思齊老師

討論重點

1. 系統功能與應用方向

- 單純偵測跌倒並發送警報太單一，建議延伸至「提升長者生活品質」的應用。
- Demo 測試：目前已能模擬手機端傳送 pose 資料並送入伺服器，LSTM 模型需滿足 20 幀輸入，每次滑動 5 幀進行判斷。
- 跌倒判斷結果將儲存至資料庫，後續可結合其他模型進行擴充。
- 人的框包含框位置及骨架點資訊，後續可用於更精確的動作判斷。

2. 語音互動與前端串接

- 問題：若直接用前端的 STT（語音轉文字），問題可能太模糊（例如「今天天氣如何？」），導致模型難以正確回答。
- 建議解法：
 - 在 Prompt 中補足使用者的基本資料（如年齡、居住地、子女資訊等）。
 - 由大語言模型協助「改寫問題」為更精確的形式（例：自動改成「請問台北市今天天氣如何？」）。
- 問題分類為 ASK / ANSWER：
 - ANSWER：可直接回答的問題 → 傳給模型處理。
 - ASK：需要補充資訊 → 回傳前端，由前端引導長者提供細節。

3. 系統資料設計與互動

- 註冊時的基本資料應寫入，並在對話 prompt 中使用，以避免回答不合邏輯（如誤判子女數量）。
- 歷史、文化等非個人化問題，則交由大語言模型自由回答。

4. 系統整合與應用路線

- 可選擇 App 呈現 或 LINE Bot 作為主要互動介面。
- LINE Bot 更貼近照護人員與家屬的使用習慣。
- 功能可透過關鍵字觸發（如「報表」、「分析報告」）來自動回傳定期生成的分析結果。

5. 系統強調的三大重點

1) 模型在手機端執行：

- 不需要昂貴設備與大量算力。
- 強調「隱私」：影像不需全程上傳，避免用戶生活被監視。

2) 模型推論邏輯：

- 單純知道動作不足以判斷狀況（例如：躺著可能是躺床或摔倒）。
- 必須結合環境因素（人物與物件之間的相對關係）來辨識危險情境。

3) N8N 自動化串接：

- 系統牽涉多種 AI 技術，需透過 N8N 串聯。
- 建議在架構圖中以淺色底框標註 N8N 負責的部分，並加上 logo 強調其價值。

6. 其他事項

- 比賽報名：需注意 9/15 繳交，9/19 公布入選名單。
- 若校內篩選未通過，可考慮改報「AI 工具組」或「產業創新組」。
- 投影片需開始製作，並針對 3 大重點進行強化。
- 預計 9 月中或下旬需完成可現場跑完流程的版本，以利檢視缺漏功能。

結論與待辦事項

1. 前端需與後端進一步確認資料串接方式（STT 與 API 流程）。

2. Demo 版本須於 9 月中完成，並能呈現從前端到後端的完整流程。
3. 確認參賽報名時程，並完成必要文件。
4. 規劃系統 prompt 機制，整合長者基本資料以提升語音互動效果。
5. 強化投影片內容，凸顯手機端運行、隱私保護、環境判斷邏輯與 N8N 串接三大賣點。
6. 9 月下旬前需能完整演示全流程，釐清必要與可刪減功能。

附錄 初評之評審建議與意見

評審建議	修正情況
語音互動如何克服鏡頭排斥感(有數據嗎)，沒有人喜歡被監控。場景可能比較適合幼兒園或是養老院。	本系統主要採「主動互動」設計，語音互動僅在長者主動開啟或呼喚系統時啟用，平時不進行錄音或監控，避免隱私與排斥感問題。鏡頭僅用於行為安全偵測，不會進行臉部辨識。場域設計以居家環境為主，並設有明顯提示燈號顯示工作狀態。

評審建議	修正情況
與系統即時語音對話可以做到那些部分，語音是雙向的嗎，LIKE 寵物，雙向溝通。	系統支援雙向語音互動：長者可主動以語音呼叫（如「我想聽新聞」、「幫今天天氣如何」），系統可語音回覆並提供即時資訊或播報，達到雙向溝通效果。

評審建議	修正情況
功能設定上的易操作、老人家會用嗎？	系統使用者設定為照護者，老人家不會實際操作到 APP。

評審建議	修正情況
法規問題，免責說明部分。	系統遵循個資法與影像處理規範，僅記錄姿態資訊（骨架資料），不儲存臉部影像與音訊內容。於安裝及註冊時會顯示「使用者授權與免責條款」，明確說明資料用途與保存期限。

評審建議	修正情況
社交貼文資料從哪邊來	社交內容來自被照護者家屬授權的 Facebook 貼文；新聞與生活資訊則由系統每日自動擷取公開新聞來源（Yahoo 新聞），經 AI 摘要後播報給長者。

評審建議	修正情況
宜講明適用的建置情境	於長者居家場域中架設鏡頭，偵測日常行為（如坐下、走動、跌倒）並即時回報給照護者端 App，同時進行語音互動與新聞播報測試，驗證操作流程與辨識準確性。

評審建議	修正情況
計算老人活動量	活動量分析依據每日姿態變化與久坐時長計算，系統會記錄長者的睡眠、久坐，透過 AI 模型自動估算活動量指數。

評審建議	修正情況
鏡頭的廣角，鏡頭可以看到的幅度多少。 遠近、清晰度	我們使用家中的舊手機一般具備 720p 或 1080p 高清解析度，且它們的廣角範圍通常可達 70-80 度，足夠覆蓋大部分室內場景。這雖然不及最新型號的智能手機，但仍能提供足夠的影像清晰度來支援基本的行為監控與跌倒偵測功能。對於日常監控，這些解析度已能有效捕捉長者的動作細節，並進行有效的後處理分析。

附錄 複評之評審建議與意見

評審建議	修正情況
專注跌倒邏輯，相近的蹲下、彎腰撿東西、做運動等，能否仍能準確判斷出跌倒。	系統中動作的模型訓練都是後端程式負責同學自行訓練出來的，目前相近動作判斷，模型還處於訓練階段
評審建議	修正情況
夜間搭配紅外線攝影機	未來在實際場域部署時，可依環境需求選用具紅外線補光功能的攝影設備，以確保夜間仍能維持穩定偵測效果。
評審建議	修正情況
多台攝影機之間怎麼辦	在實際應用上，可依空間配置（如客廳、走廊、房間）部署多台攝影機，並由後端統一接收分析結果。 未來可進一步結合時間與空間資訊，避免同一事件在不同攝影機中被重複判定，提升事件整合與回報的準確性。
評審建議	修正情況
題目是否調整為跌倒 AI 為主？	主要以「陪伴」為主，還是希望透過科技減少長者的孤獨感，同時提升家屬的安心感。
評審建議	修正情況
各跌倒種類的樣態及模型訓練及實際展示驗證偵測有效性才能收費	未來會更確保系統在陪伴與安全上的實際可行性，本專題已透過實際影像進行跌倒情境的推論展示，部分流程能被清楚觀察與理解。